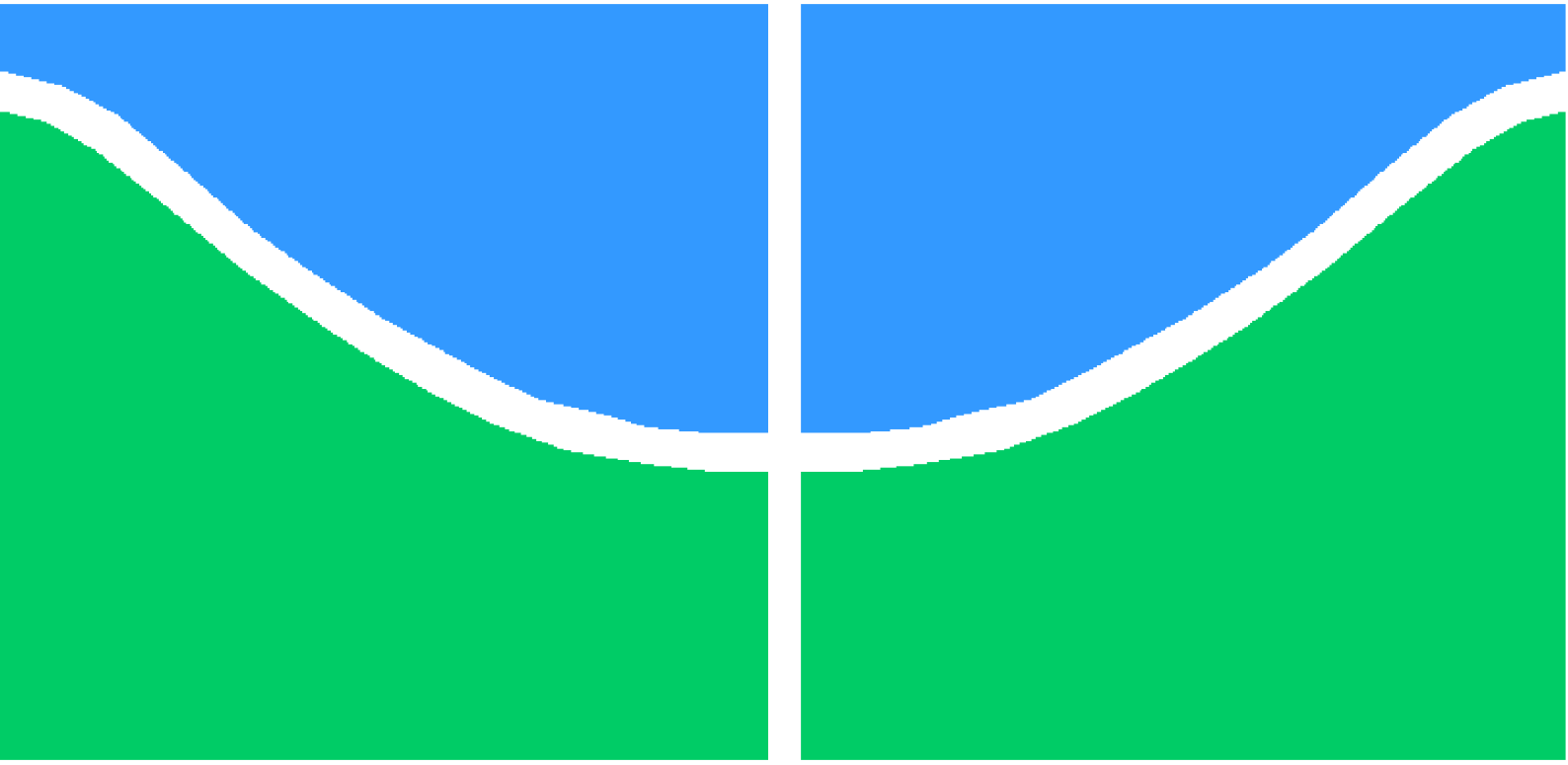


Universidade de Brasília - UnB

Faculdade de Ciências e Tecnologias em Engenharia - FCTE

Engenharia de Software

Seed: aplicativo de promoção da saúde com base na coleta de dados de hábitos de vida e estados de saúde mental em instituições acadêmicas

Autor: Gabriela de Oliveira Lemos e João Antonio Ginuino Carvalho

Orientador: Professor Dr. Wander Cleber Pereira

Brasília, DF

2025



Gabriela de Oliveira Lemos e João Antonio Ginuino Carvalho

Seed: aplicativo de promoção da saúde com base na coleta de dados de hábitos de vida e estados de saúde mental em instituições acadêmicas

Monografia submetida ao curso de graduação em (Engenharia de Software) da Universidade de Brasília, como requisito parcial para obtenção do Título de Bacharel em (Engenharia de Software).

Universidade de Brasília - UnB

Faculdade de Ciências e Tecnologias em Engenharia - FCTE

Orientador: Professor Dr. Wander Cleber Pereira

Coorientador: Professora Dra. Larissa Polejack Brambatti

Brasília, DF

2025

Gabriela de Oliveira Lemos e João Antonio Ginuino Carvalho
Seed: aplicativo de promoção da saúde com base na coleta de dados de hábitos de vida e estados de saúde mental em instituições acadêmicas/ Gabriela de Oliveira Lemos e João Antonio Ginuino Carvalho. – Brasília, DF, 2025-
104 p. : il. (algumas color.) ; 30 cm.

Orientador: Professor Dr. Wander Cleber Pereira

Trabalho de Conclusão de Curso – Universidade de Brasília - UnB
Faculdade de Ciências e Tecnologias em Engenharia - FCTE , 2025.

1. hábitos saudáveis. 2. monitoramento de hábitos. I. Professor Dr. Wander Cleber Pereira. II. Universidade de Brasília. III. Faculdade UnB Gama. IV. Seed: aplicativo de promoção da saúde com base na coleta de dados de hábitos de vida e estados de saúde mental em instituições acadêmicas

CDU 02:141:005.6

Gabriela de Oliveira Lemos e João Antonio Ginuino Carvalho

Seed: aplicativo de promoção da saúde com base na coleta de dados de hábitos de vida e estados de saúde mental em instituições acadêmicas

Monografia submetida ao curso de graduação em (Engenharia de Software) da Universidade de Brasília, como requisito parcial para obtenção do Título de Bacharel em (Engenharia de Software).

Trabalho aprovado. Brasília, DF, 20 de julho de 2025:

Professor Dr. Wander Cleber Pereira
Orientador

Professor Dr. Henrique Gomes de Moura
Convidado 1

Prof. M. Sc. Flávio Feitosa Costa
Convidado 2

Brasília, DF
2025

Agradecimentos

Primeiramente, gostaria de agradecer aos sábios, mestres e doutores que tiveram a elevação espiritual para estender a mão e ensinar quem ainda não era ninguém. Registro aqui meu total respeito e a esperança de, um dia, poder ajudar alguém da mesma forma que fizeram comigo. Agradeço também aos demais que, mesmo dificultando a jornada mais do que o necessário, me ensinaram que não há limites que não possam ser superados.

Por fim, à minha família e aos meus amigos, deixo minha mais profunda gratidão pelo apoio incondicional e pelo carinho que me sustentaram ao longo de toda a jornada acadêmica. Nenhuma conquista minha faz sentido sem vocês por trás dela.

Resumo

O ambiente universitário contemporâneo tem apresentado um aumento significativo nos casos relacionados a problemas de saúde entre estudantes, incluindo questões de saúde mental, sedentarismo, estresse acadêmico e hábitos prejudiciais ao bem-estar. Esses fatores impactam diretamente o desempenho estudantil e evidenciam a necessidade de mecanismos eficazes de monitoramento e intervenção. Diante disso, este trabalho tem como objetivo o desenvolvimento de um aplicativo *mobile* voltado à coleta e análise de dados sobre o humor, hábitos e saúde geral dentro de instituições de ensino superior (IES). A proposta busca auxiliar o direcionamento de políticas institucionais de promoção à saúde, além de oferecer aos estudantes meios de buscar ajuda dentro da IES. A metodologia adotada envolve revisão bibliográfica e documental sobre o uso de tecnologias em saúde, análises de aplicativos semelhantes e aplicação de questionários à comunidade acadêmica, a fim de levantar os requisitos funcionais e não funcionais da aplicação. Sendo assim, na próxima fase, será iniciado o desenvolvimento do software, como implementação de funcionalidades, a fim de desenvolver um meio para auxiliar alunos na adoção de hábitos saudáveis e promover a saúde mental no meio acadêmico.

Palavras-chaves: Aplicativo *Mobile*, Hábitos Saudáveis, Saúde Mental.

Abstract

The contemporary university environment has seen a significant increase in cases related to health problems among students, including mental health issues, sedentary lifestyles, academic stress, and habits that are detrimental to well-being. These factors directly impact student performance and highlight the need for effective monitoring and intervention mechanisms. Therefore, this work aims to develop a mobile application aimed at collecting and analyzing data on mood, habits, and general health within higher education institutions (HEIs). The proposal aims to help guide institutional health promotion policies and provide students with ways to seek help within the HEI. The methodology adopted involves a literature and document review on the use of health technologies, analysis of similar applications, and the administration of questionnaires to the academic community to identify the application's functional and non-functional requirements. Therefore, the next phase will begin software development, including the implementation of features, to develop a means to assist students in adopting healthy habits and promoting mental health within the academic environment.

Key-words: Mobile application, Healthy Habits, Mental Health.

Lista de ilustrações

Figura 1 – Tela inicial do Habitica . . . 29
Figura 2 – Telas do Daylio . . . 30
Figura 3 – Raia do TCC1 do BPMN do Processo Metodológico. . . . 37
Figura 4 – Raia do TCC2 do BPMN do Processo Metodológico. . . . 37
Figura 5 – BPMN do Processo Metodológico . . . 38
Figura 6 – Ciclo do Test-Driven Development. . . . 41
Figura 7 – BPMN Sprint. . . . 42
Figura 8 – BPMN Daily. . . . 43
Figura 9 – BPMN Kanban. . . . 44
Figura 10 – Canvas MVP (Seed). . . . 46
Figura 11 – Matriz de correlação do questionário. . . . 49
Figura 12 – Representação arquitetural. . . . 58
Figura 13 – Diagrama de Classe do projeto. . . . 61
Figura 14 – Diagrama de Pacotes do frontend. . . . 62
Figura 15 – Diagrama de Pacotes do backend. . . . 63
Figura 16 – Diagrama Entidade-Relacionamento do Projeto. . . . 64
Figura 17 – Diagrama Lógico do Projeto. . . . 65
Figura 18 – *Brainstorming* de Funcionalidades. . . . 94
Figura 19 – Classificação de funcionalidades - Lean Inception. . . . 94
Figura 20 – Revisão das funcionalidades propostas (Seed). . . . 95
Figura 21 – Sequenciador (Seed). . . . 96
Figura 22 – Canvas MVP (Seed). . . . 97
Figura 23 – Número de Respostas e Tempo Médio. . . . 98
Figura 24 – Questão de Público Alvo. . . . 98
Figura 25 – Questão de permanência na comunidade universitária. . . . 99
Figura 26 – Questão de Campus universitário. . . . 99
Figura 27 – Questões de Autoavaliação de hábitos. . . . 100
Figura 28 – Questões de Autoavaliação de frequência de hábitos. . . . 100
Figura 29 – Questões de Autoavaliação da saúde. . . . 101
Figura 30 – Questões de demais tópicos do bem-estar. . . . 101
Figura 31 – Respostas de demais tópicos do bem-estar. . . . 101
Figura 32 – Questão de serviços de apoio a saúde nas instituições. . . . 102
Figura 33 – Questão de aceitação de sistemas com foco em saúde. . . . 102
Figura 34 – Questão sobre funcionalidades do sistema. . . . 102
Figura 35 – Respostas sobre funcionalidades do sistema. . . . 103
Figura 36 – Comentários adicionais. . . . 104

Figura 37 – Respostas nos comentários adicionais. 104

Lista de tabelas

Tabela 1 – Cronograma das Atividades do TCC1 (2025). 36
Tabela 2 – Requisitos Funcionais 54
Tabela 3 – Requisitos Não Funcionais Classificados Pelo FURPS+. 56
Tabela 4 – Recursos Tecnológicos 57
Tabela 5 – Skills propostas para o processo de desenvolvimento. 71
Tabela 6 – Hooks propostos para o processo de desenvolvimento. 72
Tabela 7 – Comparação entre os subagentes do fluxo proposto 77

Lista de abreviaturas e siglas

APIs RESTful	*Application Programming Interfaces Representational State Transfer*
DAO	*Data Access Object*
DAST	*Dynamic Application Security Testing*
DSS	Determinantes Sociais de Saúde
IA	Inteligência Artificial
IBGE	Instituto Brasileiro de Geografia e Estatística
IDE	*Integrated Development Environment*
IES	Instituições de Ensino Superior
IOS	*iPhone Operating System*
JSON	*JavaScript Object Notation*
JWT	*JSON Web Token*
LGPD	Lei Geral de Proteção
LLM	*Large Language Models*
MVP	*Minimum Viable Product*
ORM	*Object Relational Mapping*
POO	Programação Orientada a Objeto
SAST	*Static Application Security Testing*
SDD	*Spec-Driven Development*
SGBDR	Sistema Gerenciador de Banco de Dados Relacional
TCC	Trabalho de Conclusão de Curso
TSL	*Transport Layer Securit*
UML	*Unified Modeling Language*
UnB	Universidade de Brasília

UPS	Universidades Promotoras de Saúde
UX	*User Experience*
XP	*Extreme Programming*

Lista de símbolos

α	Coeficiente de confiabilidade (Alpha de Cronbach)
N	Número de itens no conjunto de perguntas ou variáveis
$\sigma^2_{Y_i}$	Variância do i-ésimo item
σ^2_X	Variância total do somatório dos itens
$\sum$	Somatório

Sumário

1 INTRODUÇÃO . . . 23
1.1 Contextualização . . . 23
1.2 Justificativa . . . 24
1.3 Questão de Pesquisa . . . 25
1.4 Objetivos . . . 25
1.4.1 Objetivo Geral . . . 25
1.4.2 Objetivos Específicos . . . 25
1.5 Organização do Trabalho . . . 26

2 REFERENCIAL TEÓRICO . . . 27
2.1 Saúde e Bem-estar . . . 27
2.1.1 Estilo de Vida e Ambiente . . . 28
2.1.2 Saúde Mental . . . 28
2.2 Plataformas Existentes . . . 29

3 METODOLOGIA . . . 32
3.1 Heurísticas de Nielsen . . . 32
3.2 Escala de Likert . . . 33
3.3 Alpha de Cronbach . . . 33
3.4 Metodologia de pesquisa . . . 34
3.4.1 Natureza de Pesquisa . . . 35
3.4.2 Abordagem de Pesquisa . . . 35
3.4.3 Objetivo de Pesquisa . . . 35
3.4.4 Procedimento de Pesquisa . . . 35
3.4.5 Processo Metodológico . . . 36
3.4.5.1 TCC1 . . . 36
3.4.5.2 TCC2 . . . 37
3.5 Metodologia de Desenvolvimento . . . 38
3.5.1 Metodologia Ágil . . . 39
3.5.2 Uso de Inteligência Artificial . . . 39
3.5.2.1 Grilling . . . 39
3.5.2.2 Spec-Driven Development . . . 40
3.5.3 Extreme Programming (XP) . . . 40
3.5.3.1 Test-Driven Development (TDD) . . . 41
3.5.4 Scrum . . . 42
3.5.5 Kanban . . . 43

3.5.6 Lean Inception . . . 44
3.5.7 Questionário . . . 46
3.5.7.1 Análise de Confiabilidade . . . 47
3.6 Requisitos . . . 49
3.6.1 Requisitos Funcionais . . . 50
3.6.1.1 Histórias de Usuário . . . 50
3.6.1.1.1 Épico - Gerenciamento de registro do usuário. . . . 51
3.6.1.1.2 Épico - Gerenciamento de formulários. . . . 51
3.6.1.1.3 Épico - Gerenciamento de comunicação. . . . 52
3.6.1.1.4 Épico - Gerenciamento de eventos. . . . 53
3.6.1.1.5 Épico - Gerenciamento do feed. . . . 53
3.6.2 Requisitos Não Funcionais. . . . 55
3.6.2.1 FURPS+ . . . 55
3.7 Suporte Tecnológico . . . 56
3.8 Arquitetura . . . 57
3.8.1 MVC (Model-View-Controller) . . . 58
3.8.2 Backend . . . 59
3.8.2.1 API RESTful . . . 59
3.8.2.2 Hibernate . . . 59
3.8.3 Frontend . . . 60
3.8.3.1 React Native . . . 60
3.9 Representação Visual do Sistema . . . 60
3.9.1 Diagrama de Classes . . . 60
3.9.2 Diagrama de Pacotes . . . 61
3.10 Projeto de Dados . . . 63
3.10.1 Diagrama Conceitual . . . 63
3.10.2 Diagrama Lógico . . . 64
3.11 Testes de Software . . . 65
3.12 Desenvolvimento Orientado por IA . . . 66
3.12.1 Fundamentos e limitações dos modelos de linguagem . . . 66
3.12.2 *Vibe Coding* x Codificação Agêntica . . . 67
3.12.3 Spec-Driven Development . . . 68
3.12.4 Ambiente de agentes . . . 69
3.12.4.1 Agentes e subagentes . . . 69
3.12.4.2 Skills . . . 69
3.12.4.3 Hooks . . . 72
3.12.4.4 Por que o Seed não utiliza MCP . . . 73
3.12.4.5 Isolamento do agente . . . 73
3.12.5 Fluxo proposto . . . 74

3.12.6 Subagentes de teste e implementação . . . 74
3.12.6.1 Responsabilidades do subagente tester . . . 75
3.12.6.2 Responsabilidades do subagente dev . . . 76
3.12.6.3 Encadeamento e pontos de controle . . . 76
3.12.6.4 Salvaguardas e limites dos mecanismos . . . 77
3.12.7 Limitações e ameaças à validade . . . 78
3.12.8 Plano de avaliação do processo no TCC2 . . . 79

4 CONSIDERAÇÕES FINAIS . . . 80
4.1 Conclusão . . . 80
4.2 Trabalhos Futuros . . . 80

REFERÊNCIAS . . . 81

APÊNDICES 87

APÊNDICE A – ARTEFATOS DA LEAN INCEPTION . . . 88
A.1 Visão de Produto . . . 88
A.2 É - Não É - Faz - Não Faz . . . 88
A.3 Objetivos do Negócio . . . 89
A.4 Personas . . . 90
A.4.1 Primeira Persona . . . 90
A.4.2 Segunda Persona . . . 90
A.4.3 Terceira Persona . . . 91
A.5 Jornadas dos Usuários . . . 92
A.6 *Brainstorming* de Funcionalidades . . . 93
A.7 Revisão (Técnica, de Negócio e Experiência do Usuário) . . . 94
A.8 Sequenciador . . . 95
A.9 Canvas MVP . . . 96

APÊNDICE B – QUESTIONÁRIO . . . 98

1 Introdução

O ministro canadense Marc Lalonde, em 1974, criou o primeiro documento a adotar a denominação de promoção da saúde, em um informe conhecido como "O Informe Lalonde". O mesmo destaca a importância da promoção da saúde no meio universitário (Rede Brasileira de Universidades Promotoras de Saúde, 2022). O autor estimula a ideia de que a saúde está diretamente relacionada com estilo de vida, organização assistencial, meio ambiente e biologia humana, sendo assim, o conceito de saúde vai além do sistema tradicional, englobando diversos aspectos extras (DAMASCENO; PIMENTEL, 2022).

Além disso, no relatório de Lalonde é adotado pela primeira vez o termo Determinantes Sociais de Saúde (DSS), que são fatores que determinam a saúde de uma pessoa, como boa alimentação, possibilidade de lazer, boas condições de trabalho, hábitos saudáveis, entre outros (HEIDMANN et al., 2006).

Indubitavelmente, o ensino superior tem um papel fundamental no desenvolvimento de indivíduos, sociedade e culturas, o mesmo possui a responsabilidade de oferecer uma educação que transforma a vida dos estudantes, como descrito na Carta de Okanagan. Sendo assim, é essencial que as Instituições de Ensino Superior (IES) estimulem a promoção da saúde, dessa forma foi criado o movimento de Universidades Promotoras de Saúde (UPS), cujo intuito é reunir instituições que tenham o objetivo de promover a saúde e debater soluções e avanços (DAMASCENO; PIMENTEL, 2022).

Tendo isso em mente, segundo (SCHLEICH, 2006), em "Integração na Educação Superior e Satisfação Acadêmica de Estudantes Ingressantes e Concluintes", define importantes pontos que as IES devem adotar como melhores meios de inovação tecnológica, novos espaços educativos e maior conhecimento sobre os estudantes, sendo esse último um dos fundamentos para a criação desse trabalho.

Sendo assim, como descrito por Schueller et al. (2019), a utilização de prática de *mindfulness*, onde o paciente foca a atenção no momento presente e diários de humor, apresenta resultados positivos, auxiliando na redução do estresse e promovendo o bem-estar emocional. Nesse contexto, é possível a utilização de meios tecnológicos para auxiliar a promoção da saúde dentro das IES.

1.1 Contextualização

O Brasil, segundo o Instituto Brasileiro de Geografia e Estatística (IBGE), em 2014, a estimativa de depressão entre indivíduos entre 18 a 29 anos corresponde a 3,9% da população, correspondendo a um grupo de jovens adultos. Boa parte disso se deve à

necessidade de tomada de decisões, pressão social, entre outros fatores (SOUZA, 2017).

Esse período é conhecido por possuir um dos maiores riscos psicológicos, podendo gerar gatilhos emocionais e estresse duradouro (SILVEIRA et al., 2011). Também citado por Bolsoni-Silva e Guerra (2014), a universidade é capaz de agravar problemas de saúde pré-existentes ou aumentar a probabilidade do seu surgimento. Sendo assim, a universidade é um ambiente estressante, principalmente no primeiro semestre. A solidão, um fator crucial para depressão, ocorre principalmente no primeiro ano da universidade devido a um novo ambiente e à falta de experiências anteriores em contextos desafiadores e exigentes (RHODES, 2014).

Ressalta-se a importância dos hábitos saudáveis, lazer, presença religiosa e atividade física, que ajudam a promover o bem-estar e são possíveis formas de lidar com estados emocionais negativos. Outro fator crucial para o desenvolvimento de problemas psicológicos é a falta de hábitos saudáveis e menor qualidade de vida nos domínios psicológico e ambiental (BARROSO; OLIVEIRA; ANDRADE, 2019).

Além disso, Barroso, Oliveira e Andrade (2019) destaca como os hábitos de vida influenciam diretamente a relação com a solidão e a depressão. Comportamentos como acordar cansado e realizar menos refeições ao longo do dia, entre outros, revelaram um impacto significativo na qualidade de vida dos participantes.

Outro aspecto que influencia a saúde emocional é o suporte social que, como defendido por Siqueira (2008) em "Construção e validação da escala de percepção de suporte social", é definido como “o processo no qual alguém pode conseguir ajuda de ordem instrumental, financeira ou emocional”. Esse apoio é considerado essencial para a promoção e manutenção da saúde mental.

1.2 Justificativa

Devido aos altos índices de risco psicológico destacados por Silveira et al. (2011), o ambiente universitário revela-se potencialmente hostil à saúde mental dos estudantes, exigindo a implementação de estratégias de promoção à saúde que vão além de ações institucionais, como palestras, e incluam também abordagens personalizadas e voltadas ao indivíduo, como diários de humor (SCHUELLER et al., 2019).

Nesse contexto, a vigilância participativa configura-se como uma abordagem essencial no monitoramento da população acadêmica, ao permitir a identificação precoce e a resposta eficiente a situações que afetam coletivamente a saúde mental (LEAL-NETO et al., 2016). Sendo assim, a fim de fornecer uma maneira de auxiliar a promoção de saúde nas IES, é fundamental a utilização deste conceito.

Além disso, a adoção de hábitos saudáveis mostra-se eficaz no enfrentamento e na mitigação dos impactos das doenças mentais, conforme apontado por Barroso, Oliveira e Andrade (2019). Ou seja, A promoção desses hábitos pode refletir positivamente no desempenho acadêmico e na permanência dos estudantes nos cursos. Além disso, com base no conceito de UPS se da a necessidade de criar meios para auxiliar a promoção a saúde nas IES, como mencionado por Damasceno e Pimentel (2022), não há uma fórmula única para promover a saúde no ambiente universitário.

Dessa forma, é fundamental a criação de uma ferramenta para auxiliar a adoção de hábitos saúdaveis e promover a saúde mental no meio acadêmico e uma ferramenta para auxiliar profissionais de saúde acadêmicos a identificar alunos em risco. Ademais, tendo em vista o conceito de suporte social é possível a utilização da tecnologia para criação de um software que realize esse papel (SIQUEIRA, 2008).

1.3 Questão de Pesquisa

Para fundamentar o desenvolvimento deste trabalho, a questão de pesquisa foi formalmente definida: Como promover intervenções personalizadas na saúde mental de estudantes universitários por meio de um aplicativo mobile baseado na análise de dados sobre hábitos de vida e estados emocionais?

1.4 Objetivos

Este TCC possui um objetivo geral e outros oito objetivos específicos que serão aprofundados nesta seção.

1.4.1 Objetivo Geral

Este trabalho possui como objetivo geral o desenvolvimento de um aplicativo mobile que visa a promoção da saúde entre os membros da comunidade universitária, por meio da coleta e análise de dados sobre seus hábitos de vida e estados de saúde mental.

1.4.2 Objetivos Específicos

São objetivos específicos deste trabalho:

- Contribuir para a identificação de obstáculos no bem-estar dos indivíduos de determinada instituição;
- Coletar dados relevantes que subsidiem a elaboração de projetos sociais e de saúde pela instituição;

- Disponibilizar gráficos de progressos individuais para os usuários do aplicativo;
- Oferecer orientações personalizadas de acordo com o progresso individual para melhoria de aspectos que apresentem evolução insatisfatória;
- Disponibilizar um recurso passível de ser utilizado de forma ampla pela comunidade acadêmica para a aplicação de questionários relacionados à saúde da comunidade universitária;
- Servir como uma ferramenta para disseminação de informações relevantes sobre o tema de saúde entre os indivíduos do campo universitário;
- Propor uma estratégia de Promoção da Saúde aos estudantes e funcionários universitários;
- Criar indicadores institucionais que avaliem o impacto de ações de promoção da saúde implementadas na comunidade universitária.

1.5 Organização do Trabalho

O presente Trabalho de Conclusão de Curso (TCC) está dividido nos seguintes capítulos:

- **Capítulo 1 - Introdução:** o capítulo introdutório apresenta o cenário de aplicação da proposta, a justificativa da realização da pesquisa, a questão que norteia este trabalho, os objetivos pretendidos e as metodologias empregadas no desenvolvimento deste projeto.
- **Capítulo 2 - Referencial Teórico:** neste capítulo são expostos os conceitos principais que cercam a temática deste trabalho tratados em seis tópicos principais, que abordam definições sobre saúde, hábitos de vida, aplicativos já existentes, escala de likert e demais conceitos relacionados ao tema.
- **Capítulo 3 - Metodologia:** capítulo voltado para a apresentação da metodologia de pesquisa e de desenvolvimento do projeto, em que são detalhadas as abordagens utilizadas para a implementação eficaz da ferramenta de software.
- **Capítulo 4 - Desenvolvimento:** este capítulo é destinado à apresentação dos detalhes do desenvolvimento do software e os ajustes realizados para garantir o êxito da implementação.
- **Capítulo 5 - Considerações Finais:** este capítulo final expõe os resultados obtidos no trabalho até então, assim como as sugestões para desenvolvimentos futuros.

2 Referencial Teórico

Neste capítulo será explorado o referencial teórico que servirá de suporte teórico e contextual para o desenvolvimento desta pesquisa. Para tanto, realizou-se uma detalhada revisão de literatura, contemplando os principais conceitos relacionados ao tema de estudo.

2.1 Saúde e Bem-estar

O conceito de saúde é um dos mais complexos e discutidos no campo da medicina. Embora a primeiro momento pareça um termo simples e autoexplicativo, várias versões são apresentadas em sua definição. Segundo Scliar (2007), o termo não possui um único significado, mas reflete a conjuntura social, econômica, política e cultural de cada um, e depende do lugar, da época e da classe social além de valores individuais e de concepções científicas, religiosas e filosóficas.

Para Boorse (1977), por exemplo, o termo se resumia apenas à ausência de doença. Essa ideia chocava-se com o que era difundido pela Organização Mundial da Saúde (OMS) à partir da divulgação da carta de princípios de 7 de abril de 1948 que dizia que “Saúde é o estado do mais completo bem-estar físico, mental e social e não apenas a ausência de enfermidade"(SCLIAR, 2007). É essa concepção ampliada que valoriza múltiplas fatores do bem-estar que servirá como base para o desenvolvimento deste projeto.

Afinal, entender o termo "saúde"apenas como a ausência de enfermidades é reduzir a complexidade da experiência humana e ignorar variáveis cruciais que afetam diretamente a qualidade de vida dos indivíduos. Sob esse prisma surgiu o Relatório de Lalonde (1974), que determinava que a saúde poderia ser classificada em quatro áreas gerais, sendo elas: biologia, ambiente, estilo de vida e organização dos serviços de saúde (LALONDE, 1981). Essas seriam consideradas determinantes da saúde pelo autor, que destacou ainda os impactos dos hábitos de vida individuais na própria saúde (LALONDE, 1981).

Com base nessa perspectiva e com a pretensão de iniciar um monitoramento de saúde de indivíduos, propõe-se, neste trabalho, resumir as seções de biologia e de organização dos serviços de saúde em questões de disposições genéticas e de avaliação de acesso a saúde. Assim, os tópicos referentes ao estilo de vida e ao ambiente ganham maior extensibilidade e contemplam questões como hábitos alimentares, prática de exercícios físicos, uso de substâncias, frequência de interações sociais, condições de moradia, entre outros.

Vale ressaltar que a síntese de dois tópicos tão importantes enquanto determinantes da saúde não implica necessariamente em uma diferença de complexidade entre as variáveis. Essa opção reflete apenas uma decisão metodológica do projeto que busca

concentrar esforços analíticos em aspectos sobre os quais, acredita-se que uma instituição de ensino superior teria maior autonomia para promover mudanças significativas. Dessa forma optou-se pela simplificação do levantamento de dados relativos à biologia e à organização de serviços de saúde, em razão de serem variáveis mais distantes da esfera de atuação dessas instituições de ensino.

2.1.1 Estilo de Vida e Ambiente

Nestas duas categorias de determinantes da saúde citadas por Lalonde (1981), alguns estudos mostram que existe uma forte correlação entre ambas. Os resultados obtidos por Muniz e Garrido (2021), por exemplo, de forma análoga a outros estudos mencionados no artigo, concluem que o meio universitário impacta diretamente na saúde física e mental dos estudantes ao impor exigências que levam a sobrecarga dos estudantes. Esta é mais uma razão que reforça a necessidade de monitoramento de hábitos no cenário universitário e a implementação de estratégias para contornar hábitos ruins dentro dessas instituições.

Além de avaliação de ambiente acadêmico seria interessante também contemplar dados à respeito do ambiente de trabalho e familiar, uma vez que esses fatores exercem grande impacto na saúde mental e do bem-estar dos indivíduos.

No que diz respeito a um estilo de vida que promova a saúde e o bem-estar, além da óbvia necessidade biológica vital de hidratação do próprio organismo, incluem-se práticas de atividades físicas, alimentação equilibrada, regularidade no ciclo do sono (ARLINGHAUS; JOHNSTON, 2019), interações sociais, voluntariado (HELLIWELL et al., 2025) e momentos de lazer (BATISTA; RIBEIRO; JUNIOR, 2012).

Dessa forma seções de avaliação destes aspectos na vida cotidiana serão implementadas no aplicativo para acompanhamento da saúde geral da comunidade universitária.

2.1.2 Saúde Mental

Como vimos, a saúde não se resume fatores biológicos favoráveis. Aqui é posto a saúde mental em uma seção exclusiva para se avaliar já que ela envolve parâmetros profundamente subjetivos e complexos que ultrapassam o cumprimento de rotinas que visam o bem-estar, embora a existência de fortes correlações entre a prática de bons hábitos e melhoria do estado emocional. Contudo, essas correlações não esclarecem todas as condições clínicas geradas por doenças mentais (QUARTILHO, 2010).

Comenta-se em uma separação do cérebro e da mente que não nos possibilita tratar questões de saúde mental com uma explicação linear de experiências conscientes (QUARTILHO, 2010). Dessa forma, mesmo com um bom estilo de vida, é possível que ainda aja uma saúde mental comprometida.

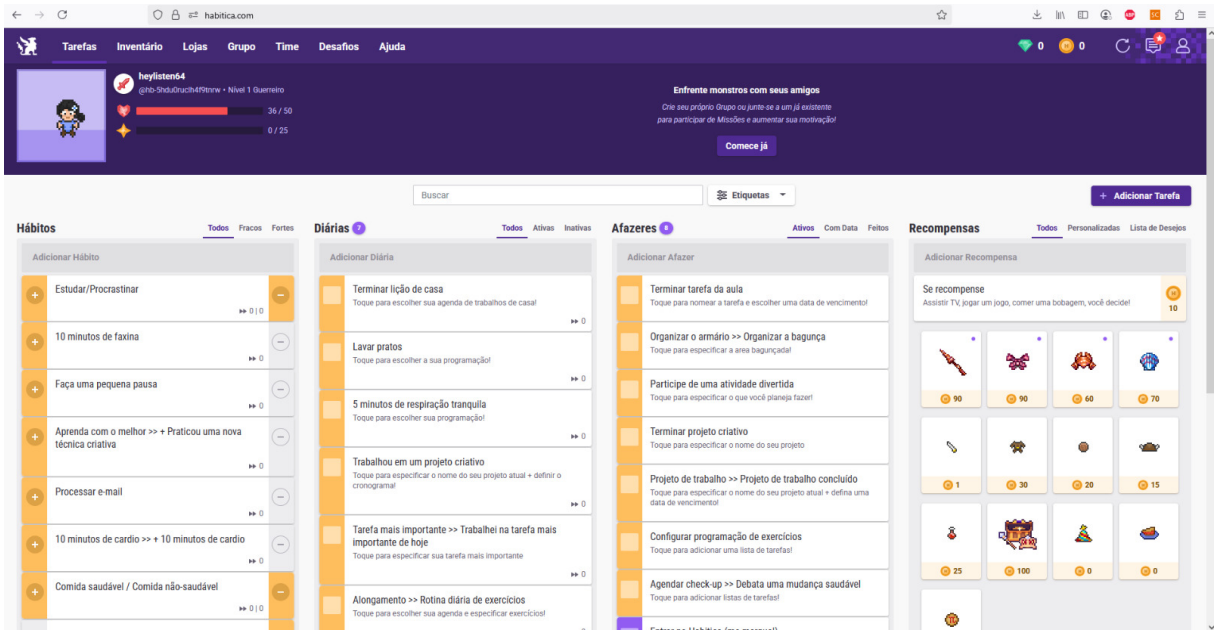
Esta variável é crucial de ser analisada, principalmente quando vimos que antes mesmo de um período pandêmico - e, por mais que não seja uma atribuição direta da Universidade de sanar este problema - já se tratava de uma das maiores reinvindicações entre os alunos da Universidade de Brasília, sendo uma das principais razões de evasão acadêmica (OLIVEIRA, 2020). Com os agravantes do contexto pós-pandemia essa questão se tornou ainda mais importante de ser avaliada (SILVA et al., 2025).

2.2 Plataformas Existentes

Para a execução deste trabalho, foi previamente realizado um levantamento de dados sobre aplicativos com propostas semelhantes de monitoramento de hábitos e do estado de saúde mental.

O Habitica, mostrado na Figura 1, foi um dos softwares encontrados neste levantamento. O aplicativo de código aberto tem como funcionalidade principal amparar a formação de hábitos novos por meio da gamificação. Ele funciona como uma combinação de rede social e um jogo pois a ferramenta te permite adicionar amigos e combater monstros em missões com eles conforme você realiza suas tarefas diárias. Cada tarefa concluída na vida real, quando registrada no aplicativo, concede pontos de experiência ao usuário que se engaja na realização de suas próprias tarefas para evoluir no jogo (HABITICA, 2025).

Figura 1 – Tela inicial do Habitica



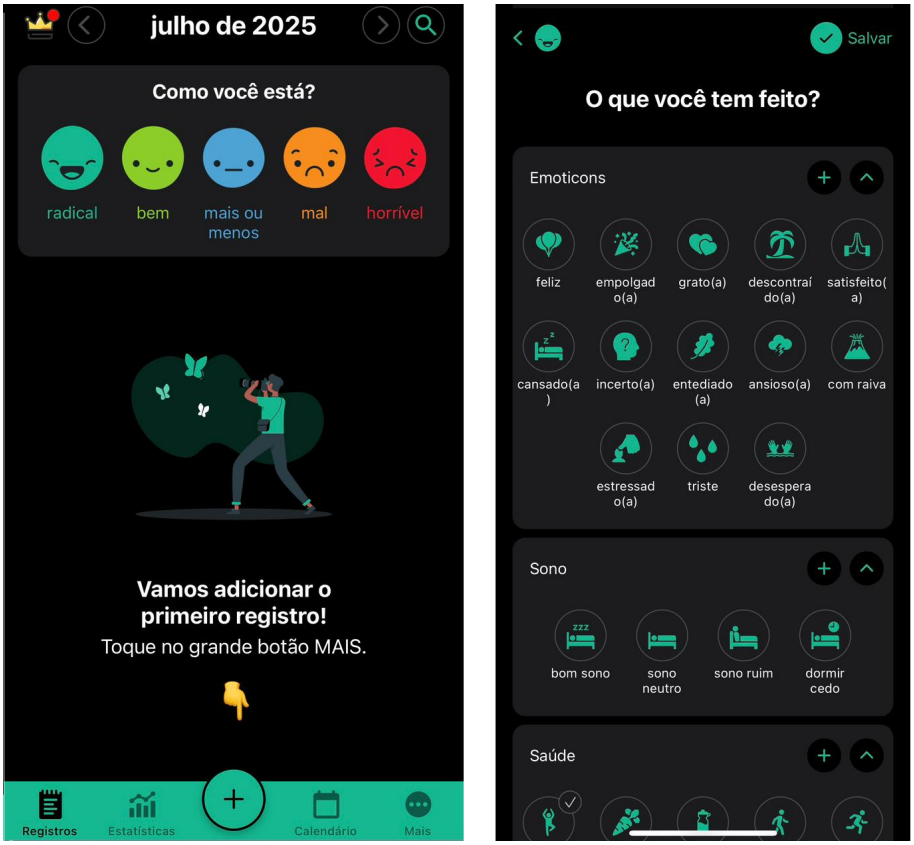
Fonte: (HABITICA, 2025).

Porém, em comparação a este sistema, o projeto em desenvolvimento se destaca por oferecer conteúdos de direcionamento personalizados dispostos em um feed dinâmico que

se adapta de acordo com a progressão individual do usuário nas diversas seções do bem-estar dispostas no projeto. Dicas, eventos específicos e demais informações relacionadas aos temas em que o usuário apresenta um progresso insatisfatório são exibidos para ele nesse espaço.

Já o Daylio se intitula como um diário de autocuidado que registra diariamente o humor e rastreia a felicidade do usuário sem a necessidade de escrita, registrando hábitos apenas com emojis, o que torna a usabilidade mais amigável (DAYLIO, 2021). Nele registramos nosso humor e a partir daí selecionamos outras opções que permeiam o nosso estado emocional, como qualidade do sono, interações sociais, frequência em atividades físicas, entre outros. Assim, o aplicativo traça, de registros simples de humor a estatísticas de correlação entre essas diferentes variáveis.

Figura 2 – Telas do Daylio



Fonte: (DAYLIO, 2021).

Entretanto nenhum dos softwares encontrados mostraram-se como uma ferramenta conjunta entre uma instituição específica e seus respectivos membros. A solução deste trabalho consiste em uma ferramenta que forneça dados da comunidade universitária e beneficie não apenas o usuário - que passa a tomar consciência de seus comportamentos por feedbacks de progressos - como também a própria instituição que poderá acompanhar indicadores coletivos e planejar ações direcionadas de promoção da saúde e consequentemente ter membros mais saudáveis e engajados em seu ambiente.

As plataformas também não oferecem uma seleção básica de seções de bem-estar, variáveis que este trabalho propõe monitorar de forma estruturada.

3 Metodologia

Este capítulo aprofunda o plano metodológico utilizado no desenvolvimento deste trabalho. Serão descritas as tecnologias empregadas, o cronograma, os artefatos de requisitos de software, diagramas de arquitetura de software e do projeto de dados, além de outros documentos elaborados.

3.1 Heurísticas de Nielsen

Segundo Nielsen (1994) existem dez princípios amplamente conhecido para proporcionar usabilidade a um sistema, conhecido como Dez Heurísticas de Nielsen. Esses princípios funcionam como diretrizes para o desenvolvimento de interfaces mais intuitivas, eficientes e centradas no usuário, sendo essas diretrizes:

1. **Visibilidade do status do sistema**: O sistema deve sempre manter os usuários informados sobre o que está acontecendo, por meio de feedback apropriado em tempo razoável.

2. **Correspondência entre o sistema e o mundo real**: A interface deve falar a linguagem do usuário, com palavras, frases e conceitos familiares, seguindo convenções do mundo real.

3. **Controle e liberdade do usuário**: Usuários frequentemente escolhem funções por engano; é necessário oferecer mecanismos claros de desfazer e refazer ações.

4. **Consistência e padrões**: Os usuários não devem ter que imaginar se diferentes palavras, situações ou ações significam a mesma coisa. O sistema deve seguir convenções.

5. **Prevenção de erros**: Melhor do que apresentar boas mensagens de erro é evitar que o erro ocorra, por meio de design cuidadoso.

6. **Reconhecimento em vez de memorização**: Minimizar a carga de memória do usuário, tornando objetos, ações e opções visíveis.

7. **Flexibilidade e eficiência de uso**: A interface deve atender tanto usuários iniciantes quanto experientes, permitindo atalhos para os mais avançados.

8. **Estética e design minimalista**: Diálogos e telas não devem conter informações irrelevantes ou raramente necessárias.

9. **Ajuda no reconhecimento, diagnóstico e recuperação de erros**: As mensagens de erro devem ser expressas em linguagem simples, indicando claramente o problema e sugerindo uma solução.

10. **Ajuda e documentação**: Mesmo que o sistema possa ser utilizado sem documentação, pode ser necessário oferecer ajuda e instruções acessíveis e objetivas.

Sendo assim, a fim de proporcionar usabilidade e acessibilidade a todos os publicos o projeto proposto visa a utilização das heuristicas, como também visa seguir as normas da LGPD (Lei Geral de Proteção) a fim de proporcionar maior segurança aos usuários (BRASIL, 2018).

3.2 Escala de Likert

A Escala de Likert é uma das principais ferramentas psicométricas, sendo amplamente utilizada nas pesquisas em ciências sociais e educacionais. Ela foi desenvolvida com o principal objetivo de quantificar atitudes, percepções e crenças, sendo uma maneira de transformar fenômenos qualitativos da experiência humana em dados quantificáveis e cientificamente tratáveis (JOSHI et al., 2015).

Além disso, do ponto de vista psicométrico, a Escala de Likert proporciona a mensuração de variáveis latentes, como sentimento ou opiniões, por meio de afirmações com os quais os participantes expressam níveis de concordância ou discordância em uma escala ordinal simétrica. Em outras palavras, a escala apresenta cinco ou sete pontos, indo de "discordo totalmente"até "concordo totalmente", permitindo capturar nuances mais refinados, sendo preferível a escala com sete pontos, pois proporciona mais opções intermediárias ao participante, o que aumenta a precisão da medida psicométrica (JOSHI et al., 2015).

Sendo assim, a Escala de Likert é fundamental para a avaliação de atitudes e percepções de natureza psicológica, proporcionando uma coleta de dados mais precisa, coerente e passível de análise estatística. Portanto, é uma ferramenta indispensável para a realização de formulários sobre o bem-estar e hábitos.

3.3 Alpha de Cronbach

O Alpha de Cronbach, proposto por Lee Cronbach em 1951, é uma maneira de avaliar quão fidedigno é um conjunto de dados. Em outras palavras, ele verifica o quanto as perguntas estão correlacionadas entre si, o que permite avaliar a confiabilidade dos dados (CRONBACH, 1951). Sendo assim, é possível calcula-lo a partir da seguinte equação:

$$\alpha = \frac{N}{N-1}\left(1 - \frac{\sum_{i=1}^{N} \sigma_{Y_i}^2}{\sigma_X^2}\right) \tag{3.1}$$

Além de indicar a confiabilidade dos dados coletados, o Alpha de Cronbach também permite verificar a concordância entre as perguntas. Isso é útil para avaliar se o conjunto de questões está bem estruturado e se mede de forma coerente o mesmo conceito (CRONBACH, 1951).

O cálculo do Alpha de Cronbach pode ser implementado em qualquer linguagem de programação Turing-completa, como Java, python, entre outras linguagens de programação modernas. Ou seja, a equação independe da linguagem de programação, sendo versátil e adaptável.

Em suma, o valor do Alpha de Cronbach varia entre -1 e 1, sendo que valores próximos de 1 ou -1 indicam alta confiabilidade. Sendo assim, basta introduzir o conjunto de dados ao algoritmo que implementa a equação do Cronbach e considera-se que valores acima de 0.6 ou menores que -0.6 possuem dados confiáveis. Contudo, valores muito baixos ou negativos indicam pouca ou nenhuma confiabilidade (CRONBACH, 1951).

3.4 Metodologia de pesquisa

Segundo Silveira, FLÔR e Machado (2011), metodologia consiste "em um estudo crítico e analítico dos métodos de investigação". Os métodos científicos a serem seguidos para conceber novos conhecimentos são inúmeros e compõem o estudo de estruturas de pesquisa que podem ser classificadas de acordo com sua natureza, seu objetivo, sua abordagem e seus procedimentos. (NASCIMENTO; SOUSA, 2016).

Conforme descrito por Zanella et al. (2006), a pesquisa apresenta diversos tipos relacionado à sua natureza, objetivos, abordagem e procedimentos. Ainda de acordo com o material, é possível dizer que a presente pesquisa possui:

- **Natureza:** aplicada, por ter como objetivo a solução de um problema humano;
- **Objetivos:** exploratória, em razão de ser uma pesquisa que busca ampliar o conhecimento sobre fenômenos da comunidade acadêmica, atrás de conhecer suas necessidades e buscar tratá-las;
- **Abordagem:** quantitativa, visto que caracteriza-se pela ampla utilização de instrumental estatístico fundamental na análise de dados coletados;
- **Procedimentos:** procedimento bibliográfico para investigação do referencial teórico aliado a produção tecnológica no contexto de desenvolvimento de software.

3.4.1 Natureza de Pesquisa

No que diz respeito à natureza da pesquisa, ela pode ser classificada como básica ou aplicada. Enquanto a pesquisa básica visa gerar novas verdades para a ciência, sem que aja compromisso com a aplicação prática no ambiente investigado, a pesquisa aplicada é direcionada à solução de problemas específicos (NASCIMENTO; SOUSA, 2016). Como o projeto busca desenvolver um aplicativo com o intuito de mitigar problemas de saúde dentro de instituições acadêmicas, podemos dizer que a natureza desta pesquisa é aplicada.

3.4.2 Abordagem de Pesquisa

Conforme também aponta NASCIMENTO e Sousa (2016) a metodologia de pesquisa pode ser definida como uma abordagem quantitativa, qualitativa ou mista. Enquanto a abordagem quantitativa valoriza respostas estatísticas e padronizadas, o método qualitativo considera as singularidades de cada sujeito objeto da pesquisa. Ainda com base no que o autor descreve, essas abordagens, apesar de possuírem características diferentes, possuem caráter complementar e não excludente (NASCIMENTO; SOUSA, 2016).

Aplicado ao contexto deste trabalho, apesar de serem considerados parâmetros subjetivos para o desenvolvimento do aplicativo, ele possui como foco a utilização de ferramentas estatísticas para análise de dados coletados e consequente solução de problemas. Dessa forma, a abordagem desta pesquisa possui caráter quantitativo.

3.4.3 Objetivo de Pesquisa

Em Silveira, FLÔR e Machado (2011) é possível identificar a ramificação da pesquisa com base em seus objetivos em: exploratória, descritiva e explicativa. A pesquisa descritiva restringe-se a interpretar a realidade sem promover mudanças. A pesquisa explicativa busca elucidar a razão de algum fenômeno. Neste trabalho não pretende-se descrever realidades ou esclarecer porquês, portanto é considerada uma pesquisa exploratória. Esta consiste em um passo inicial de um trabalho científico que assume um caráter de pesquisa bibliográfica e estudo de caso que ajuda a formular o problema de pesquisa e seus objetivos (SILVEIRA; FLÔR; MACHADO, 2011).

3.4.4 Procedimento de Pesquisa

Ainda de acordo com Silveira, FLÔR e Machado (2011), os procedimentos técnicos de pesquisa incluem as categorias: bibliográfica, documental, experimental, estudo de caso, levantamento, estudo de campo, pesquisa-ação e pesquisa participante. A pesquisa bibliográfica, que é um dos procedimentos de pesquisa utilizados neste trabalho, é um procedimento efetuado a partir de material já publicado e traz a base teórica científica para o projeto. Este procedimento está presente em todas as produções acadêmicas

Adicionalmente, no contexto de desenvolvimento de software, a produção tecnológica trata-se de outro procedimento de pesquisa adotado neste trabalho que tem como objetivo o desenvolvimento de um aplicativo a fim de solucionar problemas da comunidade universitária.

3.4.5 Processo Metodológico

De acordo com a Universidade de Brasília (UnB) o Trabalho de Conclusão de Curso (TCC) deve ser dividido em duas fases, sendo elas: TCC1 e TCC2. Sendo assim, o TCC1 tem como objetivo propor um projeto viável para desenvolvimento e desenvolver a base teórica, já o TCC2 tem o objetivo de implementar o projeto proposto, de acordo com a definição realizada no TCC1.

3.4.5.1 TCC1

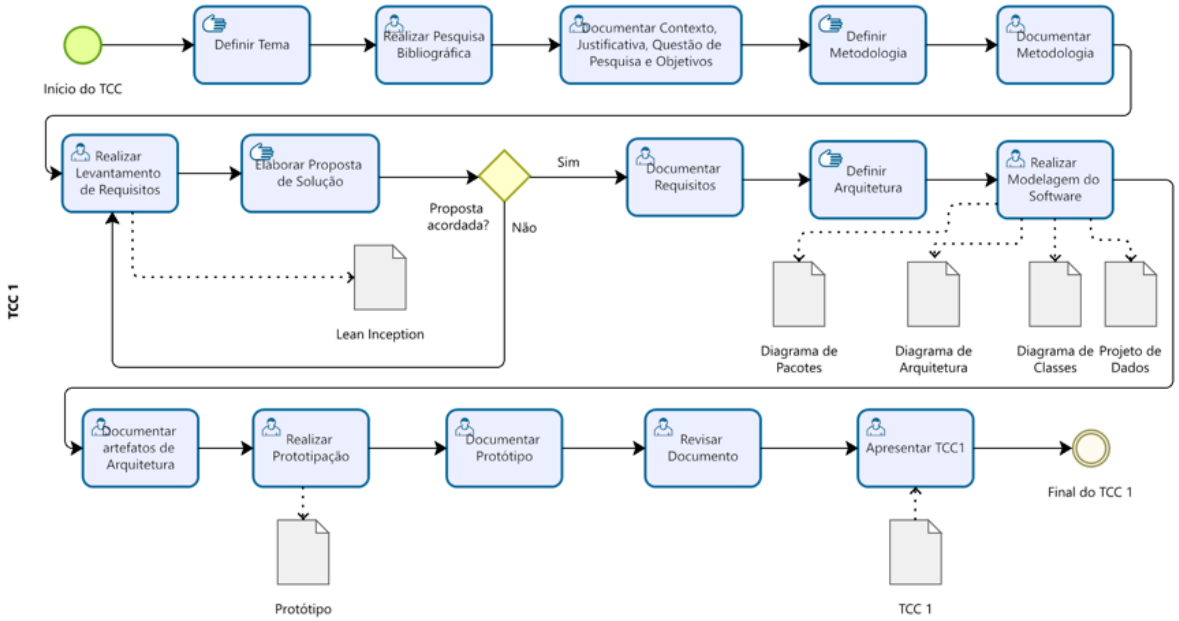
Dividimos o Trabalho de Conclusão de Curso 1 em etapas que podem ser vistas detalhadamente na Tabela 1 e na Figura 3.

Tabela 1 – Cronograma das Atividades do TCC1 (2025).

Atividade	**Abr**	**Mai**	**Jun**	**Jul**
Definir tema	X			
Definir metodologia	X			
Realizar pesquisa bibliográfica	X	X		
Elaborar proposta de solução		X		
Realizar levantamento de requisitos		X	X	
Definir arquitetura			X	
Realizar Modelagens			X	
Realizar Prototipação			X	
Revisão Geral				X
Apresentação TCC1				X

Fonte: autoria própria.

Figura 3 – Raia do TCC1 do BPMN do Processo Metodológico.

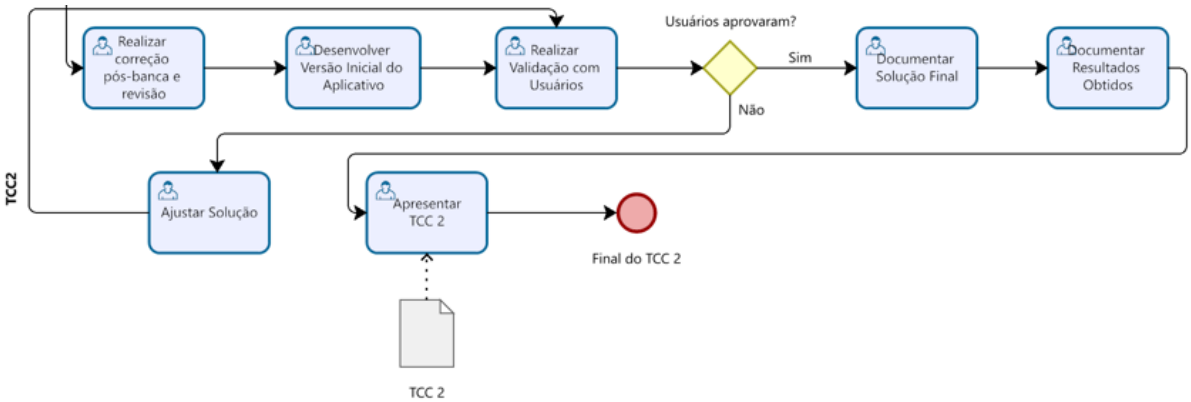


Fonte: autoria própria.

3.4.5.2 TCC2

Além disso, delineamos também o fluxo de atividades que serão realizadas no processo de desenvolvimento do Trabalho de Conclusão de Curso 2, como pode ser visto na Figura 4.

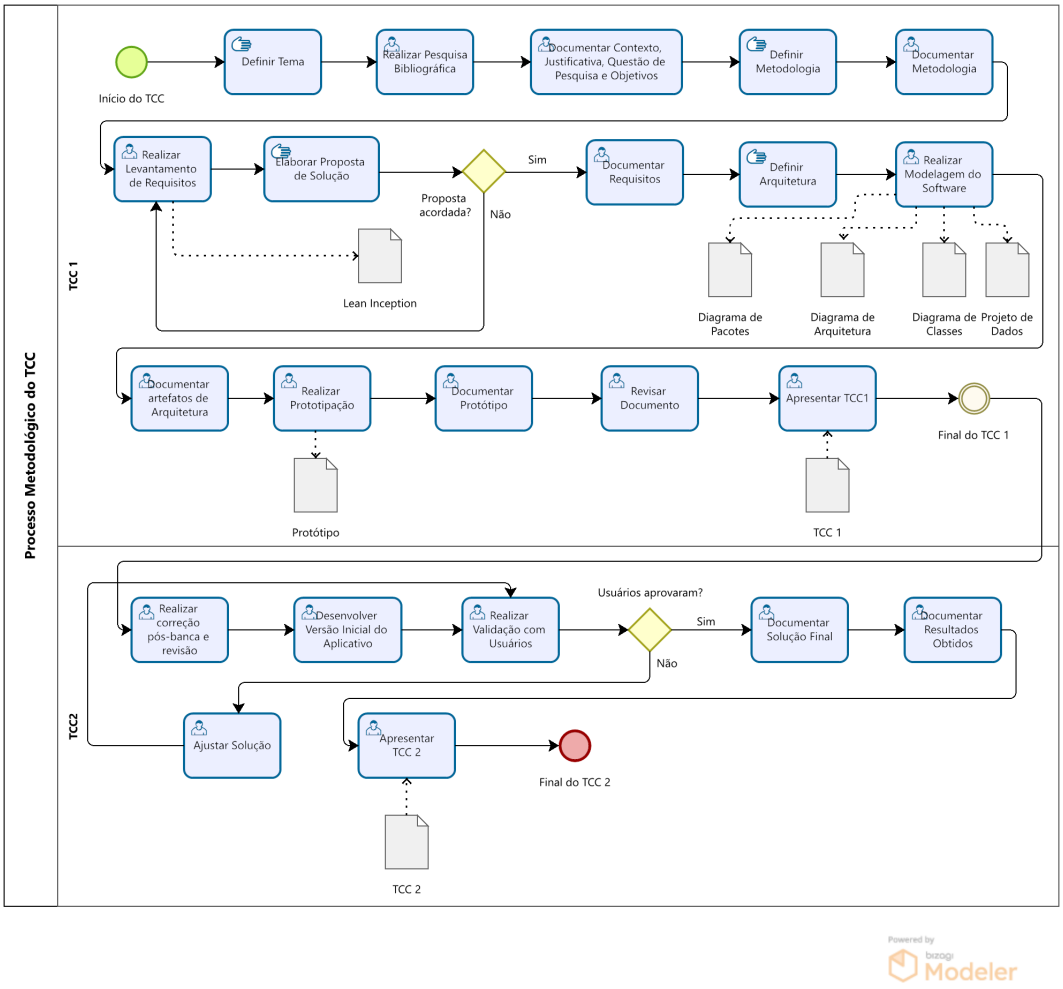
Figura 4 – Raia do TCC2 do BPMN do Processo Metodológico.



Fonte: autoria própria.

Totalizando temos o BPMN visto na Figura 5.

Figura 5 – BPMN do Processo Metodológico



Fonte: autoria própria.

3.5 Metodologia de Desenvolvimento

As metodologias de software proporcionam um conjunto de práticas e plano de trabalho que se alinha com as demandas do projeto permitindo que impulsionem a produtividade e proporcione o desenvolvimento de um produto de maior valor (KENT; ANDRES, 2004). Sendo assim, a metodologia de desenvolvimento deste trabalho segue princípios de práticas ágeis e une características do Extreme Programming (XP) ao Scrum, Kanban e Lean Inception.

No que diz respeito a metodologias ágeis, os atributos de programação em pares, aplicação de testes automatizados com TDD e a refatoração foram aproveitados da metodologia XP (HIGHSMITH, 2000). Já as características extraídas da metodologia Scrum foram os conceitos de sprints iterativas e incrementais, junto das reuniões diárias (SCHWABER; SUTHERLAND, 2013). Vale destacar que esta última metodologia coube

adaptações no contexto de uma equipe reduzida.

O Kanban, por sua vez, foi adotado como uma ferramenta de gestão visual para acompanhar o andamento das tarefas. Por fim, o Lean Inception estruturou a definição e priorização do Mínimo Produto Viável (MVP), que possibilitou a conformidade entre os envolvidos sobre a ideia do produto final.

3.5.1 Metodologia Ágil

As metodologias ágeis foram criadas no final dos anos 1990 e seu principal objetivo era solucionar as limitações dos métodos tradicionais de desenvolvimento de software, que frequentemente resultavam em produtos inflexíveis e com longos ciclos de entrega (KENT; ANDRES, 2004).

Dessa forma, o projeto proposto busca adotar metodologias ágeis como maneira de garantir maior flexibilidade, caso seja necessário a realização de ajustes, e proporcionar um processo mais produtivo, além de mitigar riscos ao longo do ciclo de vida do software.

3.5.2 Uso de Inteligência Artificial

Neste trabalho, inteligência artificial generativa será empregada como apoio à elaboração e à análise de artefatos de desenvolvimento, bem como à geração de partes da interface e do código. Modelos de linguagem de grande porte (LLMs) podem produzir texto e código a partir das instruções e do contexto fornecidos, mas suas respostas podem conter informações incorretas, inconsistentes ou não previstas nos requisitos (REFERENCIA AQ). Por essa razão, as saídas da ferramenta serão tratadas como propostas para análise, e não como decisões ou requisitos automaticamente válidos. No contexto do Seed, a tabela de suporte tecnológico prevê Claude Code como apoio à escrita, revisão e depuração de código, e Figma Make para prototipação e geração assistida da interface web; a utilização dessas ferramentas será subordinada à arquitetura, aos requisitos aprovados e à plataforma definida para a entrega (NIST, 2024; FIGMA, s.d.).

3.5.2.1 Grilling

Antes da implementação de cada funcionalidade, será conduzida uma sessão estruturada de elicitação e revisão crítica de requisitos, denominada neste trabalho *grilling*. Com base nos requisitos, histórias de usuário, arquitetura e demais documentos do projeto, o LLM formulará perguntas para evidenciar ambiguidades, contradições, condições ausentes, casos-limite e critérios de aceite que ainda não possam ser verificados. As respostas serão fornecidas e as decisões confirmadas pelos responsáveis pelo projeto; a ferramenta não deverá completar lacunas por suposição nem decidir requisitos em nome dos participantes. A atividade é registrada aqui como uma dinâmica de apoio à elicitação e à

revisão de requisitos, e não como uma técnica normatizada chamada “grilling”. A engenharia de requisitos e a necessidade de requisitos bem-formados, verificados, validados e rastreáveis são fundamentadas pela ISO/IEC/IEEE 29148; estudos recentes também investigam LLMs na geração de perguntas de seguimento em entrevistas de requisitos (ISO/IEC/IEEE, 2018; SHEN; SINGHAL; BREAUX, 2025).

3.5.2.2 Spec-Driven Development

As decisões confirmadas serão registradas em uma especificação que orientará o planejamento e a implementação da funcionalidade, seguindo o desenvolvimento orientado a especificações (*spec-driven development*). Para cada comportamento especificado, os critérios de aceite e os testes correspondentes serão definidos e revisados antes da geração ou alteração do código. A implementação assistida por IA será então comparada à especificação e submetida aos testes aplicáveis, à revisão humana e à verificação de regressões; se um requisito mudar, a especificação, os testes e o código relacionado deverão ser revistos em conjunto. Esse fluxo relaciona elicitação, especificação e TDD à metodologia já descrita no projeto, sem pressupor que a geração por IA garanta correção. A literatura sobre LLMs em engenharia de requisitos aponta desafios de validação e consistência, enquanto a documentação do GitHub Spec Kit descreve um processo de especificar, planejar, decompor tarefas, implementar e verificar/convergir (HEMMAT et al., 2025; GITHUB, 2026).

3.5.3 Extreme Programming (XP)

O objetivo do Extreme Programming é o desenvolvimento de um software robusto. Com o uso dessa metodologia de desenvolvimento, é possível desenvolver softwares com menos defeitos, com menor custo e com maior produtividade (KENT; ANDRES, 2004). Sendo assim, a utilidade dessa metodologia é justificada pela ênfase na qualidade do código e eficiência das práticas de programação.

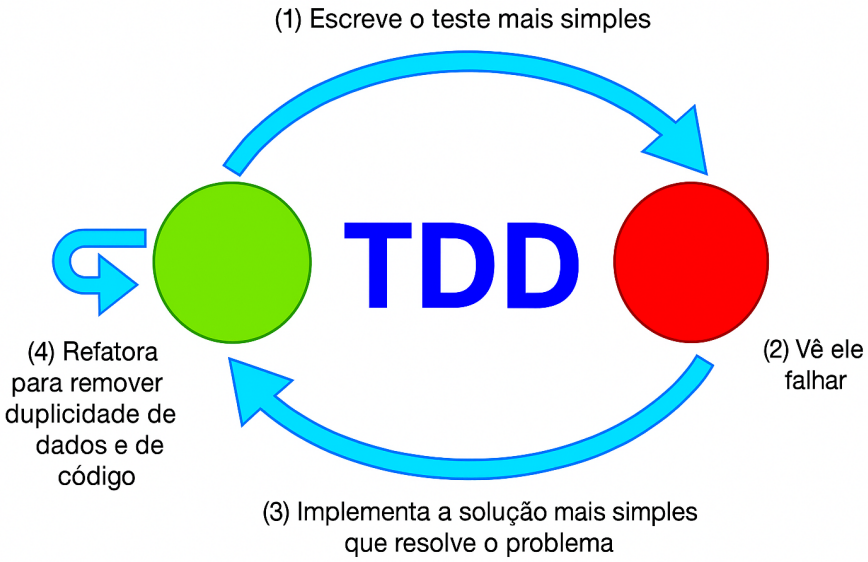
O XP segue quatro valores fundamentais: a **comunicação**, onde o time deve ser capaz de expressar suas opiniões e se comunicar de maneira eficaz; **simplicidade**, priorizando soluções simples para atingir objetivos propostos; **coragem**, onde o desenvolvedor deve ter a disposição para refatorar e implementar sem receio; por fim, **feedback**, a fim de receber retorno construtivo (KENT; ANDRES, 2004). Sendo assim, a junção desses valores favorece o desenvolvimento eficaz, abrindo espaço para inovação e novas soluções.

Segundo Highsmith (2000), o Extreme Programming possui 12 práticas desejaveis, porém, devido ao escopo do projeto, não é viável adotar todas. As práticas consideradas mais adequadas para este projeto são: programação em pares, testes, com ênfase em TDD e refatoração.

3.5.3.1 Test-Driven Development (TDD)

Com o intuito de promover o desenvolvimento e melhorar a qualidade do código, é utilizado o Test-Driven Development (TDD). Essa abordagem consiste em escrever testes automatizados antes de implementar novas funcionalidades. O ciclo segue o seguinte fluxo: primeiramente, se escreve um teste que irá falhar, em seguida é implementado o código para satisfazer o teste e depois a refatoração, conforme a Figura 6. Essa abordagem garante que o código satisfaça todos os requisitos desde o início (ANICHE, 2014).

Figura 6 – Ciclo do Test-Driven Development.



Fonte: Aniche (2014).

Estudos empíricos indicam benefícios dessa prática. Janzen (2005) observaram que programadores que utilizam TDD produzem código que passa em mais testes de caixa-preta, e Maximilien e Williams (2003) relataram redução superior a 40% na quantidade de defeitos. Ambos os trabalhos também apontam ganhos de produtividade, atribuídos ao menor tempo gasto com depuração. Em contrapartida, o TDD exige esforço adicional no início do desenvolvimento: segundo Bhat e Nagappan (2006), o tempo da fase inicial aumenta entre 15% e 30%. Considerando os benefícios de longo prazo, esse custo é considerado aceitável.

Para aplicar o TDD no desenvolvimento assistido por inteligência artificial, o projeto adota uma orquestração de três agentes com papéis separados, de modo que o mesmo agente não defina o comportamento esperado e o código que o satisfaz. O **agente orquestrador** coordena o fluxo: analisa requisitos, critérios de aceite e contratos, esclarece lacunas, executa os testes e controla a passagem entre as etapas. O **agente de testes** propõe os casos de teste a partir dos critérios de aceite, sem implementar a funcionali-

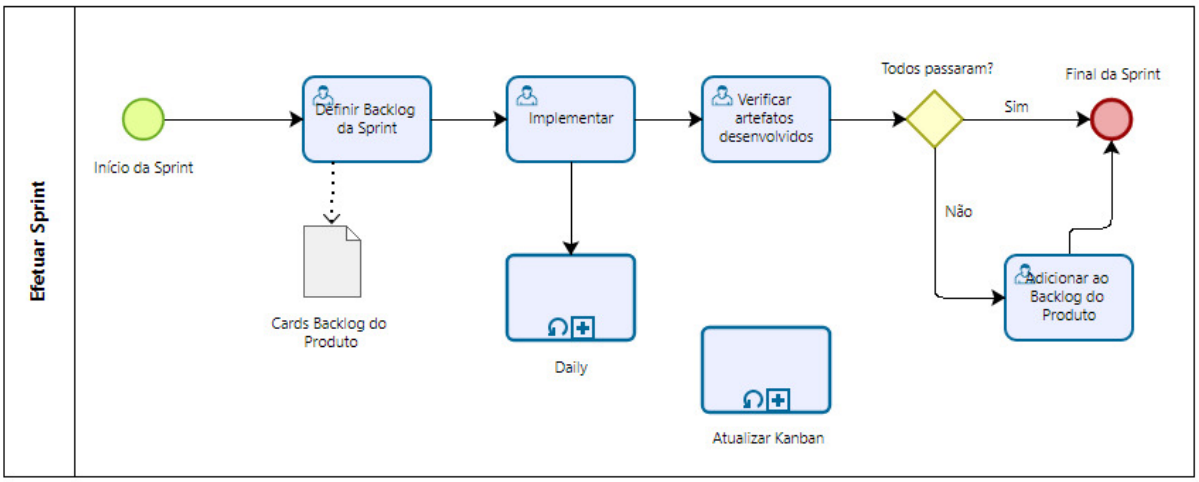
dade, aplicando partições de equivalência, análise de valores-limite e tabelas de decisão, conforme o syllabus CTFL do ISTQB (**??**). Os valores aceitos, a precisão e as respostas a entradas ausentes, inválidas ou fora de faixa devem vir dos requisitos ou dos contratos e, quando não estiverem definidos, são esclarecidos antes da criação dos testes. O **agente de implementação** escreve o código, sem permissão para remover, desativar ou enfraquecer testes.

O fluxo possui dois pontos de controle humano. No primeiro, uma pessoa revisa os testes propostos antes de qualquer implementação, pois há evidências de que suítes geradas exclusivamente por modelos de linguagem apresentam limitações (**??**). Aprovados os testes, o orquestrador os executa e confirma que falham pela ausência do comportamento especificado, e não por erro de configuração (*red*), registrando a rastreabilidade entre requisito, caso de teste e resultado. Em seguida, o agente de implementação produz o código mínimo para que os testes passem (*green*), e o código é refatorado com nova execução dos testes para verificar a preservação dos comportamentos (*refactor*). No segundo ponto de controle, uma pessoa revisa o código e os resultados antes da integração, pois a geração assistida por IA é tratada como apoio ao desenvolvimento, e não como substituta da revisão humana (**??**).

3.5.4 Scrum

De certo, um dos principais *frameworks* para organização e desenvolvimento é o Scrum, o mesmo é utilizado por equipes que usam *sprints* iterativas a fim de enfrentar projetos de forma incremental, ou seja, a fim de solucionar um problema geral, o Scrum divide partes desse problema em *sprints*, com esse processo, é gerado o artefato "Sprint Backlog"(SCHWABER; SUTHERLAND, 2013). Sendo assim, a Figura 7 apresenta a esquematização das *sprints* utilizada neste projeto.

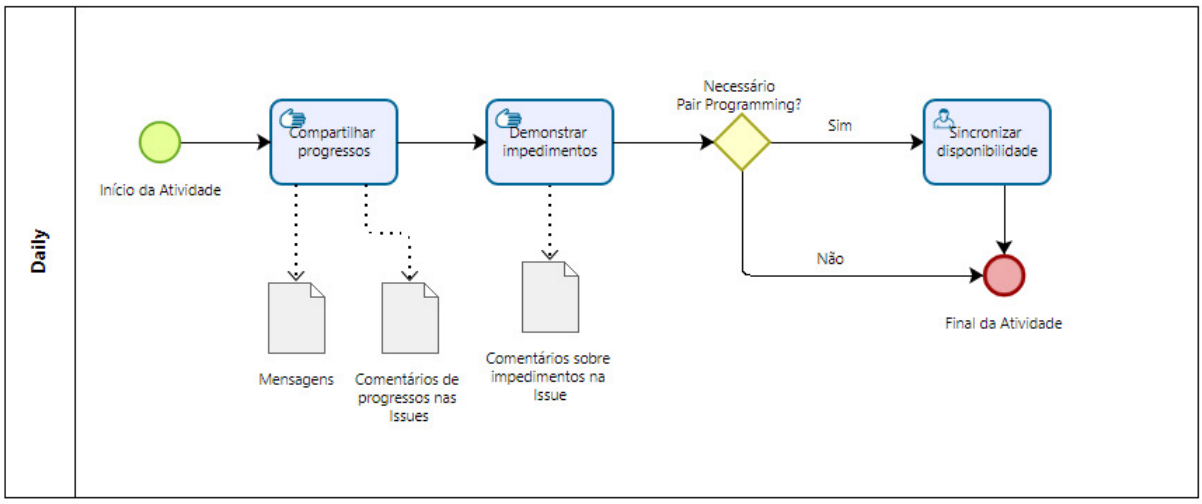
Figura 7 – BPMN Sprint.



Fonte: autoria própria.

Além disso, o Scrum se baseia em três importantes pilares, sendo eles a transparência, adaptação e inspeção. A transparência é o fator que garante que os integrantes do time saibam o que está sendo feito, para isso, durante as *sprints*, é realizado um processo chamado *daily*, conforme visualizado na Figura 8, no qual os membros compartilham seus progressos, impedimentos e planos (SCHWABER; SUTHERLAND, 2013).

Figura 8 – BPMN Daily.



Fonte: autoria própria.

A adaptação é outro fator fundamental para o Scrum, o mesmo pode ser adaptado em diversos cenários, idealmente a metodologia Scrum é realizada em um time com 3 a 10 pessoas. Entretanto, devido ao grande número de empresas com somente um único desenvolvedor, foi criado o Scrum Solo, uma adaptação do Scrum e do Personal Software Process, com o objetivo de obter os benefícios do Scrum para equipes com uma única pessoa (PAGOTTO et al., 2016).

Sendo assim, é possível adaptar elementos do Scrum para desenvolvimento em dupla, se baseando no conceito de Scrum Solo. O mínimo de membros para o Scrum se deve aos três principais papéis que o mesmo possui: o *product owner*, *scrum master* e *dev team*. Logo, a fim de ocupar todos os papéis, é necessário que ambos da dupla ocupem a posição de *dev team* e outra arbitrária (SCHWABER; SUTHERLAND, 2013).

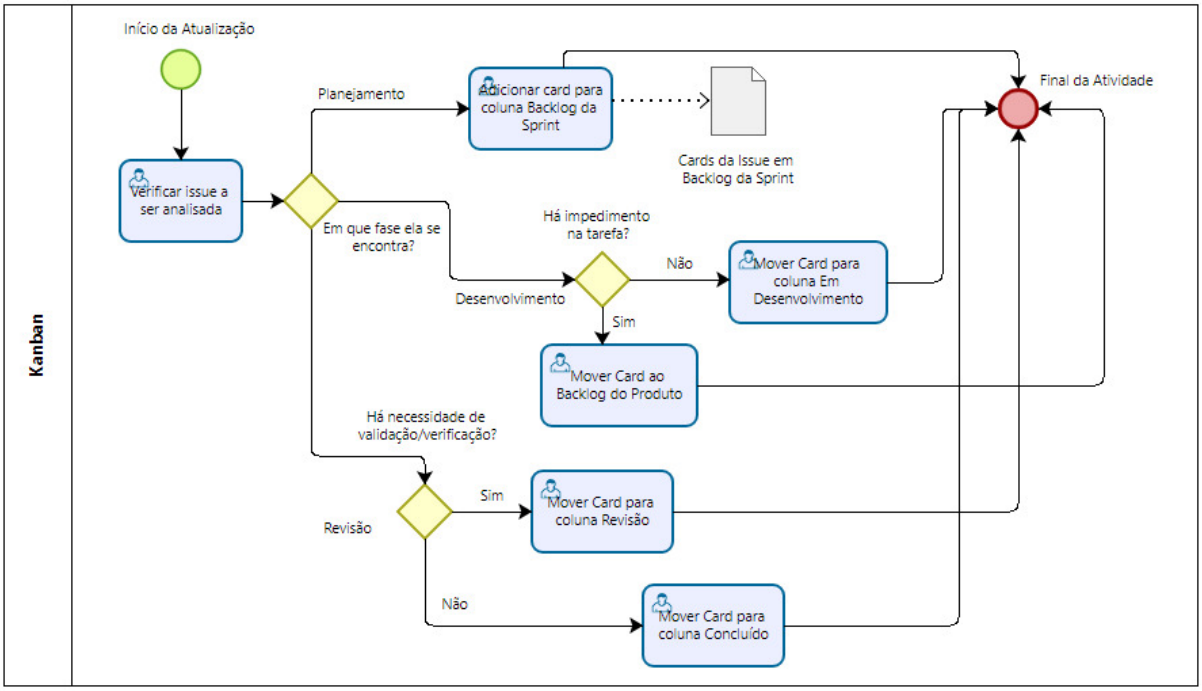
3.5.5 Kanban

A fim de favorecer a inspeção, um dos pilares do Scrum, é recomendado o uso do Kanban, devido à sua capacidade de fornecer uma visão clara sobre as tarefas em andamento. O Kanban é um sistema de gestão visual que utiliza cartões para exibir a situação geral das tarefas. Ainda mais, o Kanban melhora o desempenho em 50

Sendo assim, é possível definir o Kanban em três tipos principais: produção, mo-

vimentação e E-Kanban. O Kanban de Produção, que possui o intuito central de mostrar o quadro de desenvolvimento de um projeto em três baias: *todo*, *doing* e *done*, que representam as tarefas a fazer, em desenvolvimento e feitas respectivamente, cuja lógica pode ser visualizada na figura 9.

Figura 9 – BPMN Kanban.



Fonte: autoria própria.

Ademais, existe o Kanban de Movimentação para gerenciar processos em produção e, por fim, o E-Kanban, que é uma versão online do Kanban de produção, utilizando softwares para auxiliar a visualização do quadro de tarefas.

Em suma, as tarefas a serem disponibilizadas no Quadro Kanban encontram-se no Backlog do Produto, no qual estão todas as tarefas previstas para o desenvolvimento do projeto proposto. Dessa forma, o uso combinado do Kankan com ferramentas de *sprints* contribuirá para a organização e acompanhamento das tarefas (SKARIN, 2015).

3.5.6 Lean Inception

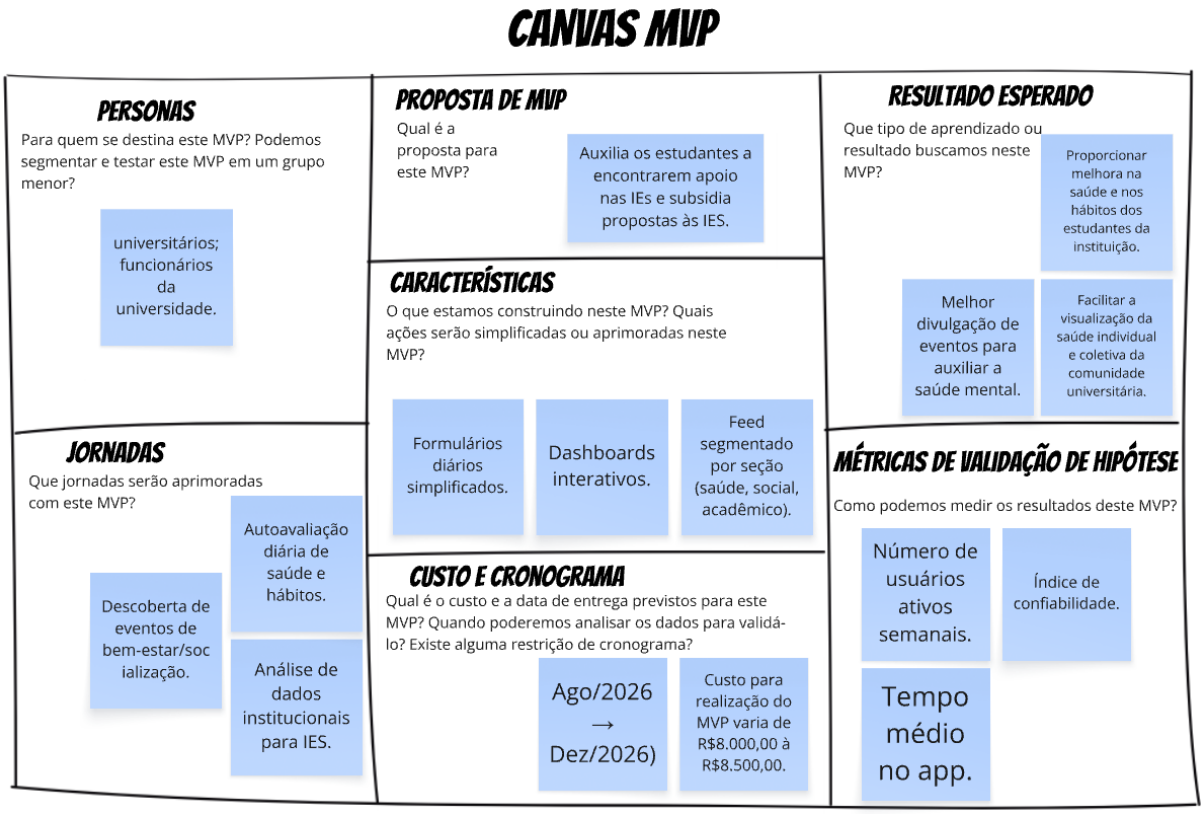
É necessário um método para a definição do mínimo produto viável (MVP). Nesse sentido, a metodologia Lean Inception se adequa a esse propósito, tendo sido desenvolvida por Paulo Caroli em 2017, conhecido por sua grande influência nas metodologias ágeis. A metodologia é amplamente utilizada para maximizar o tempo dedicado à elicitação de requisitos, partindo do mapeamento de jornadas e personas para, em seguida, priorizar as funcionalidades essenciais e definir a versão mínima do produto (CAROLI, 2017).

A metodologia consiste em quatro etapas distintas, que garantem que o time terá uma visão compartilhada para o produto ao longo do processo: visão de produto, identificação de usuários e necessidades, priorização de funcionalidades e, por fim, organização das funcionalidades mínimas do produto por meio dos artefatos "Sequenciador" e "Canvas MVP" (CAROLI, 2017). A aplicação desta metodologia exige um ambiente colaborativo e, com esse propósito, foi utilizado um template desenvolvido na plataforma Miro por Caroli et al. (2021).

Como resultado da primeira etapa, foi definida a visão geral do produto: um PWA (*Progressive Web App*), gratuito e *open source*, voltado a estudantes que carecem de assistência à saúde no meio universitário e a profissionais das Instituições de Ensino Superior (IES) que precisam monitorar seus alunos. O produto, denominado Seed, tem como diferencial auxiliar os indivíduos em questões de saúde mental por meio da coleta de dados e da oferta de informações condizentes, além de fomentar a prática de hábitos saudáveis e conectar os estudantes aos pontos de apoio disponíveis em suas IES. Esse último aspecto o diferencia de aplicativos como o Daylio, que atuam apenas na adoção de hábitos saudáveis, sem oferecer suporte institucional aos usuários.

Ao final da metodologia, é elaborado o Canvas MVP, que consolida as principais definições do produto mínimo viável em sete blocos: proposta do MVP, personas segmentadas, jornadas, funcionalidades, resultado esperado, métricas para validar as hipóteses de negócio, e custos e cronograma (CAROLI, 2017). A Figura 10 apresenta o Canvas MVP elaborado para o projeto.

Figura 10 – Canvas MVP (Seed).



Fonte: autoria própria com base em Caroli (2017).

Os demais artefatos produzidos durante a aplicação da metodologia, Visão de Produto, É - Não É - Faz - Não Faz, Objetivos do Negócio, Personas, Jornadas de Usuário, Brainstorming de Funcionalidades, Revisão e Sequenciador, encontram-se detalhados no Apêndice A.

3.5.7 Questionário

O questionário é uma ferramenta eficiente para a obtenção de dados quantitativos e qualitativos de um público-alvo extenso (VAZQUEZ; SIMÕES, 2016). Com o intuito de complementar o escopo do produto a ser desenvolvido com base na perspectiva dos possíveis usuários, foi elaborado o questionário apresentado no Apêndice B.

A primeira pergunta vista na Figura 24 tem como objetivo filtrar o público alvo do formulário. Dessa forma as respostas coletadas das questões subsequentes restringem-se apenas à parte interessada - esta que está inserida na cenário universitário. Além disso, optou-se por colher os dados da comunidade atual a fim de analisar o perfil presente dos indivíduos inseridos no contexto universitário.

As duas seguintes perguntas, observadas nas Figuras 25 e 26, compõem a primeira seção do formulário e estão relacionadas ao tempo de permanência na instituição e ao

respectivo campus do participante do formulário aplicado. Tais informações são relevantes para a análise prévia da presença de correlações entre estes dados com as autoavaliações de determinantes da saúde do colaborador - estas representadas nas Figuras 27, 28 e 29, que constituem a segunda seção do formulário.

Assim, os determinantes da saúde relevantes e considerados acessíveis de serem tratados por projetos elaborados pela universidade foram avaliados. Ademais, foram coletados dados qualitativos a respeito da presença de demais variáveis da saúde, que identificaram outros tópicos, sendo eles: **(1)** acessibilidade na instituição; **(2)** vida financeira; **(3)** acesso a cultura e **(4)** vida amorosa. Esses resultados estão ilustrados nas Figuras 30 e 31.

Por fim, na terceira seção do inquérito, torna-se evidente a necessidade do desenvolvimento de uma alternativa tecnológica de promoção da saúde na Universidade de Brasília (UnB). Isso porque é possível verificar na Figura 32 que 58% dos participantes acreditam que sua instituição de ensino não possui serviços de apoio a saúde mental ou de fomentação de bons hábitos de vida. Tendo em vista que nesta coleta de dados, apenas um dos interrogados não está inserido na UnB, trata-se de um dado alarmante, já que esta é formalmente vista como promotora da saúde.

A justificativa do desenvolvimento deste software também se alicerça na Figura 33 que mostra uma aceitação de 58% dos participantes em relação a um aplicativo voltado à saúde mental e à manutenção de hábitos de vida. Da mesma forma, os dados previamente vistos nas Figuras 27 e 29, que exibem uma avaliação mediana na saúde geral e preocupante no estado mental dos participantes, demonstram tal necessidade.

Isto posto, foi avaliado na Figura 34 as funcionalidades que esse sistema pode dispor. As sugestões exibidas na Figura 35 foram adaptadas ao escopo inicial estipulado com o intuito de obter êxito neste projeto e contribuir para a promoção da saúde na Universidade de Brasília.

3.5.7.1 Análise de Confiabilidade

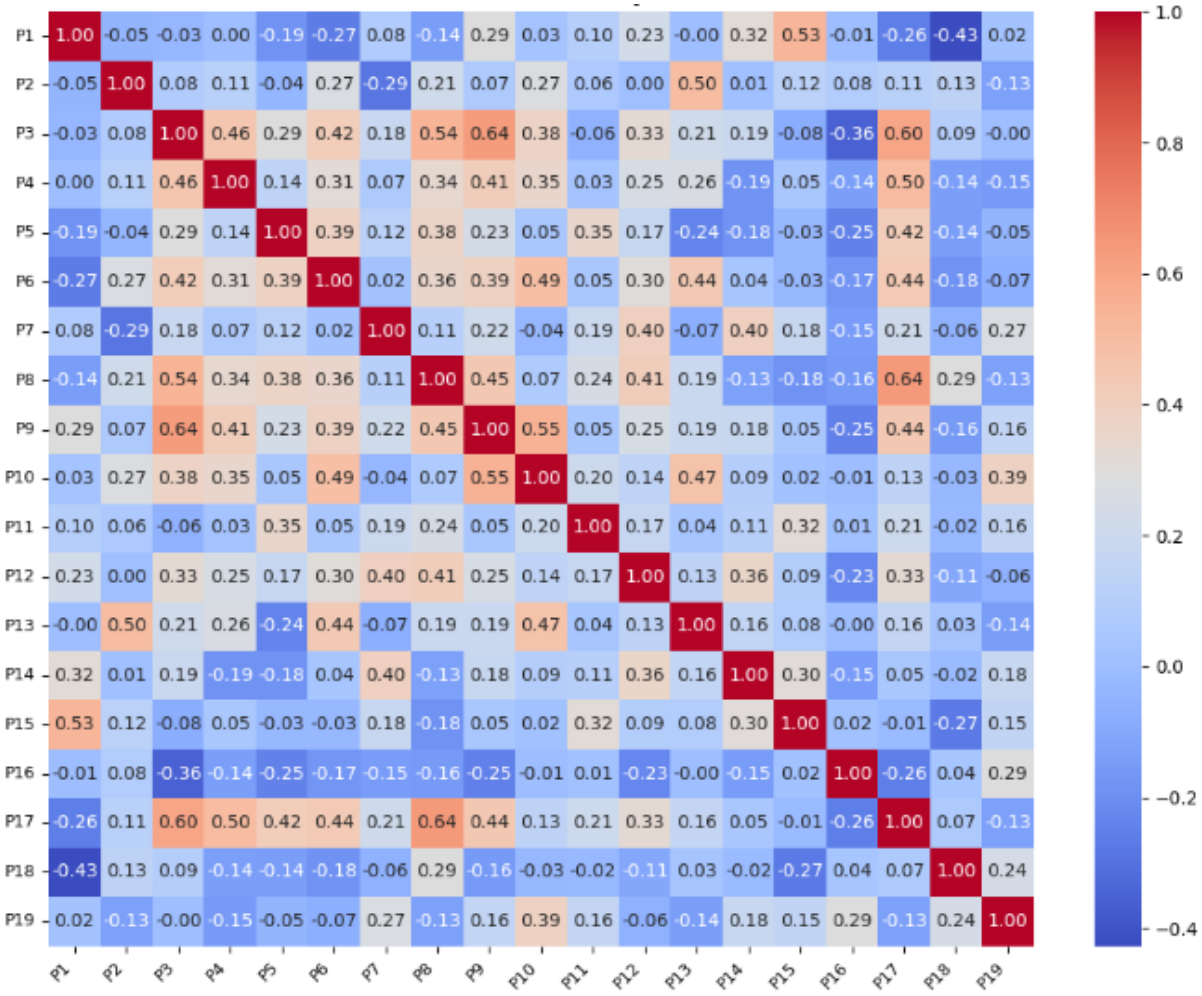
Um aspecto fundamental da avaliação do questionário proposto é a análise de sua confiabilidade. Para isso, foi calculado o coeficiente Alfa de Cronbach, cujo resultado foi de 0.6949, com intervalo de confiança de [0.519 0.828], indicando um nível aceitável de consistência interna para a utilização dos dados coletados.

Além disso, foi realizada uma análise de correlação entre os itens do questionário, conforme apresentado na Figura 11, cujas perguntas utilizadas estão listadas a seguir:

- **P1** – Há quanto tempo você faz parte da comunidade universitária?

- **P2** – Caso você esteja inserido na instituição da UnB, de qual campus você faz parte?
- **P3** – Alimentação
- **P4** – Hidratação
- **P5** – Autoconhecimento
- **P6** – Vida Acadêmica
- **P7** – Mobilidade (facilidade de logística na rotina)
- **P8** – Saúde Mental
- **P9** – Acesso à saúde
- **P10** – Condição de moradia
- **P11** – Convívio Social (amigos, família e companheiros)
- **P12** – Exercício físico
- **P13** – Realização de hobbies
- **P14** – Uso de Redes Sociais
- **P15** – Uso de substâncias (álcool, cigarro, drogas ilícitas)
- **P16** – Prática de algum vício
- **P17** – No geral, como você avalia a sua saúde? (1 estrela para péssimo até 5 estrelas para excelente)
- **P18** – Você acredita que a sua instituição oferece serviços de apoio à saúde mental ou de fomentação de bons hábitos de vida?
- **P19** – Você usaria um aplicativo da sua instituição universitária para o acompanhamento da sua saúde mental e a manutenção de hábitos de vida?

Figura 11 – Matriz de correlação do questionário.



Fonte: autoria própria.

Em suma, a matriz de correlação evidencia a relação entre as variáveis, permitindo identificar possíveis padrões de associação entre os aspectos avaliados. Observa-se, por exemplo, uma correlação moderada entre os itens **P3** (Alimentação) e **P4** (Hidratação), bem como entre **P8** (Saúde Mental) e **P17** (Avaliação geral da saúde), o que sugere uma relação entre o bem-estar físico e psicológico.

Vale ressaltar, contudo, que o questionário utilizado possui natureza exploratória e estrutura simples, o que pode limitar a profundidade das análises. Ainda que os resultados indiquem padrões confiáveis dentro da amostra coletada, não se pode generalizar integralmente as conclusões para toda a população universitária.

3.6 Requisitos

Os requisitos são especificações do sistema que visam suprir necessidades de usuários, sendo a elicitação de requisitos uma etapa crucial para o desenvolvimento de software. Esta é uma etapa fundamental para a engenharia de requisitos (REINEHR, 2020). Se-

gundo Sommerville (2011), esse processo deve ser realizado de forma sistemática com diversas técnicas, como documentação, validação, negociação, entre outras.

Sendo assim, para o projeto proposto foram realizadas personas, brainstorming e histórias de usuário para elicitar novos requisitos. Além disso, MoSCoW e revisão de funcionalidades foram utilizadas para priorização dos requisitos levantados e a classificação dos requisitos obtidos foi realizada utilizando o método FURPS+, proporcionando uma visão ampla das necessidades dos usuários e do sistema.

3.6.1 Requisitos Funcionais

Os requisitos funcionais representam as funcionalidades que o sistema deve oferecer para atender às necessidades explícitas dos usuários. Em essência, eles descrevem comportamentos e serviços que o sistema deverá executar, além de regras de negócio relacionadas ao contexto do sistema, e sua implementação resulta diretamente em entregas perceptíveis ao usuário final (REINEHR, 2020).

3.6.1.1 Histórias de Usuário

As histórias de usuário têm como principal objetivo descrever, de forma clara e concisa, funcionalidades necessárias do ponto de vista do usuário. Outro aspecto importante é a possibilidade de dividi-las em épicos, sendo grandes conjuntos de funcionalidades relacionadas que representam macro-requisitos do sistema, e *features*, sendo funcionalidades específicas derivadas dos épicos, com escopo mais restrito e bem definido (REINEHR, 2020).

Além disso, a fim de proporcionar maior controle sobre as funcionalidades criadas, é necessário um método de priorização. O método escolhido para este projeto foi o MoSCoW, que classifica os requisitos funcionais nas seguintes categorias (CARVALHO et al., 2021):

- **Must**: requisitos que o sistema obrigatoriamente deve ter;
- **Should**: requisitos importantes, mas que podem ser postergados caso necessário;
- **Could**: requisitos desejáveis, cuja implementação depende da disponibilidade de tempo e recursos;
- **Won't**: requisitos que, embora desejados, não serão implementados neste momento.

Sendo assim, os tópicos a seguir representam funcionalidades desenvolvidas usando histórias de usuário e priorizadas com MoSCoW. Cada funcionalidade é identificada por um código no formato **FX** para representar uma *feature*, e **USX** para indicar a respectiva história de usuário associada.

3.6.1.1.1 Épico - Gerenciamento de registro do usuário.

- **Feature F01**: registro do usuário.

  US1 Eu, como usuário, desejo me cadastrar no aplicativo.

  – Prioridade: deve ter.

- **Feature F02**: registro do funcionário.

  US2 Eu, como funcionário, desejo me cadastrar no aplicativo.

  – Prioridade: deve ter.

- **Feature F03**: edição de dados do usuário ou funcionário.

  US3 Eu, como usuário ou funcionário, desejo editar meus dados no aplicativo.

  – Prioridade: deve ter.

- **Feature F04**: exclusão de dados do usuário ou funcionário.

  US4 Eu, como usuário ou funcionário, desejo deletar os meus dados do aplicativo.

  – Prioridade: deve ter.

3.6.1.1.2 Épico - Gerenciamento de formulários.

- **Feature F05**: registro de formulário.

  US5 Eu, como usuário, desejo registrar diariamente minha avaliação do meu estado em cada seção do bem-estar.

  – Prioridade: deve ter.

  US6 Eu, como funcionário, desejo visualizar os dados registrados pelos usuários.

  – Prioridade: deve ter.

- **Feature F06**: visualização de dados do formulário.

  US7 Eu, como usuário, desejo visualizar meus formulários respondidos a qualquer momento.

  – Prioridade: deveria ter.

- **Feature F07**: estatísticas de formulários respondidos.

  US8 Eu, como usuário, desejo visualizar as estatísticas dos formulários respondidos em forma de gráfico a qualquer momento.

  – Prioridade: deveria ter.

- **Feature F08**: gerenciamento de formulários.

  US9 Eu, como funcionário, desejo criar formulários personalizados.

  – Prioridade: poderia ter.

  US10 Eu, como funcionário, desejo editar formulários à medida que analiso as perguntas.

  – Prioridade: deveria ter.

- **Feature F09**: bloco de anotações.

  US11 Eu, como usuário, desejo adicionar anotações livres para os formulários respondidos.

  – Prioridade: poderia ter.

3.6.1.1.3 Épico - Gerenciamento de comunicação.

- **Feature F010**: geração de notificações.

  US12 Eu, como usuário, gostaria de receber notificações para preenchimento dos formulários diários.

  – Prioridade: poderia ter.

- **Feature F11**: troca de senha.

  US13 Eu, como usuário, desejo receber a confirmação de troca de senha por e-mail.

  – Prioridade: deveria ter.

- **Feature F12**: gerenciamento de avisos.

  US14 Eu, como usuário, gostaria de receber avisos sobre o status do aplicativo por e-mail.

  – Prioridade: poderia ter.

- **Feature F13**: avisos de eventos.

  US15 Eu, como usuário, gostaria de receber avisos sobre possíveis eventos dentro do calendário do aplicativo.

  – Prioridade: poderia ter.

3.6.1.1.4 Épico - Gerenciamento de eventos.

- **Feature F014**: gerenciar evento.

  US17 Eu, como funcionário, desejo ser capaz de registrar eventos no calendário do aplicativo.

  – Prioridade: deveria ter.

  US18 Eu, como funcionário, desejo ser capaz de editar informações de eventos no calendário do aplicativo.

  – Prioridade: deveria ter.

  US19 Eu, como funcionário, desejo ser capaz de excluir eventos no calendário do aplicativo.

  – Prioridade: deveria ter.

- **Feature F15**: visualizar evento.

  US20 Eu, como usuário, desejo visualizar eventos no calendário do meu aplicativo.

  – Prioridade: deveria ter.

3.6.1.1.5 Épico - Gerenciamento do feed.

- **Feature F016**: gerenciar feed.

  US21 Eu, como funcionário, desejo ser capaz de registrar publicações no feed de cada seção do bem-estar.

  – Prioridade: deveria ter.

  US22 Eu, como funcionário, desejo ser capaz de editar publicações no feed de cada seção do bem-estar.

  – Prioridade: deveria ter.

US23 Eu, como funcionário, desejo ser capaz de excluir publicações no feed de cada seção do bem-estar.

  - Prioridade: deveria ter.

- **Feature F017**: visualizar o feed.

  US24 Eu, como usuário, desejo ser capaz de visualizar o feed para cada seção do bem-estar.

  - Prioridade: deveria ter.

Sendo assim, com base nos requisitos elicitados pelo Lean Inception com Personas e Brainstorming de Funcionalidades, juntamente com os requisitos elicitados pelas Histórias de Usuário, priorizados por Revisão de Funcionalidades e MoSCoW, logo a tabela 2 representa os requisitos funcionais para o projeto proposto.

Tabela 2 – Requisitos Funcionais

ID	**Descrição**
FUNC01	Cadastro de usuário no aplicativo.
FUNC02	Cadastro de funcionário no aplicativo.
FUNC03	Edição dos dados cadastrais de usuários e funcionários.
FUNC04	Exclusão dos dados de usuários e funcionários.
FUNC05	Registro diário de formulários pelos usuários.
FUNC06	Visualização de dados registrados por usuários (funcionário).
FUNC07	Visualização dos formulários respondidos (usuário).
FUNC08	Geração de estatísticas dos formulários respondidos.
FUNC09	Criação e edição de formulários personalizados (funcionário).
FUNC10	Inclusão de anotações livres associadas aos formulários.
FUNC11	Envio de notificações para preenchimento dos formulários.
FUNC12	Confirmação de troca de senha via e-mail.
FUNC13	Envio de avisos sobre o status do aplicativo por e-mail.
FUNC14	Envio de avisos sobre eventos no calendário do aplicativo.
FUNC15	Registro de eventos no calendário (funcionário).
FUNC16	Edição e exclusão de eventos do calendário (funcionário).
FUNC17	Visualização dos eventos no calendário (usuário).
FUNC18	Registro de publicações no feed por seção do bem-estar.
FUNC19	Edição e exclusão de publicações no feed (funcionário).
FUNC20	Visualização do feed por seção do bem-estar (usuário).
FUNC21	Exportação dos dados dos formulários via CSV.

ID	Descrição
FUNC22	Login com Google (OAuth 2.0).
FUNC23	Geração de análises e estatísticas instituicionais.

Fonte: autoria própria.

3.6.2 Requisitos Não Funcionais.

Requisitos não funcionais são aqueles que não são mapeados facilmente por meios convencionais, ou seja, são características do sistema que não estão relacionadas diretamente com as necessidades dos usuários (SOMMERVILLE, 2011). Sendo assim, Sommerville (2011) classifica esses requisitos como desempenho, segurança, usabilidade, disponibilidade, entre outros, sendo esses não perceptíveis pelo usuário, mas essenciais para o sistema.

3.6.2.1 FURPS+

FURPS é um modelo de classificação de requisitos de software criado para o Rational Unified Process, cujo principal objetivo é o conjunto de melhores práticas para organizar o processo de construção de software. Sendo assim, no FURPS, os requisitos são divididos em cinco principais categorias: (F)unctionality, (U)sability, (R)eliability, (P)erformance, (S)upportability (CAMPOS, 2008).

Além disso, existe uma extensão conhecida como **FURPS+**, onde é implementado o FURPS convencional com um tipo extra de classificação de requisitos, podendo ser requisitos de interfaces, implementações e físicos, como restrições de hardware, por exemplo (SEQUEIRA, 2017).

- **F: Funcionalidade**: Aspectos funcionais do software, esses requisitos foram mapeados pelas histórias de usuário, personas, entre outros.
- **U: Usabilidade**: Usabilidade do sistema, incluindo experiência do usuário (UX).
- **R: Confiabilidade**: Confiabilidade do sistema, como estabilidade e resistência a falhas.
- **P: Performance**: Performance do sistema, como velocidade e eficiência.
- **S: Suportabilidade**: Avalia a facilidade com que o sistema pode ser mantido e suportado ao longo do tempo.
- **+**: Requisitos como ajuda e documentação de usuário, interfaces, observações legais, entre outros.

Sendo assim, a partir das especificações de requisitos não funcionais propostas por Sommerville (2011) e necessidades do sistema, é possível identificar os seguintes requisitos não funcionais para o projeto proposto:

Tabela 3 – Requisitos Não Funcionais Classificados Pelo FURPS+.

ID	**Descrição**
USAB01	O sistema deve oferecer suporte a vários idiomas.
USAB02	O app deve responder imediatamente às ações do usuário, mesmo com carregamentos.
USAB03	O sistema deve seguir um padrão simples para facilitar o aprendizado.
USAB04	As interações devem ter feedback visual ou sonoro para orientar o usuário.
USAB05	O app deve mostrar mensagens claras em caso de erro, sem atrapalhar o uso.
CONF01	O usuário pode alterar seus dados pessoais.
CONF02	O app deve seguir a Lei Geral de Proteção de Dados (LGPD).
CONF03	O sistema deve funcionar 24 horas por dia, 7 dias por semana.
CONF04	Deve existir um plano de backup para recuperação em caso de falhas.
PERF01	O sistema deve suportar acessos simultâneos de diferentes usuários.
PERF02	O aplicativo deve realizar autenticação no tempo médio de 10 segundos.
PERF03	O tempo de resposta deve ser inferior a 2 segundos.
PERF04	O sistema deve aguentar um aumento de 20% no número de usuários.
SUPO01	O app deve estar disponível para Android e iOS, com interface semelhante.
SUPO02	O app deve se adaptar a diferentes dispositivos, como celulares e tablets.
SUPO03	O sistema deve seguir padrões de segurança reconhecidos, como ISO 27001.
PLUS01	O sistema deve oferecer suporte online, dicas e manual acessível.

Fonte: autoria própria.

3.7 Suporte Tecnológico

Nesta seção serão apresentados os recursos tecnológicos usados para o desenvolvimento do projeto, apresentados na Tabela 4.

Tabela 4 – Recursos Tecnológicos

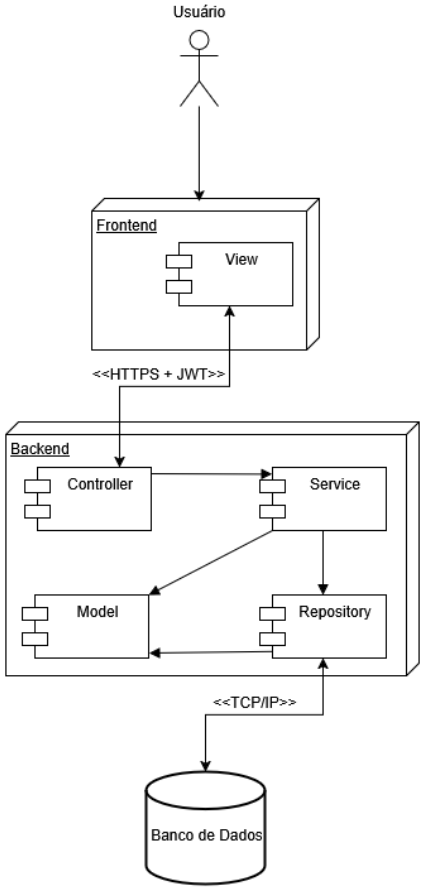
Tecnologia	**Uso no projeto**
Figma e Figma Make	Elaboração do protótipo de alta fidelidade e geração prevista do software web a partir do design (FIGMA, 2025).
Ferramentas de diagramação (Miro, Draw.io, BrModelo e Bizagi)	Elaboração dos diversos diagramas do projeto, como Lean Inception (CAROLI, 2017), UML (DRAW.IO, 2025), entidade-relacionamento e processos de negócio (BPMN).
Spring (Spring Boot, Spring Data e Hibernate)	Framework Java utilizado no back-end, incluindo configuração automática, ORM e demais APIs do ecossistema (WEISSMANN, 2014).
Git e GitHub	Versionamento e hospedagem dos repositórios do projeto, sendo eles Monografia, Front e Service (GITHUB, 2025).
Claude Code	Apoio ao desenvolvimento do software, auxiliando na escrita, revisão e depuração de código.

Fonte: autoria própria.

3.8 Arquitetura

A arquitetura segue o modelo Cliente-Servidor, sendo o *frontend* o cliente e o *backend* o servidor, a representação aquitetural a Figura 12 demonsta como é a interação entre essas camadas.

Figura 12 – Representação arquitetural.



Fonte: autoria própria com base em (WEISSMANN, 2014).

3.8.1 MVC (Model-View-Controller)

O MVC é um padrão arquitetural que permite isolar completamente as camadas de negócio e visualização através de uma camada intermediária chamada controlador. Um problema recorrente que o MVC visa solucionar é o alto acoplamento do código, logo, a implementação do mesmo produz código com menos dependências, maior qualidade e testabilidade (WEISSMANN, 2014). Isso ocorre pois as camadas são independentes entre si, ou seja, as camadas não possuem acesso direto entre si, o que torna a implementação da substituição das mesmas mais fácil.

Sendo assim, podemos descrever as funcionalidades das camadas da seguinte maneira. *View* é a camada de apresentação para o usuário, onde o mesmo irá interagir com o sistema. No caso desse projeto, será o *frontend*. As próximas duas camadas fazem parte do *backend*, o *Model* é a camada responsável por aplicar regras de negócios e consulta ao banco de dados. Por fim, o *Controller* é responsável por intermediar a *View* e *Model*.

3.8.2 Backend

O *backend* representa o núcleo do sistema, sendo responsável por intemerdiar a comunicação entre o *frontend* e o banco de dados. O mesmo, processa requisições, aplica regras de negócio, realiza validações e retorna a resposta para o usuário, garantindo operações de forma segura e autenticada no banco de dados.

Ademais, para a organização de pastas, optou-se por adotar a separação entre camadas Controller, Service, Model e Repository, uma vez que, por convenção, projetos utilizando o Spring Framework costumam seguir essa estrutura, facilitando a integração com rcursos como Beans e ORMs, além de favorecer a manutenção e coesão (WEISSMANN, 2014).

- Controller - Controle de requisições da API RESTful.
- Service - Implementação de regras de negócio do sistema.
- Model - Entidades mapeadas do SGBDR.
- Repository - Interfaces de manipulação do SGBDR.

3.8.2.1 API RESTful

A comunicação do *backend* com o *frontend* será por meio das chamas *Application Programming Interfaces Representational State Transfer* (APIs RESTful), especificamente as APIs RESTful utilizam o protocolo HTTP/1.1 como base para a comunicação entre cliente e servidor, utilizando o padrão de métodos *get*, *post*, *delete* e *put*, além disso, a troca de dados padrão envolve JSON (*JavaScript Object Notation*).

Essa comunicação deve ser criptografada e autenticada, logo é importante a utilização do *Transport Layer Security* (TSL) para criptografia e para a autorização é conveniente a utilização o Spring Security com suporte ao OAuth 2.0, visto que o mesmo permite a integração com provedores de identidade externos como Google, Facebook, entre outros. Também, é necessário a utilização do *JSON Web Token* (JWT) para transferir tokens, de forma segura e compacta, contendo as informações de autorização e autenticação entre os sistemas (NASCIMENTO, 2017).

3.8.2.2 Hibernate

A fim de solucionar o alto acoplamento do código o Spring utiliza o DAO (*Data Access Object*) que separa a lógica de acesso de dados da lógica de negócio da aplicação, em outras palavras o DAO oferece às classes clientes interfaces onde são expostas operações básicas como inserções, deleções e consultas. Essa lógica está separada em uma camada chamada *Repository* (WEISSMANN, 2014).

Ademais, um aspecto fundamental para manipulação de dados é a utilização de ORM (*Object Relational Mapping*) no qual oferece a possibilidade de manipular entidades do SGBDR como objetos da POO (Programação Orientada a Objetos) (WEISSMANN, 2014). Sendo assim, para esse projeto foi escolhido o Hibernate como provedor ORM devido a sua alta documentação e integração com o Spring.

3.8.3 Frontend

O *frontend* é a camada de visualização e interação com o usuário. Por isso, é essencial que essa interface seja clara, intuitiva e eficiente. Para garantir isso, o modelo proposto segue as 10 heurísticas de usabilidade de Nielsen.

Em especial, é necessário que o sistema seja padronizado e acessível em diferentes dispositivos móveis, independente do sistema operacional. Sendo assim, optou-se por utilizar o React Native, um *framework* que permite a criação de aplicações móveis multiplataforma.

3.8.3.1 React Native

React é uma biblioteca JavaScript baseada em componentes reutilizáveis, o que permite desenvolver sistemas recicláveis e independentes, favorecendo a manutenção e permitindo a escalabilidade (BODUCH, 2017).

Em suma, o React Native segue os mesmos princípios do React, porém, ao invés de gerar páginas *web*, o React Native compila os componentes em elementos nativos, tanto para o Android quanto para o IOS (*iPhone Operating System*). Sendo assim, o desempenho de aplicações em React Native é semelhante à utilização de linguagens nativas como Kotlin ou Swift (BODUCH, 2017).

3.9 Representação Visual do Sistema

Indubitavelmente, a Unified Modeling Language (UML) é a linguagem-padrão de modelagem internacional usada na engenharia de software. Um aspecto importante é que essa linguagem independe de linguagem de programação (GUEDES, 2018). Sendo assim, é possível a utilização para representação do Diagrama de Classes e Diagrama de Pacotes do *backend* e *frontend* desse projeto.

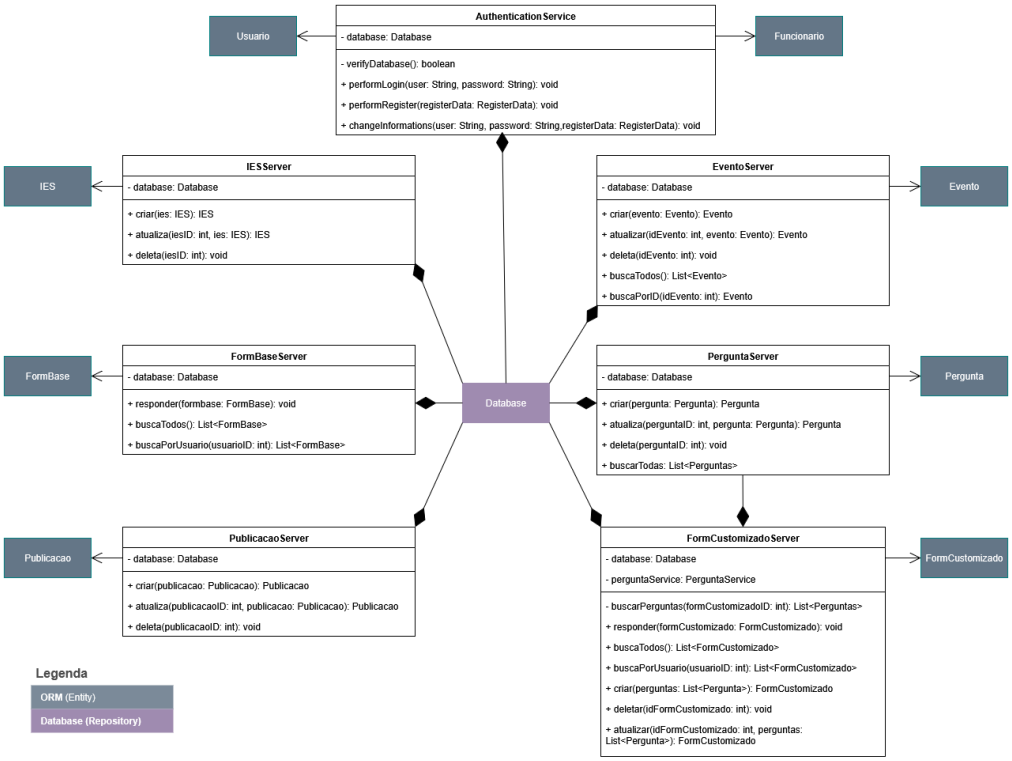
3.9.1 Diagrama de Classes

Os diagramas de classes são amplamente utilizados para representar a estrutura lógica do sistema, eles são utilizados como base para diversos outros diagramas da UML.

Nele é possível visualizar as classes, seus atributos, métodos e os relacionamentos com outras classes, como herança, associação e dependência (GUEDES, 2018).

A utilização do diagrama de classes contribui para o esclarecimento das regras de negócio do sistema. Sendo assim, a Figura 13 representa o diagrama de classe do backend onde estão dispostas as regras de negócio do projeto proposto.

Figura 13 – Diagrama de Classe do projeto.

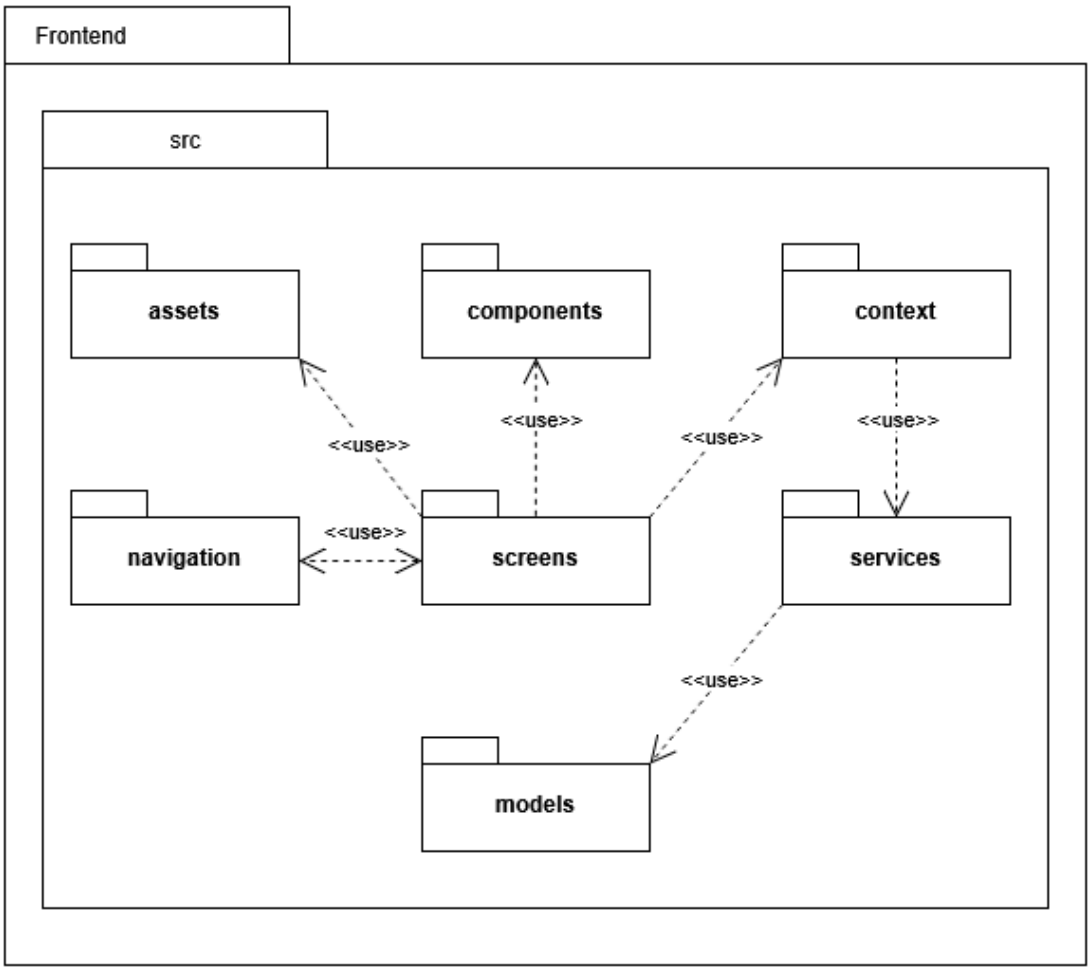


Fonte: autoria própria com base em Guedes (2018).

3.9.2 Diagrama de Pacotes

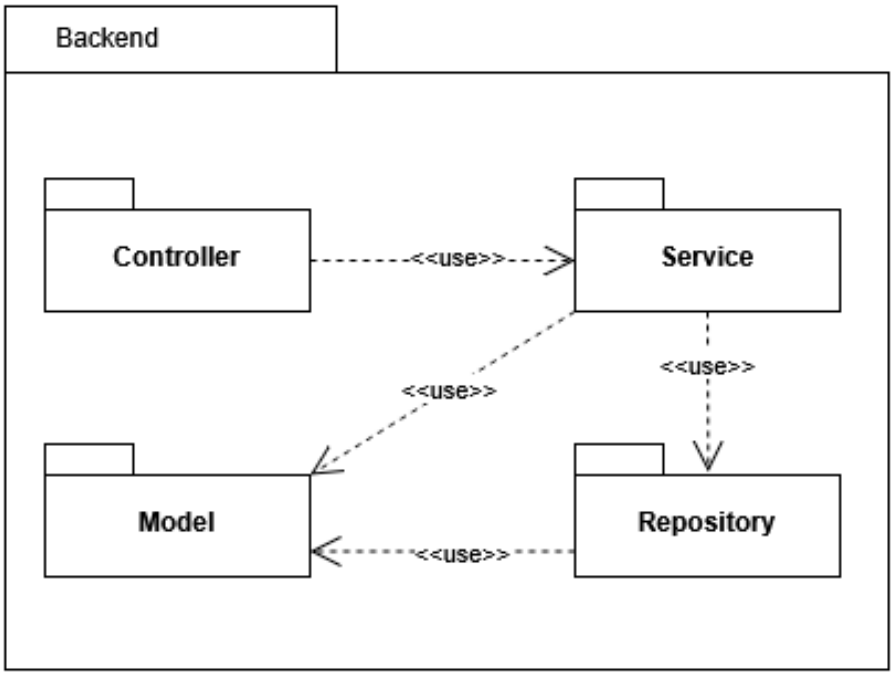
De certo, o diagrama de pacotes é a escolha ideal para demonstrar como os subsistemas ou submódulos interagem entre si. Ademais, uma das principais utilidades está na demonstração das camadas de um software ou de um processo de desenvolvimento (GUEDES, 2018). Dessa forma, as Figuras 14 e 15 apresentam a estrutura em camadas dos módulos *frontend* e *backend*, respectivamente.

Figura 14 – Diagrama de Pacotes do frontend.



Fonte: autoria própria com base em Guedes (2018).

Figura 15 – Diagrama de Pacotes do backend.



Fonte: autoria própria com base em Guedes (2018).

3.10 Projeto de Dados

O projeto de dados tem como objetivo retratar como serão persistidos os dados do sistema de forma a garantir integridade, consistência e eficiência no armazenamento das informações. Nesta etapa serão desenvolvidas as modelagens conceitual e lógica para representação do design da camada de persistência.

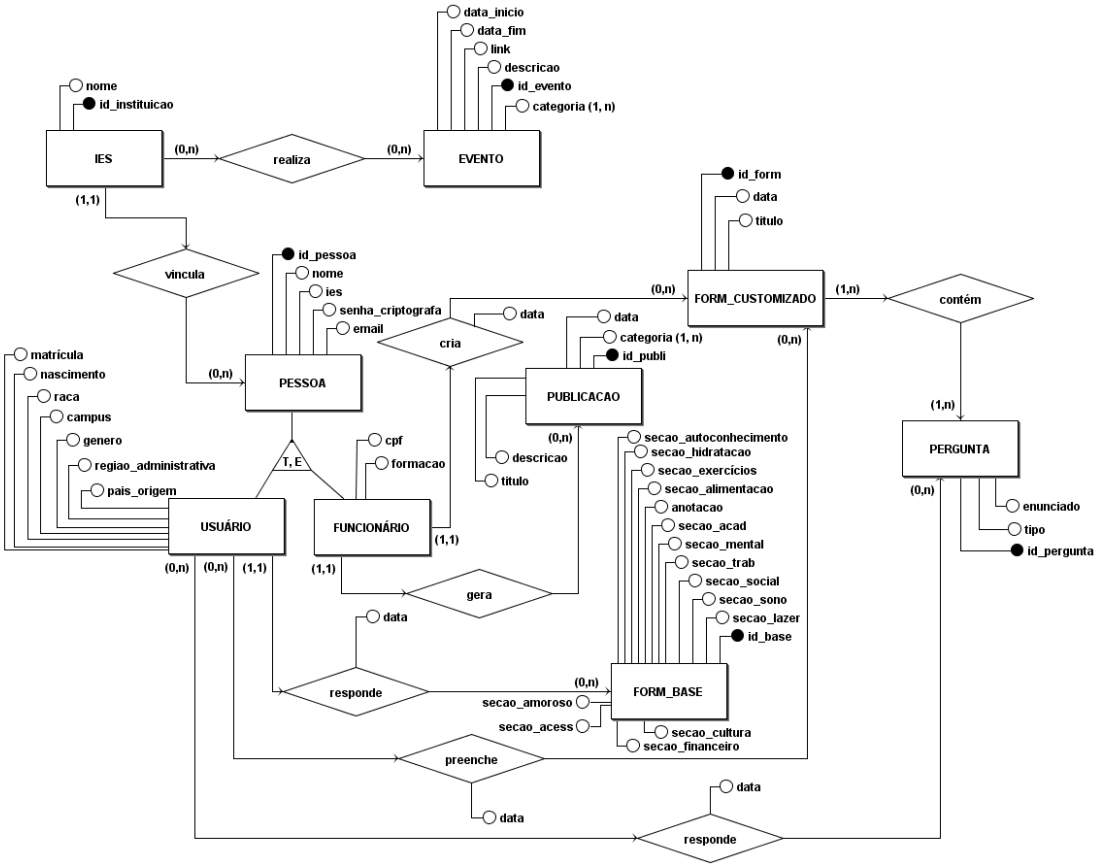
3.10.1 Diagrama Conceitual

De acordo com Elmasri e Navathe (2016) o Diagrama Entidade-Relacionamento Estendido (DEER) trata-se de uma estrutura derivada de um modelo aprimorado da modelagem Entidade-Relacionamento. Tal modelo permite representar as conexões entre entidades de um sistema e os atributos dessas entidades, além de outras especificações destinadas ao banco de dados da aplicação.

Nesta versão estendida, são incorporadas teorias de relacionamentos de classe e subclasse, em conjunto com herança de tipo, conceitos que possibilitam a representação de especializações e generalizações no diagrama (ELMASRI; NAVATHE, 2016). O Diagrama Entidade-Relacionamento Estendido desenvolvido para este projeto está apresentado na Figura 16. A notação adota tem como base os estudos de Elmasri e Navathe (2016) com

adaptações necessárias realizadas em função das limitações de correspondência de notação da ferramenta brModelo.

Figura 16 – Diagrama Entidade-Relacionamento do Projeto.



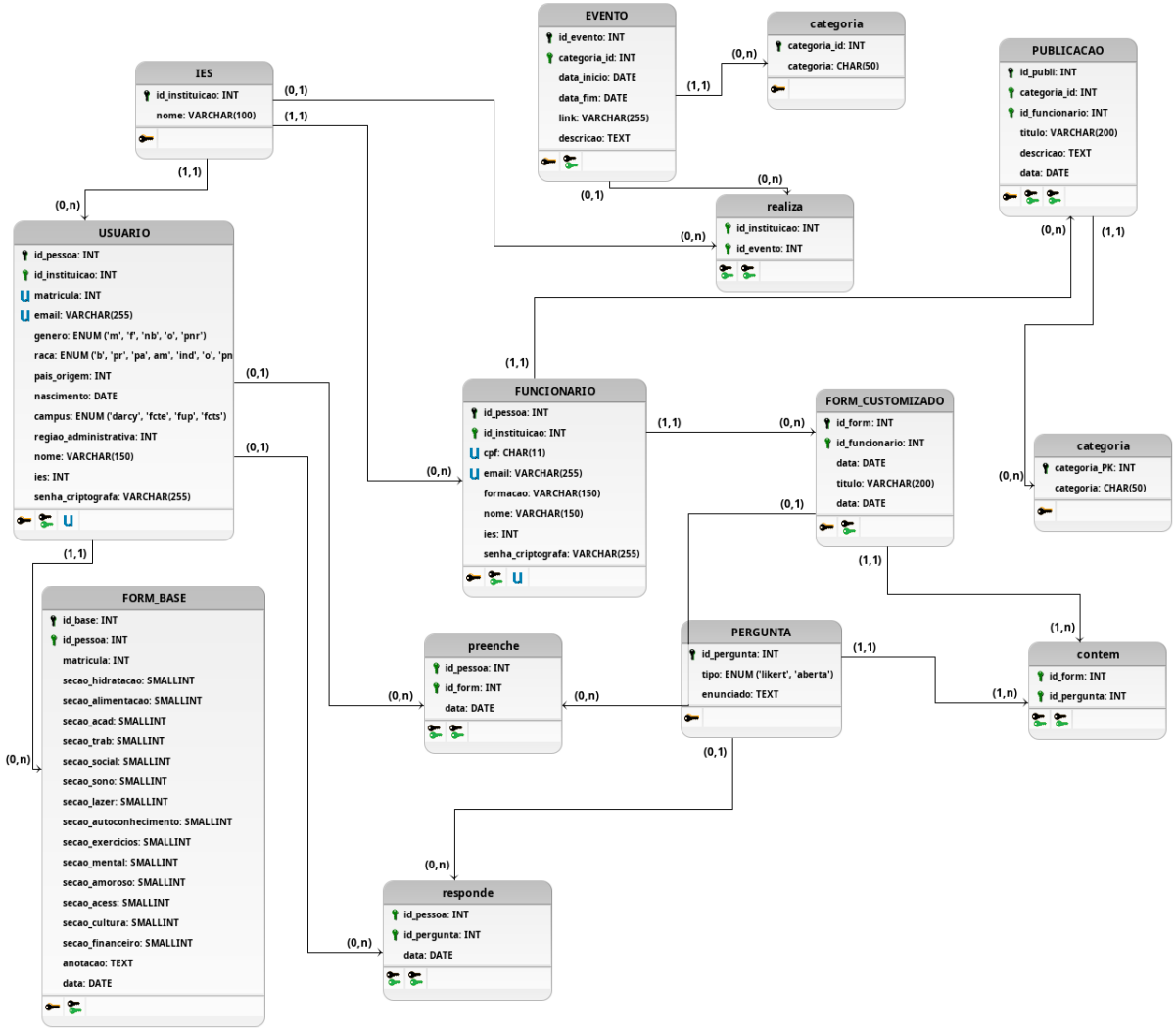
Fonte: autoria própria considerando as limitações do brModelo e com base em Elmasri e Navathe (2016).

3.10.2 Diagrama Lógico

O modelo lógico, de acordo com Heuser (2009) "é uma descrição de um banco de dados no nível de abstração visto pelo usuário do Sistema Gerenciador de Banco de Dados (SGBD)". Assim, este modelo varia conforme o SGBD adotado. Neste trabalho, os tipos de dados foram definidos de forma condizente aos tipos disponíveis no PostgreSQL, tecnologia escolhida para este projeto.

Em outras palavras, neste diagrama é possível identificar de modo gráfico as tabelas que existirão no banco de dados além de suas respectivas colunas. O diagrama lógico de dados baseado no DEER mostrado anteriormente pode ser visto na Figura 17.

Figura 17 – Diagrama Lógico do Projeto.



Fonte: autoria própria com auxílio da ferramenta de modelagem brModelo.

3.11 Testes de Software

Conforme destaca Aniche (2014), teste é o processo de execução de um produto com o objetivo de verificar se ele atende às especificações definidas e funciona corretamente no ambiente para o qual foi projetado. Além disso, testes é uma etapa essencial para o desenvolvimento de software, atualmente os Estados Unidos estimam que bugs de software lhes custam aproximadamente 60 bilhões de dólares por ano (ANICHE, 2014). Sendo assim, essa etapa além de aumentar a confiabilidade e manutenção do sistema, evita que bugs sejam inseridos.

O desenvolvimento deste projeto envolve o uso de TDD, que se mostra essencial para o desempenho no desenvolvimento. Além disso segundo Aniche (2014), o TDD asse-

gura a implementação dos requisitos funcionais elicitados, pois sua abordagem promove a utilização das especificações dos requisitos diretamente nos testes.

A estratégia de testes segue a tradicional pirâmide de testes, conforme descrito por Cohn (2010). Nesse modelo, a base é composta por testes unitários, que são executados de forma automatizada para garantir a estabilidade e a correção de componentes individuais do sistema. Para o backend em Java, optou-se pelo JUnit 5, enquanto para o frontend em JavaScript, optou-se pelo Jest. Esses frameworks são escolhidos pela facilidade de implementação de mocks e pelo suporte ao snapshot testing. Além disso, a abordagem inclui testes de caixa branca, testes de integração e testes de sistema para garantir a cobertura e a qualidade em diferentes níveis da aplicação.

Embora o software teóricamente funcione em multiplos sistemas operacionais é esperado que este projeto não irá testar todos os dispositivos. Sendo assim, para realização de testes de usabilidade serão utilizados iPhones e Androids porém com número limitado de aparelhos.

3.12 Desenvolvimento Orientado por IA

Esta seção apresenta o processo de desenvolvimento assistido por agentes de IA adotado como método para a implementação do software Seed. O processo combina especificações explícitas, agentes com permissões restritas, desenvolvimento orientado a testes e verificações de segurança, mantendo as decisões de arquitetura e de requisitos sob responsabilidade do engenheiro. Trata-se de uma proposta de método, cuja efetividade será avaliada na Subseção 3.12.8.

3.12.1 Fundamentos e limitações dos modelos de linguagem

Modelos de linguagem de grande porte (*Large Language Models*, LLMs) produzem texto de forma sequencial, ou seja, a cada passo, escolhem a próxima unidade, denominada *token*, a partir de uma distribuição de probabilidades condicionada ao contexto já existente. Como essa seleção envolve amostragem, o mesmo *prompt* pode resultar em respostas diferentes, o que torna os resultados pouco padronizados (AKITA, 2026).

Além da variabilidade, observa-se nas LLMs a tendência de modelos a concordar com as premissas apresentadas pelo usuário, fenômeno designado na literatura como *sycophancy*. Na prática, perguntar a um modelo se um texto contém erros e perguntar se ele está perfeito pode produzir respostas de teor oposto (AKITA, 2026). Essa tendência justifica uma regra do processo proposto, no qual o agente não decide requisitos nem arquitetura, e as ambiguidades são devolvidas ao engenheiro por meio da etapa de esclarecimento descrita na Subseção 3.12.3.

Essas limitações adquirem maior relevância quando o modelo deixa de apenas sugerir e passa a agir. Segundo Anthropic (2026), o uso convencional de um LLM consiste em fornecer-lhe insumos por meio de *prompts* e receber sugestões, cabendo ao usuário aplicá-las. Os agentes de IA, por sua vez, têm acesso para ler, modificar e pesquisar arquivos do ambiente de desenvolvimento e, mediante autorização, executar código diretamente, como ocorre com o Claude Code instalado na máquina do desenvolvedor.

A qualidade do código gerado também constitui motivo de preocupação. Perry et al. (2023) constataram, em estudo com participantes, que aqueles que utilizaram um assistente de IA escreveram código menos seguro e, ao mesmo tempo, demonstraram maior confiança na segurança do que produziram. Esses resultados sustentam duas escolhas do processo proposto: o engenheiro permanece como responsável pelas decisões, e a verificação de segurança é automatizada e independente do agente. Para isso, são adotadas ferramentas de análise estática (*Static Application Security Testing*, SAST) e dinâmica (*Dynamic Application Security Testing*, DAST), voltadas à identificação de vulnerabilidades, e o SonarQube, voltado à avaliação da qualidade do código.

3.12.2 *Vibe Coding* x Codificação Agêntica

Popularizado por Andrej Karpathy em fevereiro de 2025, o termo *vibe coding* refere-se a programar delegando à IA a escrita do código e dispensando a sua leitura. A literatura recente tem insistido que usar um LLM, por si só, não basta para caracterizar a prática. Segundo a fronteira traçada por Simon Willison, o código gerado por um modelo, mas revisado, testado e compreendido, equivale a empregar o LLM como assistente de digitação, e não o *vibe coding* (MICHELS et al., 2026).

A postura oposta é chamada por Beck (2025) de *augmented coding* que mantem os valores da engenharia de software tradicional, como testes, código limpo e *commits* pequenos, e se difere somente quando digitado manualmente. Sapkota, Roumeliotis e Karkee (2025) separam o *vibe coding*, de caráter interativo e exploratório, da *agentic coding*, em que agentes planejam, executam, testam e iteram com pouca intervenção humana, e sugerem que os dois paradigmas sejam combinados em um ciclo centrado no humano.

Segundo Ge et al. (2025), as práticas de desenvolvimento com agentes de código podem ser agrupadas em cinco modelos: (I) automação irrestrita, em que o agente atua com mínima restrição; (II) colaboração conversacional iterativa, baseada no diálogo contínuo entre desenvolvedor e agente; (III) desenvolvimento orientado a planejamento, em que o trabalho parte de um plano previamente definido; (IV) desenvolvimento orientado a testes; e (V) desenvolvimento aprimorado por contexto, no qual se fornece ao agente informação estruturada sobre o projeto. Sendo assim, o êxito da prática não decorre apenas da capacidade do agente, mas sim, depende da engenharia de contexto, de ambientes

de desenvolvimento consolidados e da colaboração entre humano e agente. Nesse cenário é proposto o termo **codificação agêntica**, que designa o uso de agentes sob práticas do XP: TDD, integração contínua a cada *commit* e refatoração contínua o qual será utilizado para o desenvolvimento do projeto proposto discutido na Subseção 3.12.6.

3.12.3 Spec-Driven Development

No desenvolvimento convencional, a especificação costuma orientar a escrita do código e é descartada depois, o desenvolvimento orientado por especificações (*Spec-Driven Development*, SDD) inverte essa lógica (GitHub, 2025). A especificação é escrita antes de o código ser gerado com IA e passa a ser a fonte de verdade tanto para o humano quanto para o modelo (BÖCKELER, 2025). Como o termo ainda não tem definição estável, Böckeler (2025) distingue três níveis de adoção: *spec-first*, em que a especificação é escrita antes e guia a tarefa em curso; *spec-anchored*, em que é mantida após a entrega para apoiar a evolução da funcionalidade; e *spec-as-source*, em que é o único artefato editado pelo humano e o código é gerado a partir dela.

O *Spec Kit*, kit de código aberto do GitHub, estrutura o desenvolvimento em quatro etapas principais, *specify*, *plan*, *tasks* e *implement*, com etapas opcionais de esclarecimento (*clarify*) e análise (GitHub, 2025). Os insumos do desenvolvimento são as histórias de usuário e os requisitos funcionais reunidos na especificação (*feature specification*). A etapa *clarify* esclarece as lacunas dessa especificação por meio de perguntas ao engenheiro, antes do planejamento (GitHub, 2025). O plano (*plan*) reúne as decisões técnicas, como tabelas, bibliotecas e contratos de interface, e a lista de tarefas (*tasks*) divide o trabalho em unidades pequenas e revisáveis. Como os artefatos de cada *feature* seguem uma estrutura padronizada, a revisão é facilitada e torna-se possível rastrear as histórias de usuário, os requisitos funcionais, as tarefas e os testes. A Tabela 2 apresenta os requisitos funcionais do Seed que servem de insumo para o desenvolvimento orientado por especificação.

Böckeler (2025) observa que um fluxo único de SDD pode ser desproporcional para problemas pequenos e que a revisão de muitos arquivos *markdown* é trabalhosa. A autora defende que o controle sobre o que se constrói vem de passos pequenos e iterativos, e não de uma especificação extensa elaborada de antemão. O projeto proposto atende a essa recomendação ao aplicar o fluxo em ciclos curtos, uma história de usuário por vez, de modo que o sistema se forma pela entrega sucessiva de histórias de escopo reduzido, em consonância com o Scrum e o Kanban adotados. Em cada ciclo, a etapa de esclarecimento define os critérios de aceitação e os limites numéricos das tarefas, valores que fundamentam os testes (Subseção 3.12.6).

3.12.4 Ambiente de agentes

3.12.4.1 Agentes e subagentes

Para distinguir IA generativa convencional de agente, adota-se a definição da Anthropic, que classifica como sistemas agênticos tanto os *workflows*, em que LLMs e ferramentas são orquestrados por caminhos de código predefinidos, quanto os agentes, em que o LLM dirige dinamicamente o próprio processo e o uso de ferramentas (**??**). A capacidade de ler e executar arquivos diretamente no computador, sem copiar e colar no contexto da conversa, decorre do uso de ferramentas, não é a definição em si. A mesma fonte recomenda a solução mais simples possível e observa que a autonomia dos agentes implica custos maiores e risco de acúmulo de erros. [VERIFICAR: confirmar essas ressalvas no texto original.] Por isso, o processo proposto mantém pontos de controle humanos, aproximando-se de um *workflow* supervisionado.

O ganho de acesso traz um problema de segurança: um agente com permissões amplas pode ler arquivos de variáveis de ambiente, acessar dados sensíveis ou sair do diretório do projeto. [VERIFICAR: substituir a referência a material de curso por documentação pública da Anthropic.] Como o Seed trata dados de saúde mental de estudantes, essa preocupação é central e motiva o isolamento descrito na Subseção 3.12.4.5.

Neste trabalho, o orquestrador é a sessão principal do agente (Claude Code), que conduz o esclarecimento com o engenheiro e delega tarefas a subagentes. Subagentes possuem contexto próprio e são definidos em arquivos com *frontmatter* YAML delimitado por `---`: `name` identifica o subagente; `description` orienta a delegação; `model` seleciona o modelo; `tools` enumera as ferramentas; `skills` indica habilidades pré-carregadas; e `hooks` associa verificações ao ciclo de execução (**??**). O desenho não depende de subagentes que criem outros subagentes. [VERIFICAR: confirmar essa possibilidade na documentação.] O Seed define dois subagentes, `tester` e `dev`, detalhados na Subseção 3.12.6. As ferramentas e os formatos de definição de agentes variam entre fornecedores, e o Spec Kit é concebido para funcionar com vários agentes (GitHub, 2025), o que reduz a dependência de uma única ferramenta nas especificações, embora não nos arquivos de agente. [VERIFICAR: confirmar a lista de agentes compatíveis.]

3.12.4.2 Skills

Skills são pastas de instruções, descritas em arquivos `SKILL.md`, que o agente utiliza para tarefas específicas, como revisão de código, mensagens de *commit* ou geração de testes (**??**). O agente consulta primeiro a descrição de cada skill e carrega o conteúdo completo apenas quando a tarefa corresponde a ela, o que evita ocupar o contexto com instruções desnecessárias. [VERIFICAR: confirmar esse carregamento progressivo na documentação.] Por isso, a descrição deve informar o que a skill faz e o que provoca seu uso. Skills de

leitura e análise podem ter as ferramentas limitadas por meio do campo `allowed-tools`, reduzindo pedidos de permissão e o risco de alterações indevidas. Skills podem ainda ser associadas a subagentes por meio do campo `skills`, e a possibilidade de encadeá-las entre si deve ser verificada na documentação [VERIFICAR].

A Tabela 5 lista as skills propostas para o Seed.

Tabela 5 – Skills propostas para o processo de desenvolvimento.

Skill	**Usuário**	**Função**	**Situação**
`grill-me` (ou *clarify*)	Orquestrador	Conduzir perguntas ao engenheiro até fechar critérios de aceitação, limites numéricos e decisões técnicas, sem que o agente decida por conta própria.	Conteúdo a descrever [VERIFICAR]
`unit-tests-boundary`	`tester`	Gerar testes unitários (	Declarada

3.12.4.3 Hooks

Hooks são comandos definidos pelo usuário que o Claude Code executa em pontos fixos do ciclo de uso das ferramentas, sem depender de o modelo lembrar de uma instrução (**??**). O evento `PreToolUse` ocorre antes da chamada de uma ferramenta e permite recusá-la, e `PostToolUse` ocorre depois que ela é concluída. Um *hook* que termina com código de saída 2 bloqueia a ação, e o motivo retorna ao agente (**??**). Existem ainda eventos como `Stop`, `SubagentStop`, `UserPromptSubmit` e `PreCompact`, e o conjunto muda entre versões. [VERIFICAR: conferir a lista atual na página oficial.] O critério de uso é objetivo: o que precisa ocorrer sempre é implementado como *hook*, e não como pedido no *prompt*, o que dialoga com a degradação de instruções em contextos longos discutida na Subseção 3.12.1.

Hooks não invocam skills nem comandos de conversa. Portanto, o esclarecimento de requisitos é iniciado pelo orquestrador, por solicitação do engenheiro ou por regra no arquivo `CLAUDE.md`, que reúne o onboarding do projeto para o agente. Os *hooks* atuam como travas, e não como gatilhos. Um formatador, por exemplo, é executado depois que o agente edita um arquivo, e não antes de executar código.

No Seed, os *hooks* têm duas funções: limitar o acesso do agente a variáveis de ambiente e a conteúdo fora do diretório de código, e aplicar regras de formatação e limpeza a cada geração de código. A Tabela 6 apresenta o conjunto proposto.

Tabela 6 – Hooks propostos para o processo de desenvolvimento.

Evento	**Seletor**	**Função**	**Escopo**
PostToolUse	`Edit|Write`	Formatar o arquivo alterado conforme a linguagem e devolver ao agente os erros de *lint*. As ferramentas serão definidas [VERIFICAR].	Global
PreToolUse	`Read|Edit|Write|Bash`	Bloquear acesso a arquivos de variáveis de ambiente, chaves e credenciais, como segunda camada após o isolamento do contêiner.	Global
PreToolUse	`Bash`	Bloquear comandos destrutivos, como remoção recursiva, *push* forçado, *reset* rígido e `DROP`.	Global
PreToolUse	`Read|Grep|Glob|Write|Edit`	Lista de permissões (*allow-list*): o `tester` só acessa testes e especificações.	`tester`
PreToolUse	`Write|Edit`	Lista de proibições (*deny-list*): o `dev` não altera arquivos de teste.	`dev`

Evento	**Seletor**	**Função**	**Esco**
SubagentStop	—	Comparar os testes com o *commit* da fase vermelha, executar a suíte e recusar o encerramento se algum teste foi alterado ou a suíte estiver vermelha. A implementação permanece [VERIFICAR].	Glob (`set`
PostToolUse	`*`	Registrar ferramenta, caminho e horário, sem o conteúdo, que pode conter dados pessoais, para auditoria do processo.	Glob
PreToolUse (hipótese)	Chamada de subagente	Impedir o `tester` antes de a especificação ser esclarecida e o `dev` antes do *commit* dos testes vermelhos. Hipótese de projeto, sem fonte; o nome da ferramenta que invoca subagentes deve ser conferido [VERIFICAR].	Glob
PreCompact (opcional)	—	Salvar o estado da tarefa antes da compactação do contexto.	Glob

Fonte: autoria própria, com base em **??**).

3.12.4.4 Por que o Seed não utiliza MCP

O *Model Context Protocol* (MCP) conecta agentes a ferramentas e fontes de dados externas. O Seed não o adota por três razões de projeto: o desenvolvimento depende apenas das ferramentas nativas de leitura, busca, edição e execução e das skills descritas acima; cada servidor conectado amplia a superfície de acesso do agente, contrariando o princípio do privilégio mínimo (**??**); e as descrições das ferramentas de cada servidor ocupam espaço no contexto [VERIFICAR: confirmar esse custo na documentação]. Essas razões são de desenho, não resultado de avaliação empírica.

3.12.4.5 Isolamento do agente

O princípio do privilégio mínimo estabelece que cada componente deve acessar apenas o necessário para funcionar (**??**). Para o agente, a camada principal é a execução em um contêiner ou ambiente isolado, sem acesso a variáveis sensíveis do sistema nem a diretórios fora do projeto. A documentação do Docker descreve, por exemplo, ambientes isolados em microVMs para agentes de código (**??**). [VERIFICAR: citar a solução de isolamento de Akita, com repositório, e escolher a ferramenta efetivamente usada.] Os

hooks complementam esse isolamento, sem substituí-lo. Essa composição configura defesa em profundidade e integra os cuidados com os dados pessoais tratados pelo Seed no contexto da LGPD (BRASIL, 2018), sem constituir comprovação de conformidade legal. [VERIFICAR: conferir os artigos sobre dados pessoais sensíveis de saúde.]

3.12.5 Fluxo proposto

O fluxo de cada feature compreende as etapas a seguir, ilustradas na Figura **??**. Primeiro, o engenheiro descreve a feature e o orquestrador conduz o esclarecimento até fechar os critérios de aceitação e os limites numéricos. Em seguida, são geradas e revisadas a especificação, o planejamento e as tarefas. O `tester` produz os testes a partir da especificação, que são revisados, executados e registrados em um *commit* na fase vermelha. O `dev` implementa a fase verde e refatora, mantendo os testes verdes. Por fim, a integração contínua e as ferramentas de segurança verificam o resultado, e o orquestrador reporta ao engenheiro.

A verificação de segurança combina técnicas complementares: a análise estática (SAST) examina o código-fonte sem executá-lo; a análise dinâmica (DAST) testa a aplicação em execução; e a análise interativa (IAST) instrumenta a aplicação durante a execução para observar vulnerabilidades. [VERIFICAR: citar OWASP e o NIST SP 800-218 (SSDF) para essas definições e para a escolha das ferramentas.] A inclusão dessa etapa decorre das evidências de que código gerado por IA pode conter vulnerabilidades e de que o uso de assistentes aumenta a confiança indevida do desenvolvedor (**??**PERRY et al., 2023). Um *commit* que introduza vulnerabilidade crítica retorna à implementação.

3.12.6 Subagentes de teste e implementação

A divisão em dois subagentes busca a independência entre quem define o comportamento esperado e quem escreve o código. Seu fundamento é a observação de **??**) e **??**) de que ferramentas de geração de testes que inferem requisitos a partir do código tendem a validar defeitos. No estudo, as ferramentas avaliadas descartavam testes que falhavam no código com bugs e mantinham os que passavam, e em até 68,1% das suítes finais de uma das ferramentas os testes passavam no código incorreto e falhavam no correto. Os autores defendem que o código seja escrito a partir de requisitos expressos como testes, de acordo com o TDD. Em um estudo preliminar com funções simples em Python, **??**) obtiveram testes executáveis e com cobertura total usando o código da função como entrada, mas o escopo era restrito a três funções e os autores reconhecem limitações com dependências entre componentes. Diferentemente dessas abordagens, o `tester` do Seed parte da especificação.

3.12.6.1 Responsabilidades do subagente tester

A descrição de `tester` atribui-lhe a produção de testes unitários a partir da especificação e dos critérios de aceitação de uma tarefa, após o esclarecimento de requisitos, denominado *clarify*, e antes da implementação. Para operacionalizar essa atribuição, propõe-se que o orquestrador forneça a especificação da tarefa, os critérios de aceitação, os limites numéricos esclarecidos pelo engenheiro e o contrato de interfaces definido no planejamento, denominado *plan*. Os limites devem vir do esclarecimento, e não ser inventados pelo modelo, pois bordas plausíveis nem sempre são as do domínio do Seed. A saída prevista consiste nos arquivos de teste e em uma síntese de sua localização e das pendências, sem reproduzir integralmente o código na conversa. A regra de devolver ambiguidades ao orquestrador, sem decidir requisitos, integra o protocolo proposto; sua presença nas instruções completas do agente permanece [VERIFICAR].

O Código **??** apresenta o trecho que seleciona o modelo, as ferramentas e a habilidade. O conjunto declarado contém `Read`, `Write`, `Edit`, `Grep` e `Glob`; `Bash` está deliberadamente ausente, conforme o comentário do arquivo. A exclusão do interpretador de comandos é coerente com a intenção de limitar os meios de acesso disponíveis para essa função, segundo o princípio do privilégio mínimo de **??**).

O mesmo arquivo associa o evento `PreToolUse` às cinco ferramentas por meio do padrão `Read|Grep|Glob|Write|Edit` e referencia o script `tester-allowlist.sh`. O nome do script, isoladamente, não comprova quais caminhos são autorizados. A política pretendida permite ler testes e especificações e escrever os testes, excluindo o código de produção. A opção por uma lista de permissões explícitas, ou *allow-list*, decorre de permitir apenas recursos previamente identificados: uma lista de proibições, ou *deny-list*, deixaria depender da enumeração de todas as localizações proibidas, e as buscas sem caminho varrem o repositório inteiro. Essa justificativa aplica o princípio de negação por padrão de **??**); a implementação concreta, incluindo o tratamento de caminhos e buscas, permanece [VERIFICAR].

A habilidade declarada é `unit-tests-boundary`. A documentação do Claude Code apresenta habilidades como instruções definidas em arquivos `SKILL.md` (**??**). Nas anotações do autor, sua descrição prevê JUnit 5 ou Jest, partição de equivalência e valores de borda. A partição organiza entradas em classes consideradas equivalentes para o objetivo de teste (**??**); a análise de valores limite concentra casos nas fronteiras relevantes do domínio (**??**). Para o protocolo proposto, adota-se ainda a convenção AAA, com preparação do cenário (*Arrange*), execução da operação (*Act*) e verificação do resultado (*Assert*). Essa convenção é uma escolha de organização dos testes; sua inclusão no corpo da habilidade permanece [VERIFICAR]. Os frameworks mantêm a escolha já registrada na Seção 3.11, sem substituir a revisão dos resultados esperados pelo engenheiro.

3.12.6.2 Responsabilidades do subagente dev

A descrição de `dev` estabelece que sua execução ocorre depois da revisão e do registro dos testes vermelhos em um *commit*. Cabe-lhe implementar o mínimo necessário para satisfazê-los e, em seguida, refatorar. Propõe-se que receba a especificação, o planejamento, a tarefa e a referência do commit de testes, acompanhados do resultado da execução que caracterizou a fase vermelha. A habilidade `implement-and-refactor`, declarada no arquivo, é descrita nas anotações como orientada às camadas Controller, Service, Model e Repository. O Código **??** evidencia que, além das ferramentas de leitura, busca e edição, esse agente dispõe de `Bash`.

No arquivo disponível, o hook `PreToolUse` de `dev` seleciona somente `Write|Edit` e referencia `dev-no-test-edit.sh`. A política pretendida é permitir alterações no código de produção e impedir alterações nos testes por essas ferramentas. Entretanto, os padrões da lista de proibições não puderam ser examinados [VERIFICAR]. Como `Bash` não integra esse seletor, a configuração apresentada não demonstra bloqueio de modificações de testes por comandos de terminal. A preocupação tem precedente: no agente de remediação de **??**), a instrução proíbe expressamente a modificação dos testes, o que indica que a alteração de testes é um desvio a ser evitado, embora o estudo não meça sua frequência. Caso um teste pareça contradizer a especificação, propõe-se interromper a implementação e devolver a divergência ao orquestrador, que consulta o engenheiro; o agente não deve ajustar o teste unilateralmente. A presença dessa regra na habilidade completa permanece [VERIFICAR].

Propõe-se também limitar as tentativas de correção. No estudo de **??**), o ciclo de remediação foi limitado a cinco iterações ou três falhas repetidas, e os autores observaram retorno decrescente além de três ou quatro tentativas. Esses parâmetros servem de ponto de partida para o Seed, a ser calibrado no TCC2.

3.12.6.3 Encadeamento e pontos de controle

O fluxo proposto é sequencial: o engenheiro esclarece os requisitos, o orquestrador entrega os insumos ao `tester`, e os testes produzidos são revisados e executados antes da delegação ao `dev`. Como o `tester` não dispõe de `Bash`, atribui-se a execução e o registro do commit ao engenheiro ou ao orquestrador com as permissões correspondentes. A revisão deve distinguir a falha esperada por funcionalidade ausente de problemas do ambiente ou de testes inválidos. O commit dos testes vermelhos constitui a referência de integridade para a implementação. Somente depois desse ponto se iniciam a fase verde e a refatoração, com nova execução dos testes, seguindo o ciclo discutido na Subseção **??** (ANICHE, 2014). A fase verde e a refatoração não são paralelas: a refatoração só ocorre com os testes verdes. Não se prevê execução paralela dos dois agentes sobre a mesma tarefa.

A Tabela 7 sintetiza a divisão proposta e os campos comprovados pelas cópias disponíveis. A visibilidade do `tester` depende tanto do conteúdo encaminhado pelo orquestrador quanto da implementação da restrição de acesso: limitar ferramentas não retira informações já incluídas na entrada da tarefa.

Tabela 7 – Comparação entre os subagentes do fluxo proposto

Aspecto	**tester**	**dev**
Papel	Produzir testes na fase vermelha.	Implementar a fase verde e refatorar.
Entradas propostas	Especificação, critérios de aceitação, limites do clarify e interfaces do plan.	Especificação, plan, tarefa, testes revisados e commit da fase vermelha.
Ferramentas declaradas	Read, Write, Edit, Grep e Glob.	Read, Write, Edit, Grep, Glob e Bash.
Visibilidade pretendida	Testes e especificações, incluindo contratos de interfaces.	Testes, especificações e código de produção.
Restrições propostas	Não consultar produção nem resolver ambiguidades de requisitos.	Não editar testes nem decidir divergências entre testes e especificação.
Hooks declarados	PreToolUse sobre as cinco ferramentas; referência a tester-allowlist.sh.	PreToolUse sobre Write e Edit; referência a dev-no-test-edit.sh.
Verificação final	Não declarada na cópia examinada.	SubagentStop não declarado na cópia; configuração externa [VERIFICAR].
Habilidade declarada	`unit-tests-boundary`.	`implement-and-refactor`.
Modelo declarado	`sonnet`.	`sonnet`.

Fonte: autoria própria, com base nos arquivos reproduzidos no Apêndice **??** e no protocolo proposto nesta subseção.

3.12.6.4 Salvaguardas e limites dos mecanismos

A documentação de hooks informa que `PreToolUse` ocorre antes da chamada de ferramenta e permite recusá-la. Já `SubagentStop` ocorre ao término da resposta do subagente e pode bloquear seu encerramento; não desfaz operações anteriores (**??**). Em `settings.json`, esse evento pode selecionar o tipo de agente. No frontmatter de um subagente, a documentação apresenta `Stop`, convertido em `SubagentStop` durante a execução

(**??**). Essas possibilidades documentadas não comprovam a existência de uma auditoria de integridade no Seed.

Propõe-se verificar, no encerramento de `dev`, se os testes permanecem iguais à referência registrada na fase vermelha. Essa verificação permitiria detectar posteriormente alterações persistentes feitas por vias não interceptadas, incluindo `Bash`, mas não impediria sua ocorrência. Sua implementação e abrangência permanecem [VERIFICAR]. A comparação final também não demonstraria, por si só, a ausência de alterações transitórias restauradas antes da auditoria. Por isso, instruções textuais sobre papéis são complementadas, na proposta, por restrições de ferramentas, hooks e revisão humana. O rigor específico dos hooks constitui decisão de projeto a avaliar, não prática consolidada demonstrada pela literatura citada.

Essa composição é concebida como defesa em profundidade. No desenho proposto, o isolamento por contêiner deve constituir a camada principal de separação dos recursos do ambiente, com acesso limitado a arquivos, credenciais e rede; os hooks exercem função complementar. Não foi disponibilizada configuração que permita verificar esse isolamento [VERIFICAR]. Sua adoção deve integrar os cuidados com os dados pessoais tratados pelo Seed, no contexto da LGPD já abordado no capítulo (BRASIL, 2018), sem ser apresentada como comprovação de conformidade legal.

3.12.7 Limitações e ameaças à validade

As duas configurações utilizam o mesmo seletor de modelo, `sonnet`, conforme os Códigos **??** e **??**. Assim, a separação de tarefas não demonstra independência dos erros: admite-se, como ameaça à validade, que ambos reproduzam interpretações e vieses semelhantes. Além disso, o seletor não registra uma versão fixa do modelo. Propõe-se avaliar no TCC2 a escolha de modelo por agente, registrando a identificação efetiva utilizada e comparando a adequação dos testes e das implementações sob condições controladas. A atribuição de modelos mais leves a tarefas de teste não deve ser assumida como premissa, pois a literatura sugere que a qualidade dos testes é determinante para a qualidade do código resultante.

Outra limitação diz respeito à transferência de resultados acadêmicos. **??**) utilizaram testes escritos por humanos nos experimentos, enquanto, no protocolo aqui proposto, um subagente produz os testes. A própria dissertação afirma não responder quando e como a IA deve escrever testes (**??**). Portanto, os resultados desse estudo não podem ser extrapolados como evidência de benefício da dupla `tester`/`dev`. Os experimentos também se limitam a funções e programas em Python, a modelos específicos e a *benchmarks* conhecidos, com risco de vazamento de dados de treinamento, conforme reconhecido pelos autores, e não envolvem projetos em Java com Spring. Há ainda um risco para a revisão humana: o estudo com usuários de Fakhoury et al., relatado na dissertação, indica que

participantes tiveram dificuldade de distinguir testes errados sugeridos pelo sistema (**??**). A efetividade da revisão dos critérios e dos oráculos de teste pelo engenheiro permanece uma etapa metodológica proposta, cuja efetividade também deve ser avaliada.

Por fim, cobertura estrutural não prova qualidade nem atendimento integral dos requisitos: entradas relevantes podem permanecer ausentes mesmo quando o código é exercitado (**??**). Nos experimentos de **??**), por exemplo, o primeiro teste já atingia cobertura total em 75% dos problemas do HumanEval e 92,4% dos do MBPP. Propõe-se incluir teste de mutação na avaliação do TCC2, medindo a capacidade da suíte de detectar alterações artificiais no programa. **??**) encontraram associação entre detecção de mutantes e de defeitos reais, independentemente da cobertura, no conjunto estudado. Essa evidência fundamenta a métrica complementar, sem transformá-la em prova universal de qualidade.

3.12.8 Plano de avaliação do processo no TCC2

Para que as afirmações sobre qualidade sejam sustentadas por dados, propõe-se registrar, por feature, as seguintes medidas: cobertura de testes e índice de mutantes detectados, com os operadores empregados e o tratamento de mutantes equivalentes; vulnerabilidades identificadas pelas ferramentas de análise e sua gravidade; retrabalho, medido pelo número de ciclos de correção e de devoluções ao engenheiro; violações das restrições registradas pelos *hooks*; e ambiguidades devolvidas ao orquestrador. A comparação entre modelos por agente, citada na subseção anterior, deve usar as mesmas tarefas e condições. Os resultados serão apresentados no capítulo de resultados, junto da tabela de rastreabilidade entre histórias de usuário, requisitos, tarefas e testes, e a ausência de um grupo de controle e o tamanho reduzido da equipe devem ser declarados como limitações.

4 Considerações Finais

4.1 Conclusão

Ao longo deste trabalho, foi possível aplicar conceitos fundamentais da Engenharia de Software, incluindo a definição de requisitos, modelagem de sistemas e análise de metodologias de desenvolvimento. Conforme descrito no Capítulo 1, o objetivo principal do projeto é a criação de um aplicativo móvel voltado à promoção da saúde mental e hábitos saúdaveis no contexto universitário. No entanto, a arquitetura desenvolvida apresenta flexibilidade suficiente para ser adaptada a outras plataformas, como aplicações web ou desktop. Isso é possível devido à estrutura robusta e desacoplada do *backend*, que foi projetado para garantir escalabilidade e portabilidade.

Adicionalmente, a pesquisa bibliográfica realizada, aliada à orientação de uma coorientadora com formação em Psicologia, contribuiu significativamente para o aprofundamento do conhecimento sobre saúde mental. Essa parceria interdisciplinar foi essencial para compreender as necessidades do público-alvo e delinear funcionalidades que realmente dialoguem com os desafios enfrentados pelos estudantes universitários. Assim, o projeto não apenas se beneficia de uma base técnica sólida, como também de uma fundamentação teórica consistente na área da saúde.

4.2 Trabalhos Futuros

Diante dos resultados alcançados nesta primeira etapa, torna-se viável o desenvolvimento da segunda etapa (TCC2), na qual serão implementadas as funcionalidades definidas com base nos requisitos funcionais e não funcionais levantados nesta fase inicial.

Em etapas futuras, espera-se que o projeto evolua para a construção completa do aplicativo proposto, com a implementação das funcionalidades detalhadas no protótipo de alta fidelidade apresentado no Apêndice **??**. Além disso, é recomendável uma vasta gama de usuários, envolvendo diferentes perfis da comunidade universitária. Isso permitirá conduzir testes de usabilidade, desempenho e validação com maior rigor metodológico.

Outra possibilidade relevante consiste na integração do sistema com soluções já existentes na área da saúde digital. Um exemplo interessante seria a interoperabilidade com o aplicativo *Guardiões da Saúde*, utilizado pela Faculdade de Ciências da Saúde da UnB, que adota uma abordagem de vigilância participativa. A incorporação de funcionalidades complementares, como alertas de saúde, registros de hábitos e indicadores personalizados, poderia potencializar o alcance da proposta.

Referências

AKITA, F. *My Experience with Agile Vibe Coding*. 2026. Palestra apresentada no Tropical on Rails 2026, São Paulo. Vídeo publicado no canal Tropical on Rails (YouTube). Acesso em: 20 out. 2026. Disponível em: <https://www.youtube.com/watch?v=U3bZavG8qQY>. Citado na página 66.

ANICHE, M. *Test-Driven Development: Teste e Design no Mundo Real*. 1. ed. [S.l.]: Casa do Código, 2014. Citado 3 vezes nas páginas 41, 65 e 76.

Anthropic. *Claude 101*. 2026. Curso on-line, Anthropic Academy. Acesso em: 21 out. 2026. Disponível em: <https://anthropic.skilljar.com/claude-101>. Citado na página 67.

ARLINGHAUS, K. R.; JOHNSTON, C. A. The importance of creating habits and routine. *American journal of lifestyle medicine*, SAGE Publications Sage CA: Los Angeles, CA, v. 13, n. 2, p. 142–144, 2019. Citado na página 28.

BARROSO, S. M.; OLIVEIRA, N. R. D.; ANDRADE, V. S. D. Solidão e Depressão: Relações com Características Pessoais e Hábitos de Vida em Universitários. *Psicologia: Teoria e Pesquisa*, v. 35, p. e35427, 2019. ISSN 1806-3446, 0102-3772. Disponível em: <http://www.scielo.br/scielo.php?script=sci_arttext&pid=S0102-37722019000100527&tlng=pt>. Citado 2 vezes nas páginas 24 e 25.

BATISTA, J. C.; RIBEIRO, O. C. F.; JUNIOR, P. C. N. Lazer e promoção de saúde: uma aproximação conveniente. *LICERE-Revista do Programa de Pós-graduação Interdisciplinar em Estudos do Lazer*, v. 15, n. 2, 2012. Citado na página 28.

BECK, K. *Augmented Coding: Beyond the Vibes*. 2025. Newsletter *Software Design: Tidy First?* Publicado em 25 jun. 2025. Acesso em: 20 out. 2026. Disponível em: <https://newsletter.kentbeck.com/p/augmented-coding-beyond-the-vibes>. Citado na página 67.

BHAT, T.; NAGAPPAN, N. Evaluating the efficacy of test-driven development: industrial case studies. In: *Proceedings of the 2006 ACM/IEEE International Symposium on Empirical Software Engineering*. New York, NY, USA: Association for Computing Machinery, 2006. (ISESE '06), p. 356–363. ISBN 1595932186. Disponível em: <https://doi.org/10.1145/1159733.1159787>. Citado na página 41.

BÖCKELER, B. *Understanding Spec-Driven-Development: Kiro, spec-kit, and Tessl*. 2025. Martinfowler.com, série *Exploring Gen AI*. Publicado em 15 out. 2025. Acesso em: 20 out. 2026. Disponível em: <https://martinfowler.com/articles/exploring-gen-ai/sdd-3-tools.html>. Citado na página 68.

BODUCH, A. *React and react native*. [S.l.]: Packt Publishing Ltd, 2017. Citado na página 60.

BOLSONI-SILVA, A. T.; GUERRA, B. T. O impacto da depressão para as interações sociais de universitários. *Estudos e Pesquisas em Psicologia*, v. 14, n. 2, p. 429–452, set.

2014. ISSN 1808-4281. Disponível em: <http://www.e-publicacoes.uerj.br/index.php/revispsi/article/view/12649>. Citado na página 24.

BOORSE, C. Health as a theoretical concept. *Philosophy of science*, v. 44, p. 542–573, 1977. Disponível em: <https://www.scielosp.org/pdf/physis/2007.v17n1/29-41/pt>. Citado na página 27.

BRASIL. *Lei nº 13.709, de 14 de agosto de 2018*. 2018. <https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm>. Lei Geral de Proteção de Dados Pessoais (LGPD). Citado 3 vezes nas páginas 33, 74 e 78.

CAMPOS, F. F. de. *FURPS+*. 2008. Acesso em: 3 jul. 2025. Disponível em: <https://qualidadebr.wordpress.com/2008/07/10/furps/>. Citado na página 55.

CAROLI, P. *Lean Inception: Como alinhar pessoas e construir o produto certo*. São Paulo, BR: Caroli.org, 2017. Citado 13 vezes nas páginas 44, 45, 46, 57, 88, 89, 90, 92, 93, 94, 95, 96 e 97.

CAROLI, P. et al. *Lean Inception Workshop Template | Miro*. 2021. Disponível em: <https://miro.com/templates/lean-inception-workshop/>. Citado na página 45.

CARVALHO, G. V. de et al. Proposta de priorização de requisitos de software utilizando a matriz moscow e o método multicritério sapevo-m. In: *XLI Encontro Nacional de Engenharia de Produção (ENEGEP)*. Foz do Iguaçu, PR, Brasil: [s.n.], 2021. Anais do ENEGEP 2021. Disponível em: <https://www.abepro.org.br/biblioteca/TN_STO_356_1836_41806.pdf>. Citado na página 50.

COHN, M. *Succeeding with Agile: Software Development Using Scrum*. [S.l.]: Addison-Wesley, 2010. Citado na página 66.

CRONBACH, L. J. Coefficient alpha and the internal structure of tests. *UNIVERSITY OF ILLINOIS*, 1951. Citado 2 vezes nas páginas 33 e 34.

DAMASCENO, D. D. L.; PIMENTEL, A. M. A PROMOÇÃO DA SAÚDE NO ENSINO SUPERIOR E O MOVIMENTO DE UNIVERSIDADES PROMOTORAS DA SAÚDE: CONCEITOS, CONSTRUÇÃO E DESAFIOS. In: *Educação: pesquisa, aplicação e novas tendências*. 1. ed. Editora Científica Digital, 2022. p. 285–308. ISBN 978-65-5360-100-0. Disponível em: <http://www.editoracientifica.com.br/articles/code/211106692>. Citado 2 vezes nas páginas 23 e 25.

DAYLIO. 2021. Acesso em: 1 jul. 2025. Disponível em: <https://daylio.net/>. Citado na página 30.

DRAW.IO. *Draw.io Documentation*. 2025. Acesso em: 1 jul. 2025. Disponível em: <https://www.drawio.com/doc/>. Citado na página 57.

ELMASRI, R.; NAVATHE, S. B. *Fundamentals of database systems seventh edition*. [S.l.]: pearson, 2016. Citado 2 vezes nas páginas 63 e 64.

FIGMA. *Figma Learn*. 2025. Acesso em: 1 jul. 2025. Disponível em: <https://help.figma.com/hc/pt-br>. Citado na página 57.

GE, Y. et al. *A Survey of Vibe Coding with Large Language Models*. 2025. ArXiv preprint arXiv:2510.12399. Acesso em: 20 out. 2026. Disponível em: <https://arxiv.org/abs/2510.12399>. Citado na página 67.

GITHUB. *About Git*. 2025. Acesso em: 27 jun. 2025. Disponível em: <https://docs.github.com/en/get-started/using-git/about-git>. Citado na página 57.

GitHub. *Spec Kit: toolkit for Spec-Driven Development*. 2025. Repositório de código (GitHub). Acesso em: 20 out. 2026. Disponível em: <https://github.com/github/spec-kit>. Citado 2 vezes nas páginas 68 e 69.

GUEDES, G. T. A. *UML 2 - uma abordagem prática*. [S.l.]: Novatec Editora, 2018. Citado 4 vezes nas páginas 60, 61, 62 e 63.

HABITICA. 2025. Acesso em: 1 jul. 2025. Disponível em: <https://habitica.com/>. Citado na página 29.

HEIDMANN, I. T. B. et al. Promoção à saúde: trajetória histórica de suas concepções. *Texto & Contexto - Enfermagem*, v. 15, n. 2, p. 352–358, jun. 2006. ISSN 0104-0707. Disponível em: <http://www.scielo.br/scielo.php?script=sci_arttext&pid=S0104-07072006000200021&lng=pt&tlng=pt>. Citado na página 23.

HELLIWELL, J. F. et al. World happiness report 2025. Wellbeing Research Centre, University of Oxford, 2025. Citado na página 28.

HEUSER, C. A. *Projeto de banco de dados*. [S.l.]: Bookman Porto Alegre, 2009. v. 6. Citado na página 64.

HIGHSMITH, J. Extreme programming. *Cutter Consortium*, 2000. Citado 2 vezes nas páginas 38 e 40.

JANZEN, D. S. Software Architecture Improvement through Test-Driven Development. *University of Kansas, Electrical Engineering and Computer Science - Lawrence, Kansas USA*, 2005. Citado na página 41.

JOSHI, A. et al. Likert scale: Explored and explained. *British Journal of Applied Science & Technology*, 2015. Citado na página 33.

KENT, B.; ANDRES, C. *Extreme programming explained: embrace change*. 2. ed. [S.l.]: addison-wesley professional, 2004. Citado 3 vezes nas páginas 38, 39 e 40.

LALONDE, M. A new perspective on the health of canadians: a working document (1974). *Ottawa: Ministry of Supply and Services Canada*, 1981. Citado 2 vezes nas páginas 27 e 28.

LEAL-NETO, O. B. et al. Detecção digital de doenças e vigilância participativa: panorama e perspectivas para o brasil. *Revista de Saúde Pública*, p. 17, 2016. Citado na página 24.

MAXIMILIEN, E.; WILLIAMS, L. Assessing test-driven development at ibm. In: *25th International Conference on Software Engineering, 2003. Proceedings.* [S.l.: s.n.], 2003. p. 564–569. Citado na página 41.

MICHELS, D. L. et al. *Vibe Coding: Practice, Performance, Productivity, and Risk – A State-of-the-Art Review*. 2026. ArXiv preprint arXiv:2608.20446. Acesso em: 20 out. 2026. Disponível em: <https://arxiv.org/abs/2608.20446>. Citado na página 67.

MUNIZ, G. d. B. A.; GARRIDO, E. N. Mudanças de hábitos e saúde dos estudantes após ingresso na universidade. *Revista Psicologia, Diversidade e Saúde*, v. 10, n. 2, p. 235–245, 2021. Citado na página 28.

NASCIMENTO, A. E. *OAuth 2.0: Proteja suas aplicações com o Spring Security OAuth2*. [S.l.]: Editora Casa do Código, 2017. Citado na página 59.

NASCIMENTO, F. P. d.; SOUSA, F. Classificação da pesquisa. natureza, método ou abordagem metodológica, objetivos e procedimentos. *Metodologia da Pesquisa Científica: teoria e prática–como elaborar TCC. Brasília: Thesaurus*, 2016. Citado 2 vezes nas páginas 34 e 35.

NIELSEN, J. Enhancing the explanatory power of usability heuristics. *ACM Press*, p. 152–158, 1994. Citado na página 32.

OLIVEIRA, E. d. S. L. d. Acesso e permanência de estudantes cotistas na universidade de brasília: estratégias para democratização da educação superior. 2020. Citado na página 29.

PAGOTTO, T. et al. Scrum solo: Software process for individual development. In: *2016 11th Iberian Conference on Information Systems and Technologies (CISTI)*. [S.l.: s.n.], 2016. p. 1–6. Citado na página 43.

PERRY, N. et al. Do users write more insecure code with AI assistants? In: *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*. Copenhagen, Denmark: Association for Computing Machinery, 2023. Disponível em: <https://doi.org/10.1145/3576915.3623157>. Citado 2 vezes nas páginas 67 e 74.

QUARTILHO, M. J. *Saúde mental*. [S.l.]: Imprensa da Universidade de Coimbra/Coimbra University Press, 2010. Citado na página 28.

Rede Brasileira de Universidades Promotoras de Saúde. *Regimento Interno da Rede Brasileira de Universidades Promotoras de Saúde (ReBraUPS)*. 2022. Aprovado em Assembleia Geral da ReBraUPS em 11 de agosto de 2022. Citado na página 23.

REINEHR, S. *Engenharia de Requisitos*. Porto Alegre: SAGAH, 2020. Citado 2 vezes nas páginas 49 e 50.

RHODES, J. L. Loneliness: How Superficial Relationships, Identity Gaps, and Social Support Contribute to Feelings of Loneliness at Pepperdine University. 2014. Disponível em: <https://digitalcommons.pepperdine.edu/cgi/viewcontent.cgi?article=1006&context=pjcr>. Citado na página 24.

SAPKOTA, R.; ROUMELIOTIS, K. I.; KARKEE, M. *Vibe Coding vs. Agentic Coding: Fundamentals and Practical Implications of Agentic AI*. 2025. ArXiv preprint arXiv:2505.19443. Acesso em: 20 out. 2026. Disponível em: <https://arxiv.org/abs/2505.19443>. Citado na página 67.

SCHLEICH, A. L. R. *INTEGRAÇÃO NA EDUCAÇÃO SUPERIOR E SATISFAÇÃO ACADÊMICA DE ESTUDANTES INGRESSANTES E CONCLUINTES*. Dissertação (Mestrado) — Universidade Estadual de Campinas, 2006. Disponível em: <http://www.bdae.org.br/bitstream/123456789/1130/1/tese.pdf>. Citado na página 23.

SCHUELLER, S. M. et al. The use of mobile apps to promote mental health: A cross-sectional survey of mental health app use among university students. *Journal of Medical Internet Research*, v. 21, n. 6, p. e13537, 2019. Citado 2 vezes nas páginas 23 e 24.

SCHWABER, K.; SUTHERLAND, J. *O Guia do Scrum: Um guia definitivo para o Scrum: As regras do jogo*. scrumguides.org, 2013. Disponível em: <https://scrumguides.org/docs/scrumguide/v1/Scrum-Guide-Portuguese-BR.pdf>. Citado 3 vezes nas páginas 38, 42 e 43.

SCLIAR, M. História do conceito de saúde. *Physis: Revista de saúde coletiva*, v. 17, p. 29–41, 2007. Disponível em: <https://www.scielosp.org/pdf/physis/2007.v17n1/29-41/pt>. Citado na página 27.

SEQUEIRA, C. S. L. *Especificação rigorosa de requisitos de usabilidade para software*. Dissertação (Dissertação de Mestrado) — Instituto Superior Técnico – Universidade de Lisboa, Lisboa, Portugal, nov 2017. Orientador: Prof. Alberto Manuel Rodrigues da Silva. Citado na página 55.

SILVA, F. A. da et al. Saúde mental: a universidade desafiada a agir para garantir a permanência estudantil num contexto de adoecimento. *Revista de Gestão e Secretariado*, v. 16, n. 1, p. e4513–e4513, 2025. Citado na página 29.

SILVEIRA, C. et al. Saúde mental em estudantes do ensino superior: experiência da consulta de psiquiatria do centro Hospitalar São João. *Acta Médica Portuguesa*, v. 24, p. 247–256, 2011. ISSN 1646-0758. Disponível em: <https://www.actamedicaportuguesa.com/revista/index.php/amp/article/view/1504>. Citado na página 24.

SILVEIRA, C. R.; FLÔR, R. d. C.; MACHADO, R. R. Metodologia da pesquisa. *Florianópolis: IFSC*, 2011. Citado 2 vezes nas páginas 34 e 35.

SIQUEIRA, M. M. M. Construção e validação da escala de percepção de suporte social. v. 13, n. 2, p. 381–388, 2008. ISSN 1413-7372. Disponível em: <http://www.scielo.br/scielo.php?script=sci_arttext&pid=S1413-73722008000200021&lng=pt&nrm=iso&tlng=pt>. Citado 2 vezes nas páginas 24 e 25.

SKARIN, M. *Real-World Kanban Do Less, Accomplish More with Lean Thinking*. [S.l.]: The Pragmatic Bookshelf, 2015. Citado na página 44.

SOMMERVILLE, I. *Software Engineering*. [S.l.]: Pearson, 2011. (International Computer Science Series). ISBN 9780137035151. Citado 3 vezes nas páginas 50, 55 e 56.

SOUZA, D. C. Condições emocionais de estudantes universitários: estresse, depressão, ansiedade, solidão e suporte social. 2017. Disponível em: <https://bdtd.uftm.edu.br/bitstream/tede/507/5/Dissert%20Deise%20C%20Souza.pdf>. Citado na página 24.

VAZQUEZ, C. E.; SIMÕES, G. S. *Engenharia de Requisitos: software orientado ao negócio*. [S.l.]: Brasport, 2016. Citado na página 46.

WEISSMANN, H. L. *Vire o jogo com Spring Framework*. [S.l.]: Editora Casa do Código, 2014. Citado 4 vezes nas páginas 57, 58, 59 e 60.

ZANELLA, L. C. H. et al. *Metodologia da pesquisa*. [S.l.]: SEAD/UFSC Florianópolis, 2006. Citado na página 34.

Apêndices

APÊNDICE A – Artefatos da Lean Inception

Este apêndice apresenta os artefatos complementares produzidos durante a aplicação da metodologia Lean Inception, cuja visão geral e principais resultados foram apresentados na Seção 3.5.6.

A.1 Visão de Produto

A visão de produto é um artefato cujo objetivo principal é estabelecer, de forma clara e concisa, a proposta central do produto a ser desenvolvido. Ela descreve o propósito do produto, o público-alvo e os principais diferenciais em relação ao mercado (CAROLI, 2017). A partir dessa definição, a elaboração deste artefato proporcionou o seguinte resultado:

- **Para**: Estudantes e profissionais em meio universitário.
- **Cujo**: Os estudantes carecem de assistência à saúde e os profissionais das IEs precisam monitorar seus alunos.
- **O**: Seed.
- **É um**: *Progressive Web Apps* (PWA).
- **Que**: Auxilia os estudantes a encontrarem apoio nas IES.
- **Diferentemente do**: Daylio ou outros aplicativos que não atuam para as IES.
- **O nosso produto**: Auxilia os indivíduos dentro do meio universitário em questões de saúde mental ao coletar dados e oferecer informações condizentes, além de fomentar a prática de hábitos saudáveis.

A.2 É - Não É - Faz - Não Faz

Essa atividade tem como objetivo refinar a ideia inicial do produto, com foco na definição do MVP e na eliminação de funcionalidades excedentes ou desnecessárias. Neste momento, é fundamental realizar questionamentos técnicos, como aspectos de portabilidade, funcionalidades, entre outros (CAROLI, 2017). Após essa etapa, foi configurado da seguinte maneira para o projeto em questão:

- **É**: PWA, gratuito, Ferramenta de analise de dados, *Open source*.
- **Não é**: chat com profissionais de saúde, rede social.
- **Faz**: relatórios sobre o bem-estar do estudante, formulários para auxiliar profissionais, feedbacks personalizáveis, fornecer pontos de ajuda nas IES.
- **Não faz**: não expõe dados individuais, não propõe diagnósticos, não agenda consultas

A.3 Objetivos do Negócio

O objetivo de negócio tem como finalidade esclarecer de forma conjunta o propósito do produto. Nesta etapa, os participantes devem compartilhar o entendimento sobre os resultados esperados do produto (CAROLI, 2017). Ao final da atividade, os objetivos são consolidados e agrupados, para o projeto os principais objetivos foram:

- **Gerenciamento de Dados**:
  - Anotações livres;
  - Exibição em gráficos e *dashboards*;
  - Análise de confiabilidade dos dados;
  - Correlação de dados de formulários;
  - Formulários diários sobre seções do bem-estar;
  - Formulários personalizados.
- **Seções do bem-estar**:
  - Seções sobre sono, hidratação, social, alimentação, exercício físico, saúde mental, autoconhecimento, lazer, acadêmico/trabalho, financeiro.
  - Garantir acessibilidade, funcionalidade identificada a partir da pesquisa realizada com estudantes.
- **Soluções de auxílio**:
  - Disponibilização de eventos da IES do usuário;
  - *Feed* (mural) de informações;
  - Análise estatística das informações públicas dos usuários.
- **Notificações**:
  - Alerta para preencher formulário;
  - Alerta para eventos.

A.4 Personas

Após a definição do produto, torna-se essencial compreender profundamente o perfil dos usuários que irão interagir com ele. Para isso, são elaboradas personas, sendo essas representações fictícias, porém realistas, de usuários típicos, baseadas em comportamentos, necessidades e objetivos reais (CAROLI, 2017). Para o projeto proposto, foram criados três perfis de usuários.

A.4.1 Primeira Persona

- **Perfil**:
  - Nome: José.
  - Idade: 23 anos.
  - Profissão: Estudante universitário.
  - Hábitos: Possui hábitos de vida pouco saudáveis.
- **Objetivos**:
  - Melhorar o rendimento e o foco na vida acadêmica;
  - Adotar práticas mais saudáveis no dia a dia;
  - Desenvolver maior autoconsciência e bem-estar.
- **Frustrações**:
  - Baixo desempenho acadêmico;
  - Má experiência com aplicativos de difícil usabilidade;
  - Sensação de sobrecarga e desorganização na rotina.
- **Necessidades**:
  - Sugestões práticas e direcionadas para melhorar aspectos específicos da rotina;
  - Visibilidade clara de suas dificuldades e pontos a desenvolver;
  - Lembretes diários que incentivem pequenas ações de melhoria;
  - Ferramenta para registrar livremente suas reflexões e anotações pessoais.

A.4.2 Segunda Persona

- **Perfil**:
  - Nome: Marina.

  - Idade: 32 anos.
  - Profissão: Psicopedagogo.
  - Hábitos: Hábitos saudáveis.

- **Objetivos**:
  - Conseguir acompanhar os estudantes por uma ferramenta de análise de dados;
  - Realizar análises que contribuam para identificar problemas e orientar ações na instituição.

- **Frustrações**:
  - Dificuldade em monitorar os estudantes;
  - Dados em pesquisas anteriores não confiáveis ou incompletos;
  - Baixa qualidade ou ausência de ferramentas eficazes para análise e coleta.

- **Necessidades**:
  - Ferramenta capaz de identificar carências e gargalos institucionais com clareza;
  - Apoio na coleta e organização de informações confiáveis para os projetos.

A.4.3 Terceira Persona

- **Perfil**:
  - Nome: Ana Maria
  - Idade: 20 anos
  - Profissão: Estudante de Engenharia de Energia
  - Hábitos: Hábitos não saudáveis, com problemas psicológicos.

- **Objetivos**:
  - Encontrar meios de socialização e participar de eventos acadêmicos e sociais;
  - Equilibrar sua vida acadêmica com atividades de bem-estar;
  - Sentir-se mais conectada com a comunidade universitária.

- **Frustrações**:
  - Baixo desempenho na dimensão social do bem-estar;
  - Dificuldade em encontrar espaços e oportunidades de socialização;
  - Falta de ferramentas intuitivas que ajudem na integração com o ambiente universitário.

- **Necessidades**:
  - Calendário de eventos sociais e acadêmicos centralizado e de fácil acesso;
  - Aplicativo com boa usabilidade, fácil de usar e com aprendizado rápido;
  - Garantia de privacidade e conformidade com a LGPD (Lei Geral de Proteção de Dados), pois tem medo de seus dados vazarem.
  - Ferramenta que auxilie na organização de atividades voltadas ao bem-estar;

A.5 Jornadas dos Usuários

A jornada do usuário representa o percurso que um usuário realiza para atingir um objetivo específico ao interagir com o produto. O principal objetivo desse artefato é mapear e documentar os possíveis pontos de contato entre o usuário e o sistema, identificando suas ações, expectativas e eventuais dificuldades ao longo do caminho (CAROLI, 2017). A seguir, são apresentados os artefatos resultantes da aplicação dessas jornadas às personas do projeto:

- **Primeira Persona (José)**:
  1. Acorda cansado após uma noite mal dormida;
  2. Toma café com pressa antes da aula;
  3. Chega à universidade com dificuldade para se concentrar;
  4. Acessa o aplicativo de bem-estar no intervalo entre as aulas;
  5. Visualiza um painel com indicadores de saúde e desempenho acadêmico;
  6. Percebe que está com baixa pontuação em “hábitos de sono” e “produtividade”;
  7. Recebe sugestões práticas para melhorar a rotina diária (ex.: dormir mais cedo, usar técnica Pomodoro);
  8. Define metas semanais para pequenas melhorias;
  9. Ativa lembretes personalizados para ajudar a manter a rotina;
  10. Ao fim da semana, acessa um relatório com seu progresso e ajustes sugeridos;
  11. Sente-se motivado a continuar usando a ferramenta.
- **Segunda Persona (Marina)**:
  1. Inicia o expediente pela manhã em sua sala de atendimento psicopedagógico;
  2. Abre o aplicativo de acompanhamento do bem-estar dos estudantes;
  3. Verifica os alertas de estudantes com preenchimento pendente ou irregular nos formulários;

4. Acessa o painel de *dashboards* para visualizar o panorama geral das dimensões do bem-estar;
5. Filtra os dados por curso, período e dimensão (sono, saúde mental, acadêmico, entre outros);
6. Identifica padrões e possíveis riscos a partir da análise de confiabilidade dos dados;
7. Correlaciona informações de diferentes formulários para aprofundar a investigação de um caso específico;
8. Registra anotações livres sobre observações e encaminhamentos necessários;
9. Gera um relatório estatístico para apresentar à coordenação da instituição;
10. Compartilha as informações relevantes com a equipe multidisciplinar responsável pelo acompanhamento;
11. Finaliza a sessão com os dados e anotações salvos automaticamente na nuvem.

- **Terceira Persona (Ana Maria)**:

1. Acorda e toma café assistindo vídeos de humor para relaxar;
2. Abre o aplicativo no celular para ver os eventos do dia;
3. Explora o calendário e encontra um evento cultural no campus;
4. Verifica se há vagas disponíveis e se inscreve;
5. Confere o nível do seu bem-estar social no painel do aplicativo;
6. Recebe uma recomendação de evento com base em seu perfil e interesse por arte;
7. Aproveita para registrar uma anotação pessoal sobre sua experiência anterior;
8. Vai ao evento;
9. Se sente mais conectada e motivada a participar de novos eventos.

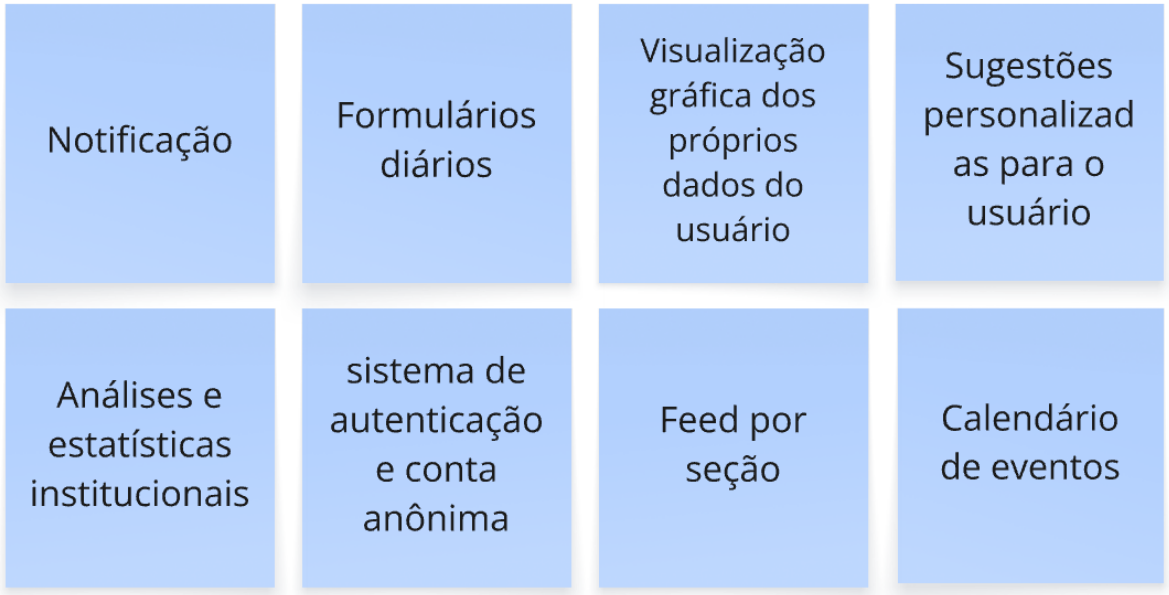
A.6 *Brainstorming* de Funcionalidades

Após as etapas anteriores, nas quais são obtidos os objetivos do produto e as necessidades dos usuários. *Brainstorming* de Funcionalidades atua como uma maneira de fornecer funcionalidades que atendam às necessidades das personas criadas (CAROLI, 2017).

Durante a sessão, são feitas perguntas como: “Quais funcionalidades são essenciais para que esta persona atinja seus objetivos?”, percorrendo cada persona individualmente. Ao final, todas as sugestões são consolidadas, formando um panorama inicial

das funcionalidades desejadas para o produto (CAROLI, 2017). A Figura 18 representa o *Brainstorming* de funcionalidades elaborado para o projeto.

Figura 18 – *Brainstorming* de Funcionalidades.



Fonte: autoria própria.

A.7 Revisão (Técnica, de Negócio e Experiência do Usuário)

A revisão técnica, de negócio e de experiência do usuário (UX) tem como objetivo avaliar as funcionalidades levantadas durante o *brainstorming*, classificando-as com base em três critérios principais: valor de negócio, esforço técnico e impacto na experiência do usuário (CAROLI, 2017).

Figura 19 – Classificação de funcionalidades - Lean Inception.



Fonte: Caroli (2017).

Em seguida, com base na classificação das funcionalidades, é utilizado o Gráfico do Semáforo, cujo objetivo principal é avaliar o nível de confiança da equipe de desenvol-

vimento em relação às tarefas propostas. Esse gráfico auxilia na análise da clareza sobre o que deve ser feito versus como deve ser feito (CAROLI, 2017).

Por meio desta etapa, as funcionalidades levantadas durante o *Brainstorming* de Funcionalidades foram avaliadas quanto ao seu valor de negócio, esforço técnico e impacto na experiência do usuário (UX). A Figura 20 apresenta o resultado da revisão realizada para o projeto.

Figura 20 – Revisão das funcionalidades propostas (Seed).

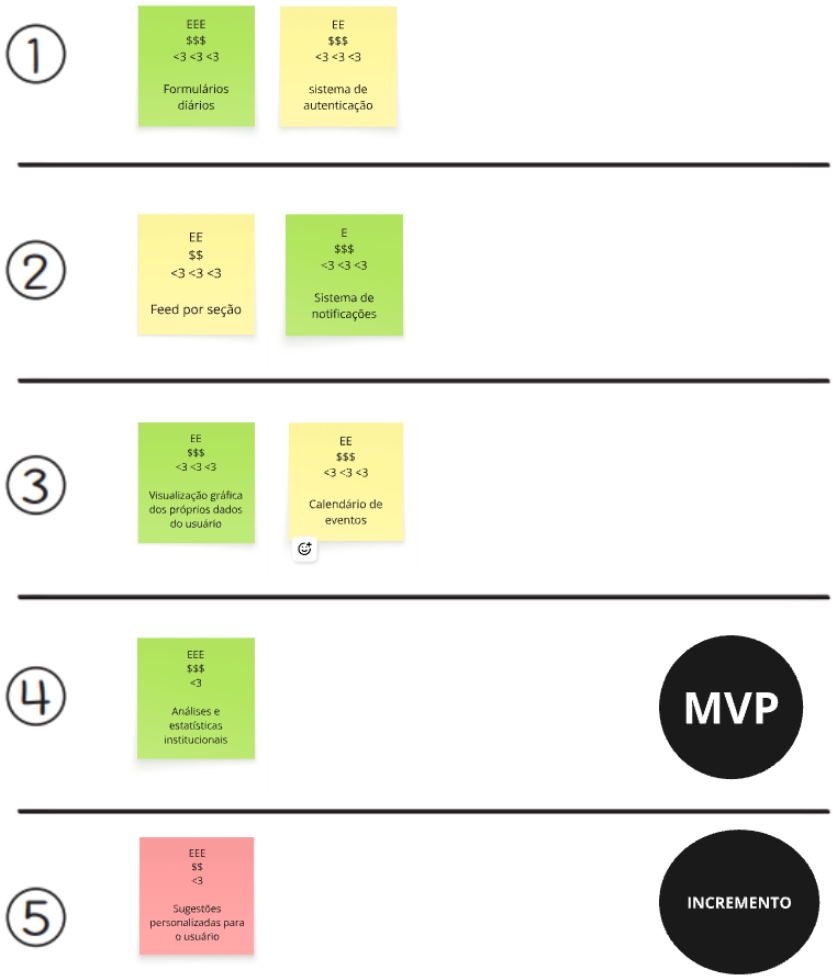


Fonte: autoria própria com base em Caroli (2017).

A.8 Sequenciador

O sequenciador tem como objetivo organizar as funcionalidades em ondas, de forma a distribuir o trabalho progressivamente e evitar a sobrecarga da equipe de desenvolvimento (CAROLI, 2017). A Figura 21 apresenta o sequenciador elaborado para o projeto.

Figura 21 – Sequenciador (Seed).



Fonte: autoria própria com base em Caroli (2017).

A.9 Canvas MVP

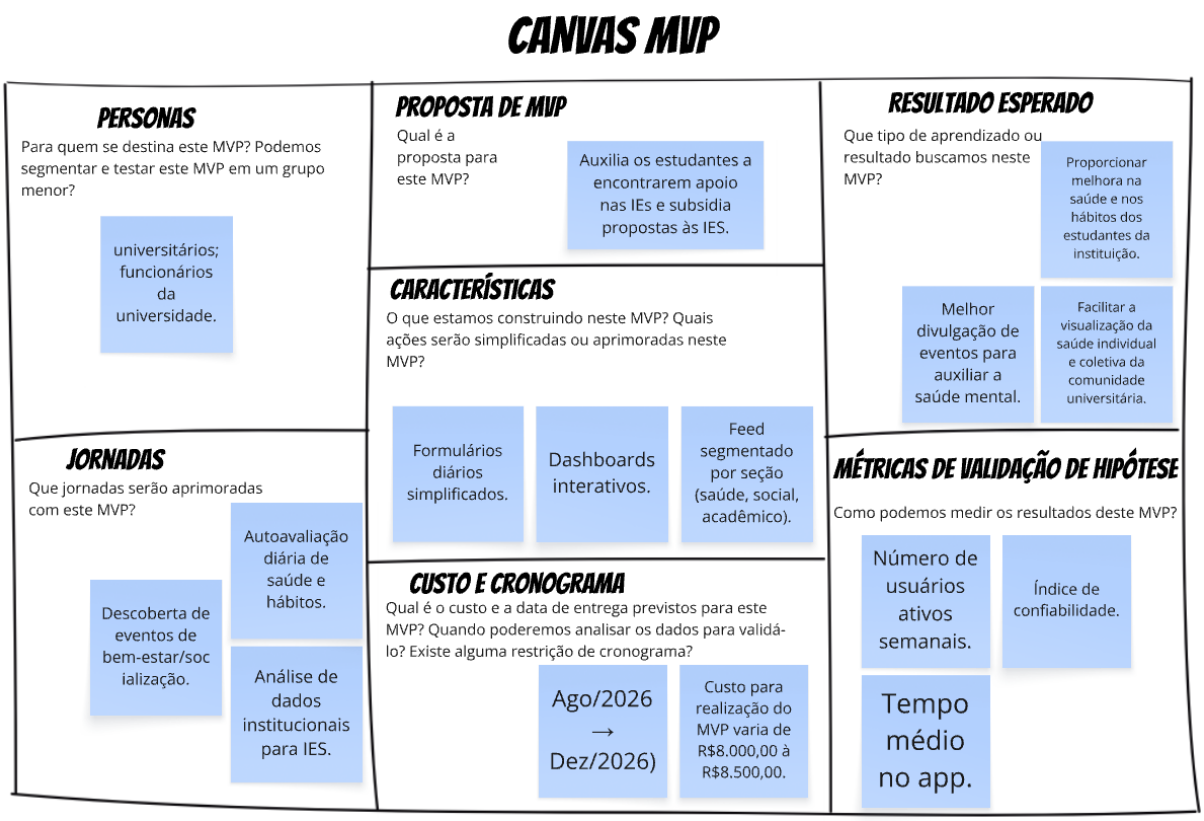
Ao final da Lean Inception, é elaborado o Canvas MVP, que consolida as principais definições do produto mínimo viável em sete blocos estruturados (CAROLI, 2017). Esses blocos são:

1. Proposta do MVP;
2. Personas Segmentadas;
3. Jornadas;
4. Funcionalidades;
5. Resultado esperado;
6. Métricas para validas as hipóteses de négocio;

7. Custos e cronograma.

A Figura 22 representa o produto final gerado a partir da aplicação da metodologia Lean Inception, reunindo as informações essenciais para a construção e validação do MVP.

Figura 22 – Canvas MVP (Seed).

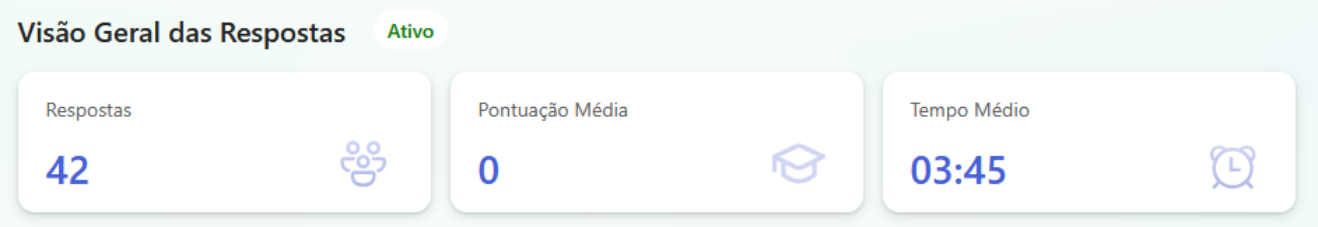


Fonte: autoria própria com base em Caroli (2017).

APÊNDICE B – Questionário

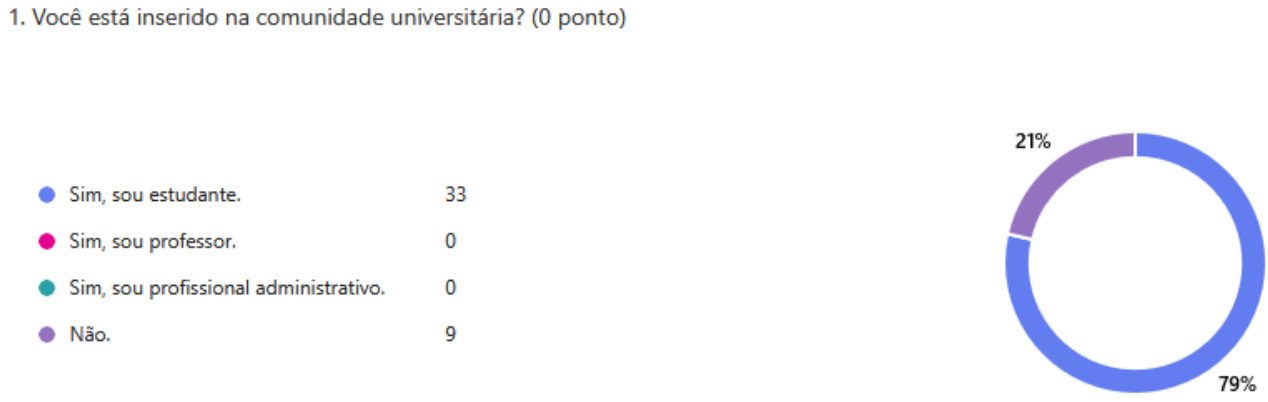
As perguntas do questionário com suas respectivas respostas do questionário elaborado durante a etapa de elicitação de requisitos podem ser vistos nas Figuras 23 a 36. O formulário pode ser acessado neste link.

Figura 23 – Número de Respostas e Tempo Médio.



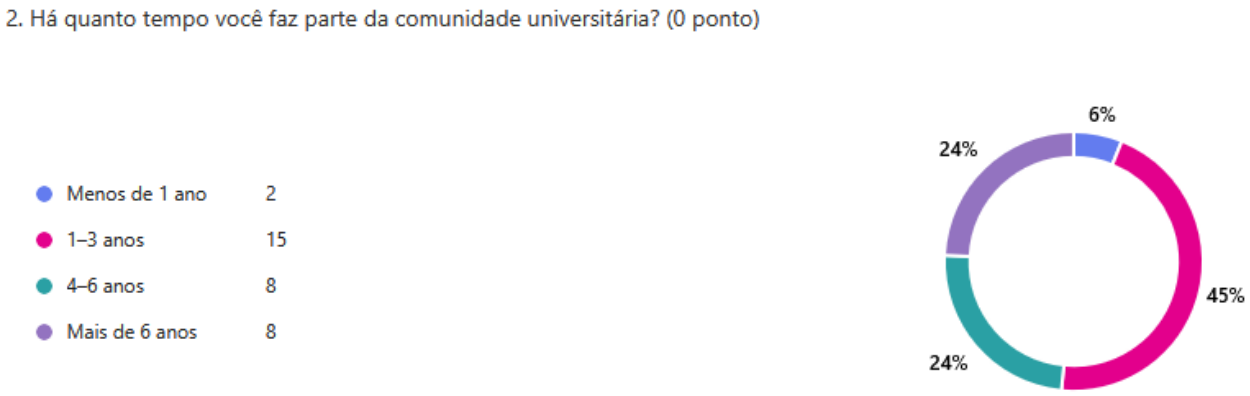
Fonte: autoria própria.

Figura 24 – Questão de Público Alvo.



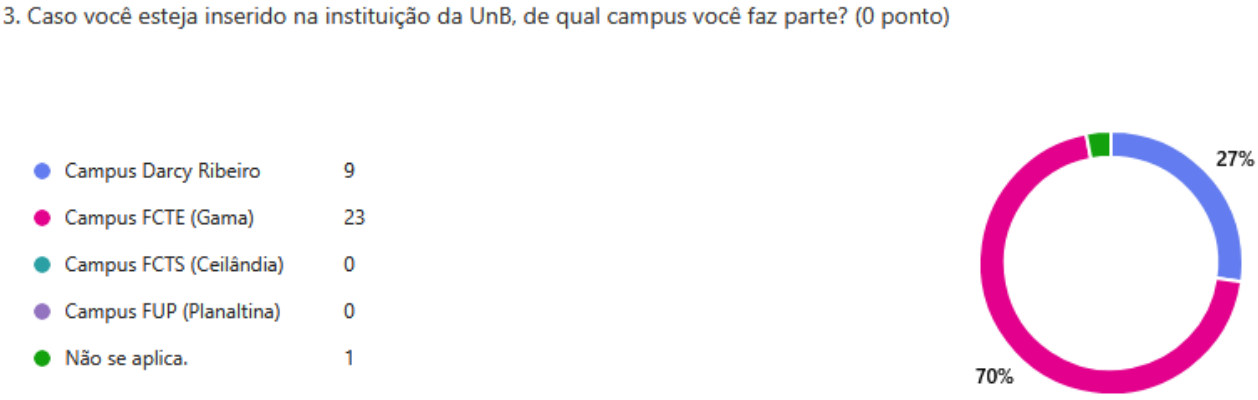
Fonte: autoria própria.

Figura 25 – Questão de permanência na comunidade universitária.



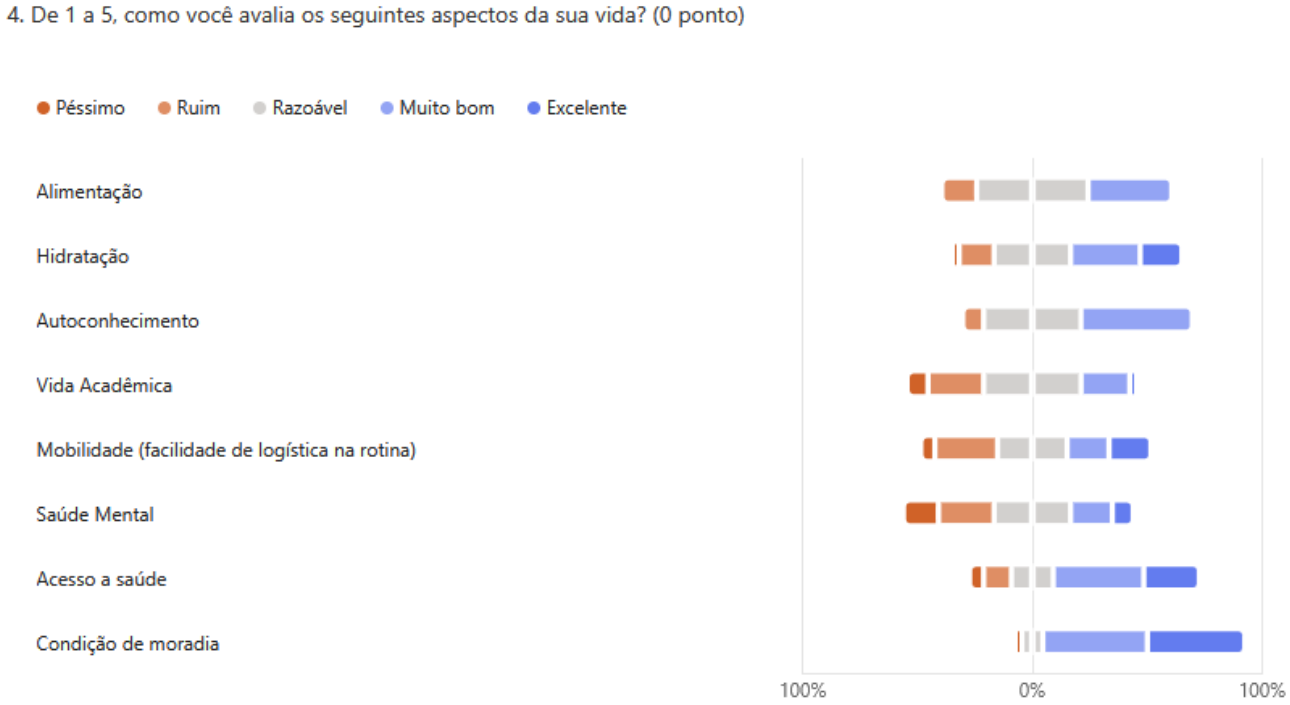
Fonte: autoria própria.

Figura 26 – Questão de Campus universitário.



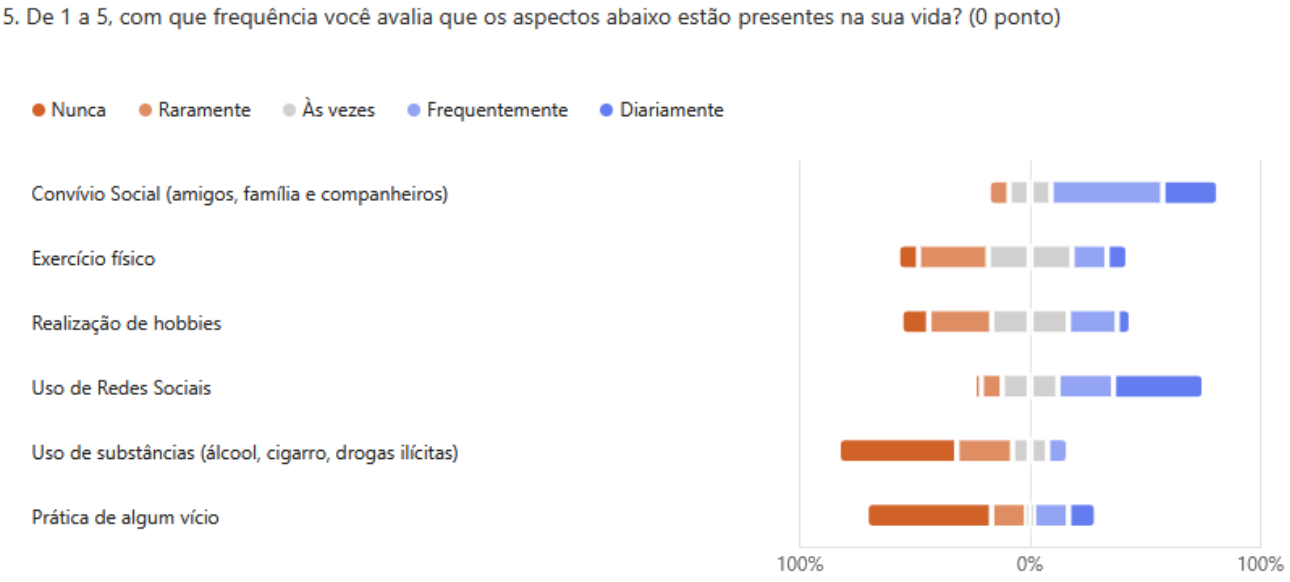
Fonte: autoria própria.

Figura 27 – Questões de Autoavaliação de hábitos.



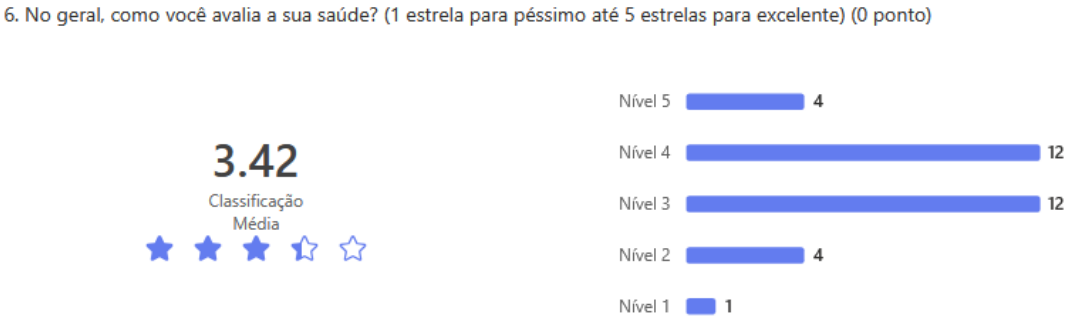
Fonte: autoria própria.

Figura 28 – Questões de Autoavaliação de frequência de hábitos.



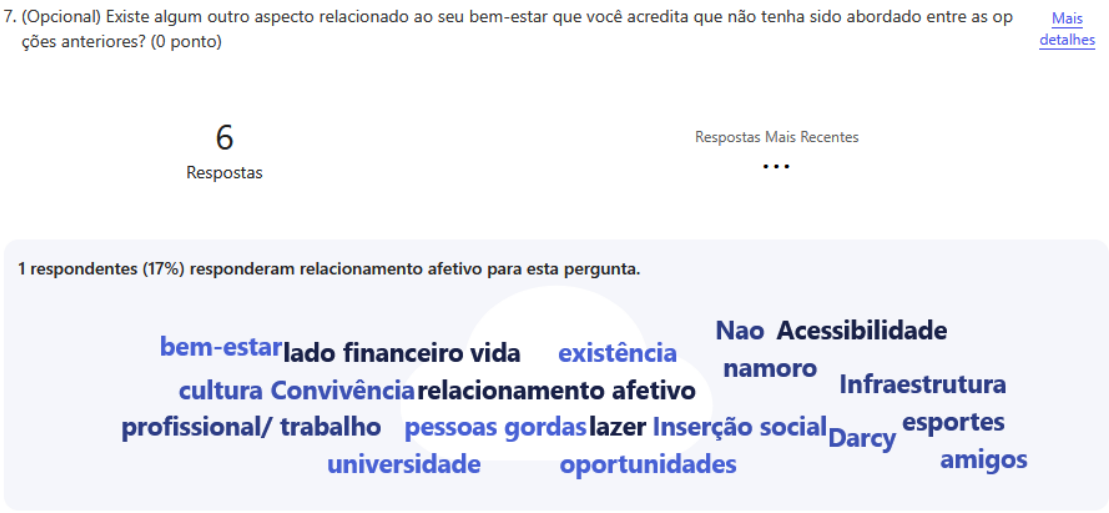
Fonte: autoria própria.

Figura 29 – Questões de Autoavaliação da saúde.



Fonte: autoria própria.

Figura 30 – Questões de demais tópicos do bem-estar.



Fonte: autoria própria.

Figura 31 – Respostas de demais tópicos do bem-estar.

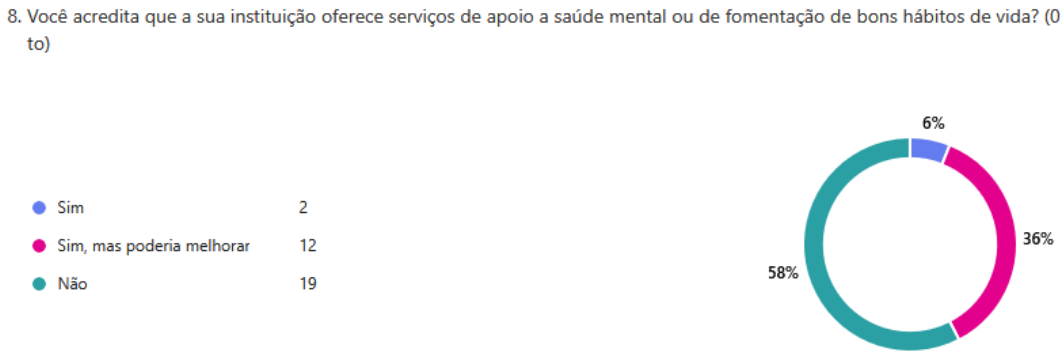
7. (Opcional) Existe algum outro aspecto relacionado ao seu bem-estar que você acredita que não tenha sido abordado entre as opções anteriores?

6 Respostas

ID ↑	Nome	Respostas
1	anonymous	Acessibilidade! Infraestrutura do Darcy não é apropriada para pessoas gordas.
2	anonymous	Acredito que o lado financeiro influencia no meu bem-estar
3	anonymous	Inserção social (profissional/ trabalho, lazer, esportes, cultura etc.) e/ou oportunidades de vida.
4	anonymous	Nao
5	anonymous	Convivência ou existência de amigos na universidade
6	anonymous	relacionamento afetivo (namoro)

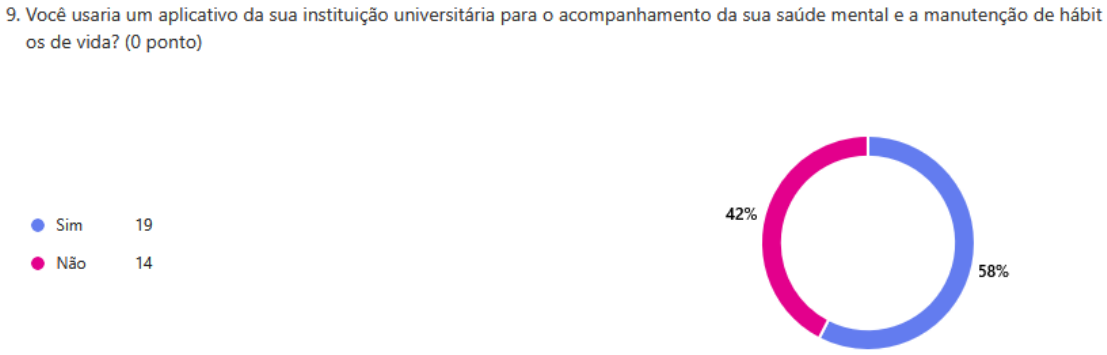
Fonte: autoria própria.

Figura 32 – Questão de serviços de apoio a saúde nas instituições.



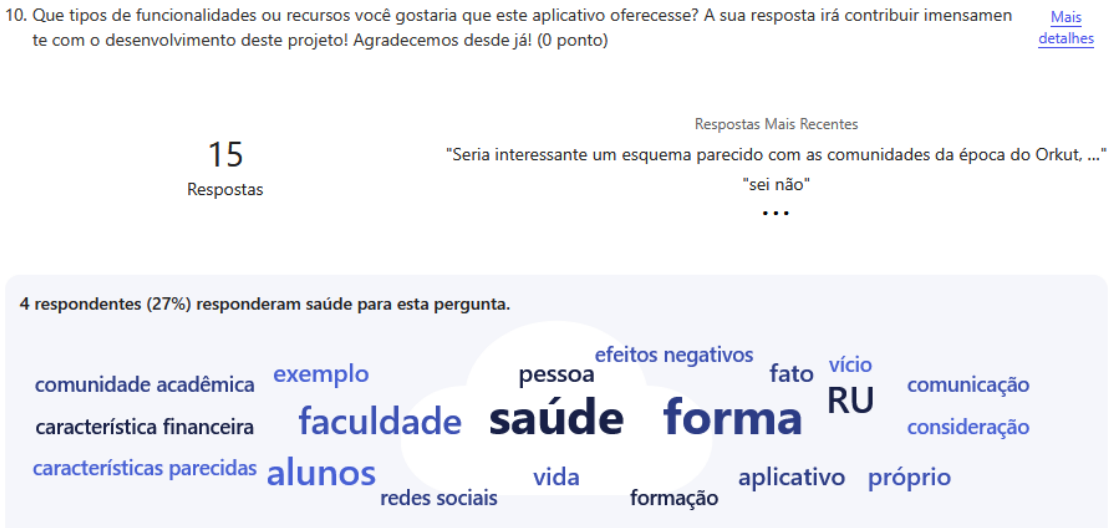
Fonte: autoria própria.

Figura 33 – Questão de aceitação de sistemas com foco em saúde.



Fonte: autoria própria.

Figura 34 – Questão sobre funcionalidades do sistema.



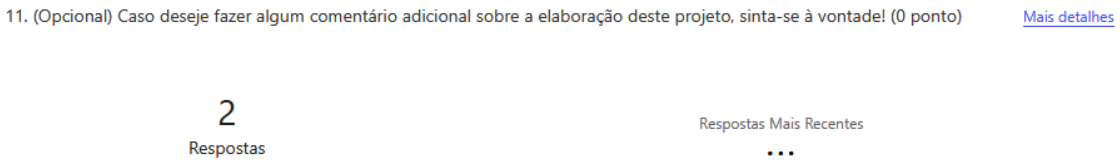
Fonte: autoria própria.

Figura 35 – Respostas sobre funcionalidades do sistema.

ID ↑	Nome	Respostas
1	anonymous	eventos acadêmicos na faculdade.
2	anonymous	Trackeamento de hábitos, relatórios mensais de progresso, gamificação
3	anonymous	Funcionalidade que ajudasse a identificar momentos de crise baseado em reconhecimento de emoções (com inputs do próprio usuário) classificando certos estados mentais de 1 a 5, por exemplo e sugerir rotas de melhora, seja física, médica ou emocional.
4	anonymous	Ouvidoria que de fato chegasse à administração da faculdade. Mural com projetos de saúde mental em atividade na faculdade.
5	anonymous	Acompanhamento real que pudesse ajudar a saúde dos alunos de maneira coletiva
6	anonymous	Recurso de gestão de calorias (principalmente do RU), dados sobre hidratação, um calendário para organização mental...
7	anonymous	Integração com plataformas externas e sistema de metas
8	anonymous	um campo para desabafa de forma anônima
9	anonymous	Locais onde se aplicam meu seguro de saúde
10	anonymous	Apoio a novos alunos e a alunos estrangeiros
11	anonymous	Agendamento de terapias
12	anonymous	Não sei se ajudaria de alguma coisa pra ser sincera, considerando que um dos principais problemas da faculdade é a forma como muitos docentes lidam com os alunos, onde 90% ou mais acham que todos os alunos possuem estrutura e dinheiro pra se manter na faculdade e somente estudar, muitas vezes dificultando mais ainda a vida de alunos que estudam e trabalham, o que não condiz com a realidade de boa parte dos alunos especialmente da FCTE, que não oferece moradia estudantil para estudantes de fora, uma bolsa auxílio moradia que não condiz com a realidade dos aluguéis do gama, além de serem muitas vezes cruéis com os alunos. Casos de assédio sexual e/ou moral por exemplo são comuns e nunca tratados, seja por medo (devido a visão do professor como autoridade e uma pessoa influente para a carreira), seja pela própria universidade não levar a sério de verdade a saúde dos alunos e bem estar deles favorecendo sempre a opinião dos professores, ou até mesmo por eles próprios estarem em uma posição que decide sobre esses casos. Portanto não acredito que um aplicativo seria de fato efetivo nisso.
13	anonymous	Ferramenta que indicasse o tempo de uso da tela do celular
14	anonymous	sei não
15	anonymous	Seria interessante um esquema parecido com as comunidades da época do Orkut, a ideia seria conseguir agrupar pessoas com características parecidas e, daí, conseguir lidar com cada grupo. Isso, além de facilitar o processo de melhoria, fortaleceria a comunidade acadêmica. Outro ponto, seria nessas comunidades uma liberdade maior de comunicação, Posts e afins. Porém, há uma consideração, existe o perigo de formação de certas bolhas sociais, o que é um efeito comum em mecanismos encontrados emr redes sociais, de qualquer modo, como a premissa do aplicativo é educacional e de fortalecimento da comunidade, dificilmente parta para algo do tipo, já que esses efeitos negativos são mais fortemente vistos pela característica financeira encontrada em redes sociais (já que o vício e formação de bolhas é lucrativo).

Fonte: autoria própria.

Figura 36 – Comentários adicionais.



Fonte: autoria própria.

Figura 37 – Respostas nos comentários adicionais.

11. (Opcional) Caso deseje fazer algum comentário adicional sobre a elaboração deste projeto, sinta-se à vontade!

2 Respostas

ID ↑	Nome	Respostas
1	anonymous	acho que o aplicativo deveria ser completamente anônimo (sem possibilidade de ranking, de troca de pontos, etc. Acho que ser minimamente público pode desestimular o uso (por vergonha).
2	anonymous	Sugiro algum tipo de avaliação voltado para os alunos do projeto 60+. Por vezes a convivência é a aceitação desse público não tem sido como deveria ser. Pude observar que ainda há muito preconceito contra alunos do projeto.

Fonte: autoria própria.